

# Introduzione Embedded Systems

## Introduzione ai Sistemi Embedded

L'evoluzione dei sistemi informatici ha portato allo sviluppo degli embedded system come risposta alla necessità di dispositivi più piccoli, economici, efficienti e specifici per determinate applicazioni. La loro diffusione ha rivoluzionato numerosi settori, creando nuove opportunità per l'innovazione e l'automazione.

Linux è diventato il sistema operativo scelto per un gran numero di applicazioni integrate seguendo questo approccio, come router Internet, sistemi di navigazione satellitare GPS, dispositivi di archiviazione collegati in rete, ecc....

La prospettiva quindi di avere una realtà processor-centrica (tale per cui il software fosse alla base dell'elaborazione delle informazioni) e una miniaturizzazione dei circuiti integrati, ha portato alla nascita dei cosiddetti **sistemi embedded**.

Le ragioni che hanno condotto allo sviluppo di tali sistemi sono molteplici, tra cui troviamo:

1. **La crescente complessità funzionale:** La necessità di aggiungere funzionalità avanzate ai dispositivi integrati ha richiesto l'integrazione di grandi quantità di software. Questo è stato reso possibile dall'aumento esponenziale della densità dei semiconduttori, descritto dalla **legge di Moore, che afferma che il numero di transistor in un circuito integrato raddoppia approssimativamente ogni due anni.** Assieme alla legge del ritorno accelerato, ciò ha permesso lo sviluppo di dispositivi sempre più potenti e complessi.
2. **Efficienza e miniaturizzazione:** la combinazione HW/SW ottimizzata nei dispositivi embedded consente di ridurre le dimensioni fisiche e i costi di produzione. Questo è un passo fondamentale per la realizzazione di dispositivi compatti a basso consumo

## Sistemi Embedded

**Si definisce sistema embedded (o sistema integrato) un sistema di elaborazione delle informazioni incorporato in un prodotto di maggiori dimensioni;**

Il problema tecnico legato ai processi fisici di un sistema integrato è quello della gestione di un tempo di concorrenza e di computazione di sistema.

### ▼ CYBER PHISICAL SYSTEMS

Si introduce così il concetto di "**cyber-physical systems**" (sistemi informatico-fisici) o CPS che sono integrazioni di calcolo mediante processi fisici.

I sistemi CPS si riferiscono a sistemi ICT (Information and Communication Technologies) integrati di nuova generazione che sono interconnessi attraverso l'Internet of Things (IoT) che permette loro quindi di collaborare; si parla in questo contesto di "Industria 4.0".

Un CPS differisce da un tradizionale sistema di controllo digitale principalmente

nella sua struttura: anziché avere un controllore che riceve un riferimento e confronta l'errore con il segnale di

retroazione abbiamo un cyber che rappresenta la parte computazionale che è più avanzata e include oltre al controllo

funzioni di calcolo avanzate, comunicazione e analisi dati.

Contrariamente ai computer generici riprogrammabili, un sistema embedded ha dei compiti noti già durante lo sviluppo, che eseguirà dunque grazie ad una combinazione hardware/software studiata per la tale applicazione. **Grazie a ciò l'hardware può essere ridotto ai minimi termini per contenerne lo spazio occupato limitando così anche i consumi, i tempi di elaborazione (maggiore efficienza) ed il costo di fabbricazione.** Inoltre l'esecuzione del software è spesso in tempo reale per permettere un controllo deterministico dei tempi di esecuzione.

**In sostanza, i sistemi embedded sono sistemi di calcolo, comprendenti ogni tipo di calcolatore al di fuori di quelli progettati per essere di utilità generica.**

Più in generale, rispetto un processore general purpose:

1. **Scopo e utilizzo:** un sistema embedded svolge compiti specifici all'interno di un sistema più grande, mentre un processore viene progettato per una vasta gamma di compiti.

2. **Hardware:** un sistema embedded è progettato per essere altamente efficiente dal punto di vista energetico (spesso alimentato a batteria) ed ha risorse limitate in termini di memoria (RAM e ROM) e capacità di elaborazione.  
Un processore general purpose tende a consumare più energia e ha accesso a risorse più abbondanti.
3. **Architettura e design:** un sistema embedded sfrutta architetture specifiche ottimizzate per un compito, e fa uso di interfacce come GPIO, ADC, DAC, I2C, UART, SPI. Un general purpose invece utilizza architetture più versatili (es: x86) e supporta una vasta gamma di periferiche come USB, HDMI, PCIe etc.
4. **Software:** nei sistemi embedded si esegue software dedicato;

## Applicazioni Sistemi Embedded

Le applicazioni che fanno uso di sistemi embedded sono molteplici e corrispondenti a diverse aree ad esempio:

### 1. Automazione di fabbrica

Al fine di ottimizzare ulteriormente le tecnologie di produzione, è possibile utilizzare la tecnologia CPS/IoT. La tecnologia CPS/IoT è la chiave per una produzione più flessibile favorendo il raggiungimento dell'obiettivo per l'industria 4.0.

### 2. Robotica

La robotica è anche un'area tradizionale in cui sono stati utilizzati sistemi embedded CPS.

### 3. Trasporto e mobilità

Elettronica in ambito automotive (le auto di oggi contengono una quantità significativa di elettronica);

### 4. Smart City (città intelligenti)

Le smart city si riferiscono a strategie di pianificazione urbanistica che migliorano la qualità di vita in città, e cercano di soddisfare le esigenze ed i bisogni dei cittadini.

# Introduzione a Unix

## Cos'è UNIX

E' un **Sistema Operativo: un software che astrae l'hardware**. Esso è scritto in C, è "**machine Independent**" e non dipende dall'hardware in cui è installato.

(

*Un **sistema operativo** è un software che gestisce le risorse hardware di un computer e fornisce un'interfaccia tra l'utente e la macchina. Coordina l'esecuzione dei programmi, gestisce la memoria, i file, i dispositivi di input/output e assicura che le diverse applicazioni possano funzionare correttamente. Inoltre, funge da intermediario tra l'hardware e i programmi applicativi, permettendo agli utenti di interagire con il computer in modo semplice ed efficiente.)*

Unix non è monolitico perché, pur avendo un kernel centrale, si basa su una **filosofia modulare**. In un sistema monolitico, tutte le funzionalità sono strettamente integrate nel kernel. Al contrario, Unix segue l'approccio "fai una cosa e falla bene", in cui diverse componenti (comandi, utility, shell) sono indipendenti e comunicano con il kernel, permettendo una maggiore flessibilità e semplicità nella manutenzione e sviluppo del sistema.

## Struttura UNIX

### Kernel:

- Il nucleo (bios) è il primo strato di software che mette a disposizione le "handle" per parlare con l'hardware, e risiede in una memoria non volatile.

### Shell:

- Racchiude i processi principali: La **shell** è un'interfaccia a riga di comando (CLI) che permette agli utenti di interagire con il sistema Unix. **Interpreta i comandi dell'utente e li invia al kernel per l'esecuzione**. Esistono diverse shell, come Bash, C shell, Korn shell, ecc. **La shell permette anche l'esecuzione di script per automatizzare compiti ripetitivi.**

### Utility/Comandi:

- Strumenti e programmi preinstallati che svolgono operazioni specifiche (es. gestione file, editor di testo).

### Servizi Esterni:

- Software Applicativi, File System (La struttura in cui sono memorizzati e organizzati i file), DBMS (Database Management System) è un software che consente di creare, gestire e manipolare database)

## Perchè usare Unix

### Caratteristiche:

- Multi User- Multi Tasking
- Tutto estinto in **file di diversa tipologia**;
- Facile da programmare a livello di sistema;
- Gratis;

### Multi - user

- Ogni utente ha un suo "ambiente", chiamato account, caratterizzato da un username ed una password. Ogni utente, tramite il suo account può usufruire delle risorse del computer. La **shell** in tutto ciò funge da interfaccia tra user e sistema (finché è nell'account);

### Multi - Tasking

- Il **multitasking** in Unix consente agli utenti di eseguire più comandi o programmi contemporaneamente. Ciò significa che, invece di attendere il completamento di un'operazione prima di eseguirne un'altra, gli utenti possono gestire più processi in parallelo.

### Esecuzione dei comandi in primo piano e in background:

- **Primo piano:** Quando un comando viene eseguito semplicemente digitandolo nella shell (es. `<comando>` ), questo viene eseguito in **primo piano**. Ciò significa che la shell è "bloccata" fino al completamento del comando, e l'utente non può eseguire altri comandi nel frattempo.
- **Background:** Se l'utente vuole continuare a usare la shell mentre un comando è in esecuzione, può lanciare il comando in **background** aggiungendo un simbolo "&" alla fine del comando (es. `<comando>&` ). In

questo modo, il comando continua a essere eseguito, ma la shell rimane libera per accettare nuovi comandi.

### Controllo dei processi in esecuzione:

- **Ctrl-Z**: Se un comando in esecuzione in primo piano deve essere temporaneamente fermato, si può utilizzare la combinazione di tasti **Ctrl-Z**. Questo mette in pausa (sospende) il processo.
- **bg**: Una volta sospeso, il processo può essere spostato in **background** utilizzando il comando `bg`, permettendo all'utente di continuare a interagire con la shell mentre il comando sospeso riprende l'esecuzione in background.
- **fg**: Se si desidera riportare un processo in background nuovamente in **primo piano**, è possibile utilizzare il comando `fg`, che riporta il controllo alla shell e impedisce di eseguire altri comandi fino al completamento del processo.

---

## UNIX FileSystem

### UNIX Filesystem Tree

Il **UNIX Filesystem Tree** è la struttura gerarchica di file e directory utilizzata dai sistemi Unix e Unix-like (come Linux). Questa struttura è organizzata in modo ad albero, con una singola directory radice (root), rappresentata dal simbolo `/`, da cui discendono tutte le altre directory e file.

#### 1. Gerarchia ad albero:

La struttura inizia dalla directory radice

`/` e da lì si ramifica in altre directory e sottodirectory. Ogni directory può contenere file o altre directory.

#### 2. Directory principali:

- **/**: La root directory è il punto di partenza del filesystem. Tutto nel sistema Unix è organizzato sotto questa directory.
- **/bin**: Contiene i file binari eseguibili essenziali per il sistema, come comandi di base del SO;
- **/etc**: Ospita i file di configurazione del sistema (es. configurazioni di rete, password di sistema).

- **/sbin:** Contiene eseguibili per comandi di sistema e amministrazione, spesso utilizzati da amministratori di sistema per compiti critici, come il ripristino del sistema o la gestione di servizi. Questi comandi sono simili a quelli in **/bin**, ma sono pensati principalmente per utenti con privilegi amministrativi;
- **/usr:** Usato per memorizzare software e file di sistema che non sono essenziali per l'avvio (come i programmi installati dagli utenti e le librerie aggiuntive):
  - **/usr/bin:** Contiene i **programmi eseguibili** utilizzati dagli utenti e dai sistemi, ma che non sono essenziali per l'avvio del sistema.
  - **/usr/include:** Ospita i file di intestazione (header files) per il linguaggio di programmazione C e altri linguaggi, necessari per compilare programmi. Questi file contengono definizioni di funzioni, strutture e macro che permettono di utilizzare librerie esterne durante la compilazione.
  - **/usr/lib:** Contiene le librerie condivise utilizzate dai programmi presenti in **/usr/bin**. Le librerie sono collezioni di codice che i programmi possono riutilizzare, come librerie di collegamento dinamico (.so);
  - **/usr/local:** Usato per installare software compilato e installato dall'utente che non è fornito dal sistema operativo stesso. Il software qui presente è spesso personalizzato o gestito manualmente dall'amministratore e non interferisce con i pacchetti di sistema;
  - **/usr/share:** Memorizza **file di dati non eseguibili** che possono essere condivisi tra applicazioni;
- **/var:** Memorizza file variabili come log di sistema, file temporanei e spool di stampa;
- **/dev:** Directory che contiene i file di dispositivi (device files), rappresentazioni dei dispositivi hardware del sistema (es. dischi rigidi, terminali).
- **/home:** Contiene le directory personali degli utenti. Ogni utente ha una sottodirectory dentro **/home** dove sono memorizzati i propri file personali.

- **/lib**: Contiene le librerie di sistema necessarie per eseguire i programmi nei binari di sistema:
  - Definizione i **Libreria**: è un insieme di funzioni, classi e procedure predefinite che possono essere utilizzate dai programmatori per facilitare lo sviluppo di applicazioni software. Possono essere:
    - **Librerie statiche**: Collegate al programma al momento della compilazione, diventando parte del file eseguibile finale;
    - **Librerie dinamiche**: Caricate in memoria durante l'esecuzione del programma. Permettono un uso più efficiente della memoria e facilitano l'aggiornamento del software senza dover ricompilare;
- **/mnt**: Directory utilizzata per **montare temporaneamente altri filesystem**, come unità USB o partizioni di dischi. Qui possono essere montati file system esterni per l'accesso temporaneo.
- **/opt**: Usata per installare **software aggiuntivo di terze parti**. Le applicazioni installate qui non fanno parte del sistema di base e vengono gestite separatamente;
- **/proc**: Una directory **virtuale** che rappresenta le informazioni sul sistema e sui processi in esecuzione;
- **/root**: La **directory home** dell'utente **root** (l'amministratore di sistema). Contrariamente a **/home**, che contiene le directory personali degli utenti standard, **/root** è riservata all'amministratore.

## UNIX File Pathnames

In Unix, i **pathnames** (nomi di percorso) sono utilizzati per identificare la posizione di un file o di una directory all'interno del filesystem. Esistono due tipi principali di pathnames: **assoluti** e **relativi**.

### 1. Pathnames Assoluti

Un **pathname assoluto** è una sequenza di nomi di directory separati da barre (/) che indica il percorso completo per raggiungere un file o una directory, partendo dalla **directory radice** (root) del filesystem, rappresentata dal simbolo **/**.

- **Esempio**: Se un file **f1** risiede nella posizione **/d1/d2/d3**, può essere raggiunto direttamente utilizzando il suo pathname assoluto:

`/d1/d2/d3/f1`

I pathnames assoluti sono indipendenti dalla posizione attuale dell'utente nel filesystem, pertanto, possono essere utilizzati da qualsiasi punto della struttura di directory.

## 2. Pathnames Relativi

Un **pathname relativo** specifica il percorso di un file o di una directory rispetto alla directory attuale dell'utente.

- Per facilitare la navigazione, Unix utilizza i seguenti simboli:
  - `./`: rappresenta la **directory corrente**.
  - `../`: rappresenta la **directory madre** (la directory superiore rispetto a quella corrente).
- **Esempio**: Se l'utente si trova nella directory `/d11/d12/d13` e desidera accedere al file `f1` memorizzato in `/d21/d22/d23`, può farlo utilizzando un pathname relativo: `../../d21/d22/d23/f1`. Qui, `../..` indica di risalire due directory dalla posizione corrente per raggiungere la directory in cui si trova `f1`.
- Inoltre, la shell Bash definisce anche `~/` come la **directory home** dell'utente corrente. Questo consente di accedere facilmente ai file e alle directory all'interno della propria home.

## UNIX File Properties

In Unix, i file hanno diverse **proprietà** che forniscono informazioni importanti sul loro stato e sulle loro caratteristiche. Per visualizzare queste proprietà, si utilizza il comando `ls`, che elenca i file presenti in una directory.

### Comando `ls`

- `ls`: Questo comando mostra l'elenco dei file e delle directory nella directory corrente.
- `ls -l`: Utilizzando l'opzione `l` (long format), il comando fornisce un elenco dettagliato delle proprietà di ciascun file e directory. Le informazioni visualizzate includono:
  - **Tipo di file**: Indica se è un file normale(-), una directory(d), un link simbolico(l), ecc.

- **Permessi:** Mostra i permessi di lettura (r), scrittura(w) ed esecuzione(x) per il proprietario del file-il gruppo-gli altri utenti. Es: *r-x: solo lettura ed esecuzione*
- **Numero di collegamenti:** Indica quanti link puntano a quel file.
- **Proprietario:** Mostra il nome dell'utente che possiede il file.
- **Gruppo:** Mostra il gruppo di utenti a cui il file appartiene.
- **Dimensione:** Indica la dimensione del file in byte.
- **Data e ora:** Mostra la data e l'ora dell'ultima modifica al file.
- **Nome del file:** Infine, viene visualizzato il nome del file o della directory.
- `ls -la`: Utilizzando le opzioni `l` e `a` insieme, si ottiene un elenco dettagliato che include anche i file nascosti (i file il cui nome inizia con un punto `.`).

### Esempio di Output

Ecco un esempio di output del comando `ls -la`:

```
drwxrwxr-x 2 user group1 55 Nov 18 09:00 .
```

Ecco un esempio di output del comando `ls -l`:

```
drwxrwxr-x 2 user1 group1 96 Nov 19 19:49 f1
```

Ecco un esempio di output del comando `ls -l` con un link:

```
lrwxrwxrwx 1 user1 group1 3 Nov 20 08:57 f5 -> f4
```

(Se ho un **Hard Link**, le modifiche di f4 si ripercorrono ricorsivamente su f5, **Simbolic link**, le modifiche di f4 non si ripercuotono, se elimino f4, f5 punterà a nulla);

## UNIX File Permissions

In Unix, i permessi dei file determinano chi può leggere, scrivere ed eseguire un file. I permessi possono essere modificati utilizzando il comando `chmod` il quale modifica i permessi del file specificato, `<perm>` rappresenta il codice binario dei permessi:

### Esempi

#### 1. Senza permessi:

- `chmod 000 f1` rimuove tutti i permessi. Il file non è leggibile, scrivibile né eseguibile da nessuno.

## 2. Solo lettura per il proprietario:

- `chmod 400 f1` permette solo al proprietario di leggere il file.

## 3. Permessi di lettura e scrittura per il proprietario, lettura per il gruppo e altri:

- `chmod 644 f1` consente al proprietario di leggere e scrivere, mentre il gruppo e gli altri possono solo leggere.

## 4. Permessi completi per il proprietario, lettura ed esecuzione per il gruppo e altri:

- `chmod 755 f1` consente al proprietario di leggere, scrivere ed eseguire, mentre il gruppo e gli altri possono solo leggere ed eseguire.

### Alternativa:

Il comando `chmod` può essere utilizzato anche in un modo alternativo:

- `chmod +` : Aggiunge permessi.
- `chmod -` : Rimuove permessi.
- Definendo i permessi come:
  - **u**: Proprietario (user)
  - **g**: Gruppo (group)
  - **o**: Altri (others)

### Esempi

#### 1. Aggiungere permesso di lettura per il proprietario:

- `chmod u+r f1` consente al proprietario di leggere il file.

#### 2. Aggiungere permesso di scrittura per il gruppo:

- `chmod g+w f1` consente al gruppo di scrivere nel file.

#### 3. Aggiungere permesso di esecuzione per altri:

- `chmod o+x f1` consente agli altri di eseguire il file.

---

## Come usare UNIX

### Comandi Unix

I **comandi Unix** sono programmi che risiedono nel disco all'interno di directory specifiche, che sono indicate dal **percorso dell'utente**. Questo percorso è una lista di directory in cui il sistema cerca i comandi quando un utente li esegue.

### Dettagli sul Funzionamento

- **Percorso dell'Utente:** Il sistema Unix utilizza una variabile di ambiente chiamata **PATH** per determinare dove cercare i comandi. Questa variabile contiene una lista di directory, separata da due punti (:). Quando un utente inserisce un comando nel terminale, il sistema controlla queste directory in ordine fino a trovare il programma corrispondente.
- **Modifica del PATH:** Gli utenti possono modificare la variabile **PATH** per includere altre directory, consentendo così di eseguire comandi che potrebbero non trovarsi nelle directory predefinite. Questo è utile quando si installano nuovi programmi o si desidera eseguire script personalizzati.

### Comandi di Aiuto:

- **man:** Il comando `man` (manual) viene utilizzato per visualizzare il manuale di un comando specifico. Fornisce informazioni dettagliate sul comando, comprese le sue opzioni, la sintassi e l'uso: `man <command>`;
- **apropos:** Il comando `apropos` cerca tra le pagine del manuale e restituisce un elenco di comandi e funzioni che corrispondono a una parola chiave fornita. È utile quando non si conosce il nome esatto di un comando, ma si ha un'idea di ciò che si desidera fare: `apropos <topic>`;

### Esempio su alcuni comandi:

1. **cat:** Visualizza il contenuto di un file.
2. **cd:** Cambia la directory corrente.
3. **clear:** Pulisce il terminale da output precedenti.
4. **chgrp:** Cambia il gruppo di appartenenza di un file.
5. **chown:** Cambia il proprietario di un file.
6. **chmod:** Modifica i permessi di accesso ai file.
7. **cp:** Copia file o directory.
8. **cut:** Estrae porzioni di testo da file.
9. **echo:** Stampa un messaggio o il valore di una variabile sullo schermo.
10. **find:** Cerca file e directory nel filesystem.

11. **grep**: Cerca una stringa di testo all'interno di file.
12. **ls**: Elenca i file e le directory nella directory corrente.
13. **mkdir**: Crea una nuova directory.
14. **mv**: Sposta o rinomina file o directory.
15. **pwd**: Mostra il percorso della directory corrente.
16. **ps**: Mostra i processi attivi nel sistema.
17. **rmdir**: Rimuove una directory vuota.
18. **rm**: Rimuove file o directory.
19. **sed**: Modifica il testo in flusso o in file.
20. **set**: Configura variabili di ambiente o opzioni di shell.
21. **sudo**: Esegue un comando con privilegi di superuser.
22. **ssh**: Stabilisce una connessione sicura a un altro computer su una rete.
23. **tail**: Mostra le ultime righe di un file.
24. **vi**: Un editor di testo per modificare file.
25. **wc**: Conta il numero di righe, parole e caratteri in un file.
26. **kill**: Invia un segnale a un processo, solitamente per terminarlo.

## Redirection

Per impostazione predefinita, i comandi Unix inviano il loro output alla console, che comprende sia l'

**"standard output" (stdout) (1)** che l'**"standard error" (stderr) (2)**. Tuttavia, è possibile reindirizzare uno o entrambi questi flussi verso un file.

Assumendo che si stia utilizzando la shell Bash, la sintassi per la ridirezione è la seguente:

- `cmd > f1` : Reindirizza l'output standard (stdout) al file `f1`. Se `f1` esiste già, il suo contenuto verrà sovrascritto.
- `cmd 1> f1` : Equivalente al precedente; reindirizza solo l'output standard a `f1`.
- `cmd 2> f2` : Reindirizza l'output di errore standard (stderr) al file `f2`. Se `f2` esiste già, il suo contenuto verrà sovrascritto.

- `cmd 2> f2 1> f1` : Reindirizza l'output di errore standard a `f2` e l'output standard a `f1`.
- `cmd > f12 2>&1` : Reindirizza l'output standard a `f12`, e poi reindirizza l'output di errore standard verso lo stesso file `f12`. In questo modo, sia l'output che gli errori verranno scritti nel file `f12`.
- `cmd >& f12` : Questa sintassi è equivalente alla precedente; reindirizza sia l'output standard che l'output di errore standard al file `f12`.

## Pipes

Le

**pipe** consentono di collegare l'output di un comando all'input di un altro comando, creando una sequenza di operazioni. Questo è utile per elaborare i dati in modo più efficiente senza dover salvare i risultati intermedi in file.

Assumendo che si stia utilizzando la shell Bash, la sintassi per le pipe è la seguente:

- `cmd1 | cmd2` : Reindirizza l'output standard (stdout) del comando `cmd1` all'input standard (stdin) del comando `cmd2`. In questo modo, il risultato di `cmd1` viene utilizzato direttamente da `cmd2`.
- `cmd1 |& cmd2` : Reindirizza sia l'output standard (stdout) che l'output di errore standard (stderr) del comando `cmd1` all'input standard del comando `cmd2`. Questo è utile se si desidera che `cmd2` riceva sia i risultati corretti che eventuali messaggi di errore generati da `cmd1`.

Le pipe possono essere combinate con la ridirezione. Ad esempio, è possibile reindirizzare l'output di un comando a un file e, allo stesso tempo, passare un'altra parte dell'output a un altro comando tramite pipe.

---

# Introduzione Circuiti Integrati

## Storia ed Evoluzione tecnologica dei Circuiti Integrati

Dagli anni '80 le aziende iniziarono a progettare sistemi elettronici integrati custom, che condensavano tutte le funzioni in un unico circuito specifico per l'applicazione, i cosiddetti ASIC. Tali circuiti avevano un grosso problema:

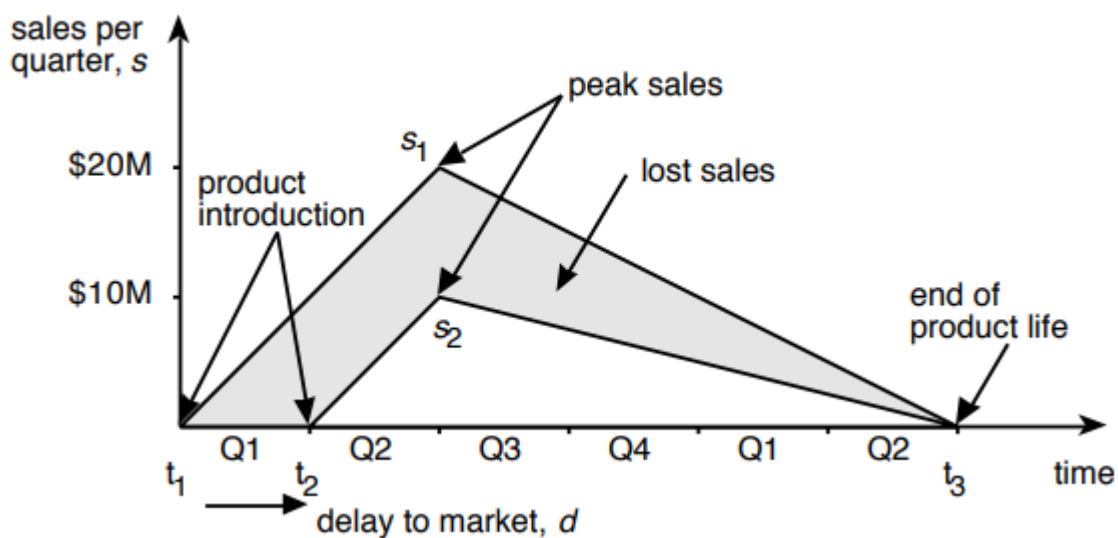
*Erano costosi da realizzare e risolvevano un problema specifico per cui non erano riutilizzabili.*

Il costo di un Circuito Integrato possiamo calcolarlo come  $F + N \cdot V$  dove  $F$  sono **i costi fissi** (profit model, le spese di progettazione, il training del personale, gli apparati hardware e gli strumenti software),  $N$  il numero di dispositivi prodotti e  $V$

**i**

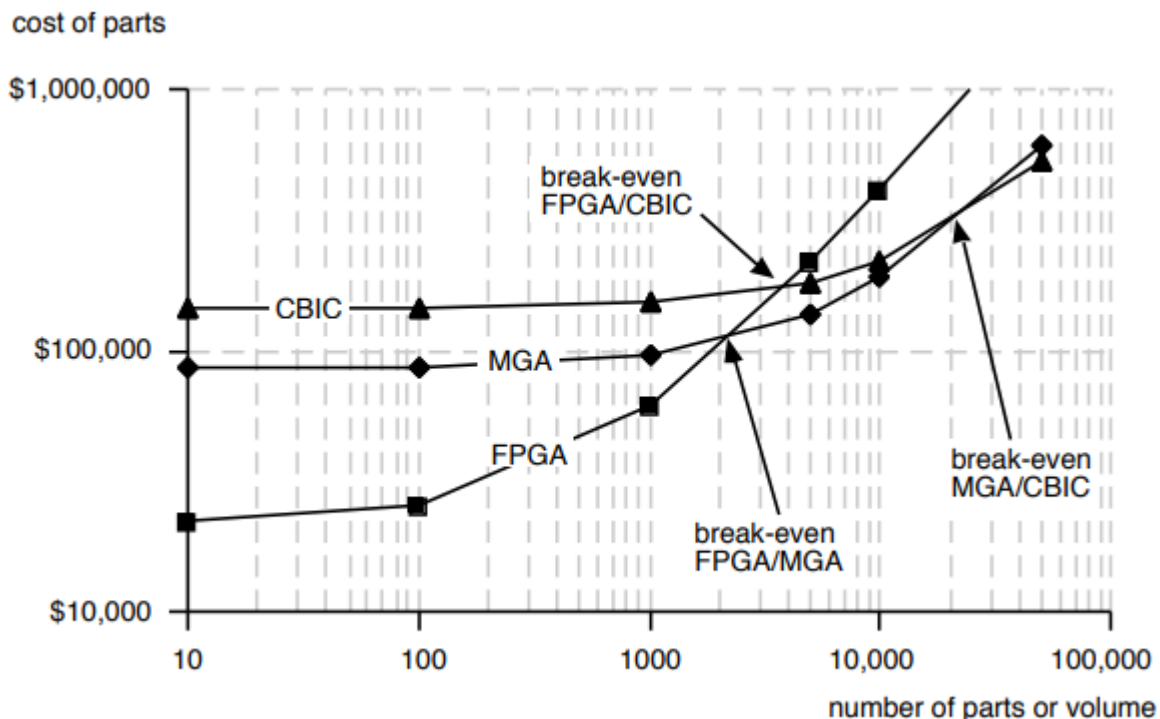
**costi variabili**

(i costi aziendali, gli stipendi, il processo tecnologico, la materia prima).



La tecnologia è diventata esponenziale dal punto di vista delle performances, ma purtroppo è diventata esponenziale anche dal punto di vista dei costi, in

**particolare dei costi fissi.** Quindi la finestra temporale nella quale è conveniente attingere a questa tecnologia integrata è andata a restringersi proprio a causa dei costi e moltiplicato anche per la complessità crescente dei circuiti, l'andamento della curva di vendita è partito da un minimo Iniziare quando il costo era basso ma la tecnologia era insufficiente, poi con l'evolversi ha raggiunto un picco delle vendite che alla fine sono diminuite a causa degli elevati costi di produzione che si sono in seguito presentati.



La strada da seguire rimaneva quella dell'abbattimento dei costi fissi.

**A tal proposito si sono sviluppate delle alternative: i circuiti semi custom (MGA e CBIC) e i circuiti programmabili (PLD, FPGA).**

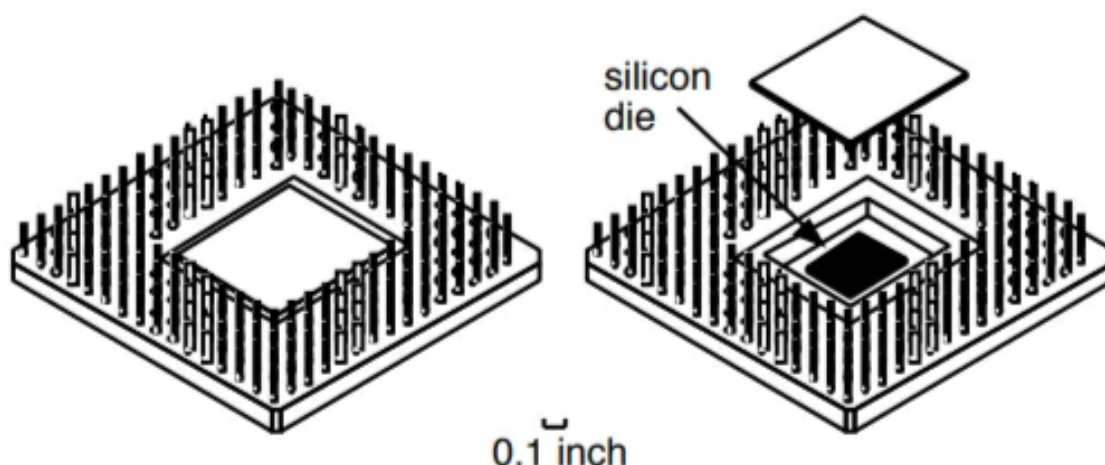
È importante sottolineare che per programmabilità si intende l'operazione che trasforma un supporto hardware vergine in uno specifico circuito, e che non si tratta di un software che va all'interno di un processore.

## I circuiti Integrati

**I circuiti integrati (IC, Integrated Circuits)** sono dispositivi elettronici che combinano un gran numero di componenti elettronici, come transistor, diodi,

resistori e condensatori, su un singolo chip di materiale semiconduttore, solitamente il silicio. Essi costituiscono il cuore dei sistemi elettronici moderni e possono essere classificati in due categorie principali:

1. **Circuiti Non Programmabili**
2. **Circuiti Programmabili**



## Circuiti non Programmabili

I circuiti non programmabili sono dispositivi la cui funzionalità è definita durante la fase di progettazione e produzione. Una volta fabbricati, il loro comportamento non può essere modificato. Questa categoria comprende:

1. **Circuiti Custom:**

Conosciuti anche come **ASIC** (Application-Specific Integrated Circuits), sono progettati su misura per una specifica applicazione o funzionalità. Questi circuiti tuttavia, richiedono alti costi di progettazione e realizzazione;

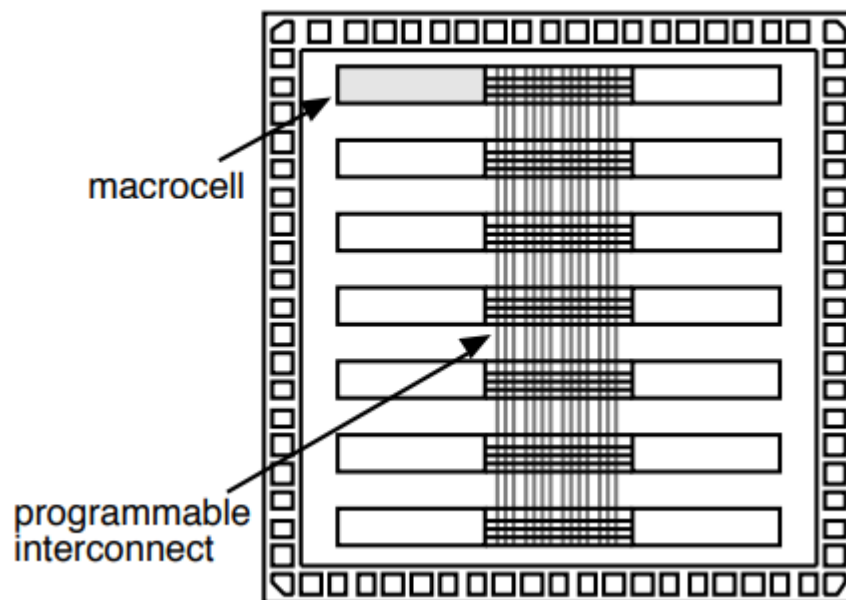
2. **Circuiti Semicustom:**

Offrono un compromesso tra circuiti standard e custom, consentendo una certa personalizzazione senza i costi elevati degli ASIC. Fra di essi troviamo ad esempio gli **MGA (Masked Gate Arrays)**, suddivisi in **Channeled Gate Arrays**, ovvero MGA con canali predefiniti per le interconnessioni, e **Channelless Gate Arrays**, dove i canali sono eliminati;

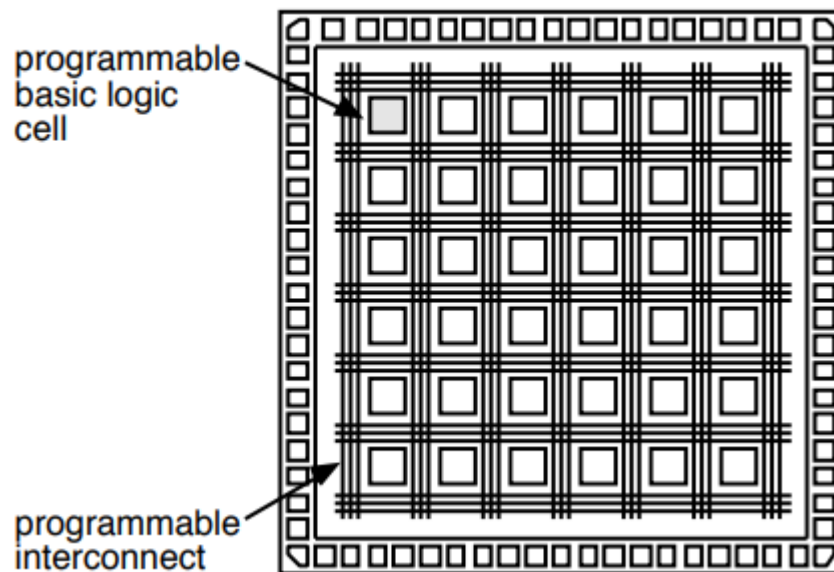
## Circuiti Programmabili

I circuiti programmabili sono dispositivi la cui funzionalità può essere definita o modificata dall'utente dopo la produzione. Questa flessibilità li rende ideali per una vasta gamma di applicazioni. Essi comprendono:

- **PLD (Programmable Logic Devices):** Dispositivi che possono essere programmati per eseguire funzioni logiche semplici.



- **FPGA (Field-Programmable Gate Arrays):** Circuiti integrati programmabili avanzati che possono implementare funzioni logiche complesse. Essi sono composti da blocchi logici configurabili, interconnessi tra loro con connessioni programmabili.



Per **connessione programmabile**, si intende, sia nei PLD, che negli FPGA, delle **reti configurabili di interruttori elettronici** che stabiliscono percorsi logici tra i blocchi del dispositivo. Il segnale digitale (1 o 0) è rappresentato da livelli di tensione, e queste connessioni possono essere programmate e riprogrammate per modificare il comportamento del circuito.

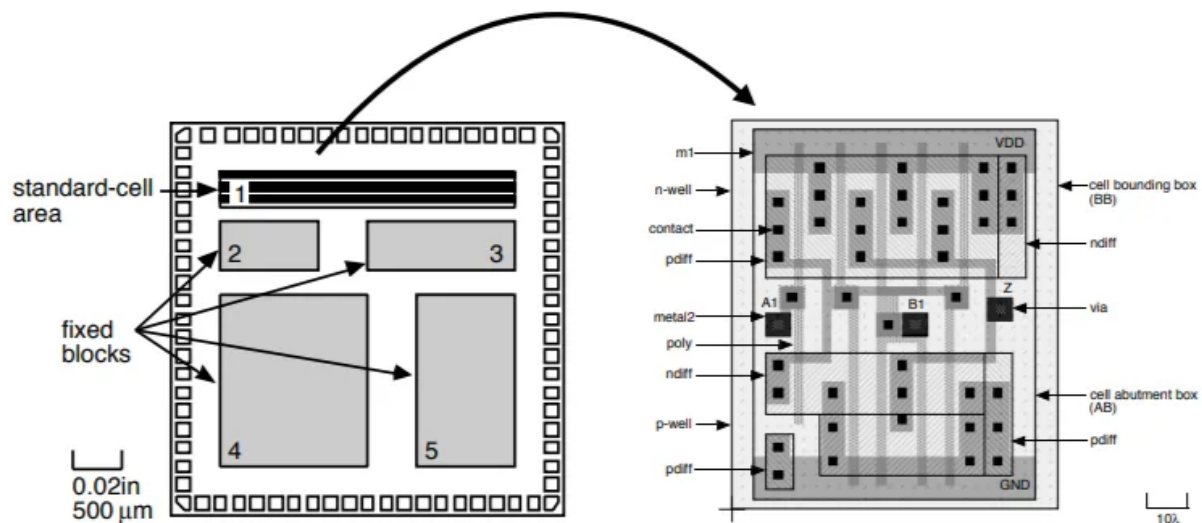
## Circuiti Semi-Custom

### Standard Cells o CBIC

Le **Standard Cells** sono celle logiche che rappresentano un blocco funzionale specifico come:

- Porte logiche (AND, OR, NOT)
- Latch e flip-flop
- Multiplexer

Essi sono pre-progettati e ottimizzati che vengono utilizzati nella progettazione di circuiti integrati (IC), come nei circuiti integrati su misura (Custom ICs) e nei circuiti basati su celle (Cell-Based IC o **CBIC**).



## PROPRIETA'

Il costo fisso delle standard-cells si abbatte molto rispetto ad altre tipologie di circuiti integrati come per esempio i circuiti custom, e allo stesso tempo si può mantenere la variabilità e la personalizzazione del circuito secondo il nostro progetto prestabilito. Realizzando le interconnessioni tra i vari blocchi fissi riusciamo, infatti a personalizzare il circuito e di conseguenza a realizzare ciò che ci serve per un progetto specifico.

## FIXED BLOCK

I **Fixed Block** (o blocchi fissi) sono parti di un circuito integrato che rappresentano funzionalità predefinite e complesse, come memoria (RAM, ROM), processori, o blocchi analogici. A differenza delle **standard cells**, i **Fixed Block** hanno dimensioni e posizioni pre-determinate all'interno del layout del chip e non possono essere ridimensionati o riposizionati liberamente.

Otteniamo quindi un mix tra i componenti che sono di utilizzo comune per realizzare un circuito integrato e celle standard da interconnettere come vogliamo per realizzare una logica specifica che può servirci in un determinato progetto.

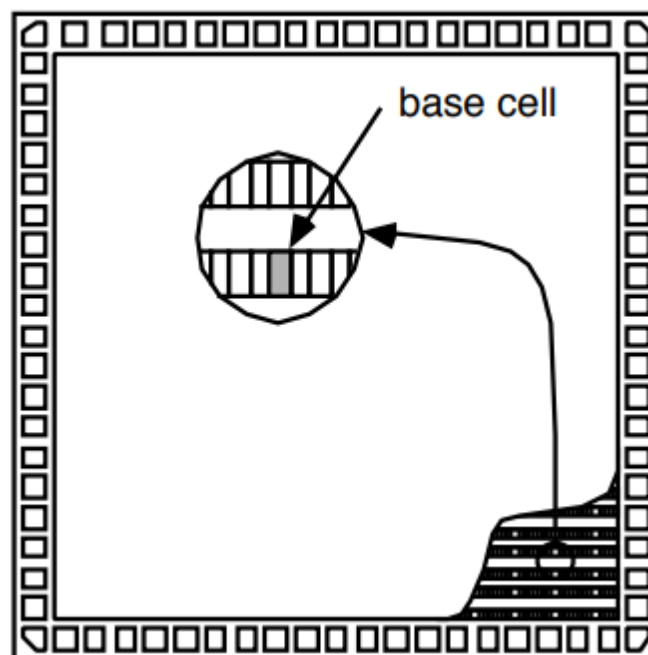
## Gate Array

I **Gate Array** sono circuiti integrati semi-custom in cui la struttura di base dei transistor è già predisposta, ma le interconnessioni vengono personalizzate durante la fase di progettazione. Esistono diverse varianti di **Gate Array**, ciascuna con differenze strutturali e funzionali:

### 1. Channelled Gate Array:

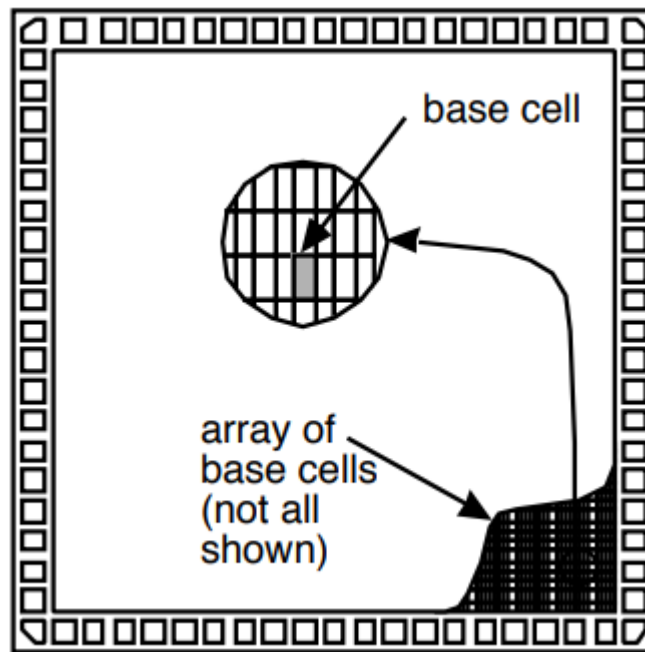
Struttura a canali predefiniti per le interconnessioni tra i blocchi di logica (transistor e porte logiche). Questi canali sono spazi vuoti nel layout che vengono utilizzati per realizzare le connessioni personalizzate durante il processo di fabbricazione.

In questi chip in concetto di standard-cell è esteso a tutto il circuito: possiamo customizzare le interconnessioni sfruttando lo spazio predefinito tra le righe di base cells. Una differenza è che l'altezza è predefinita mentre in un CBIC è definito dal progettista



### 1. Channelless Gate Array (o Sea of Gates)

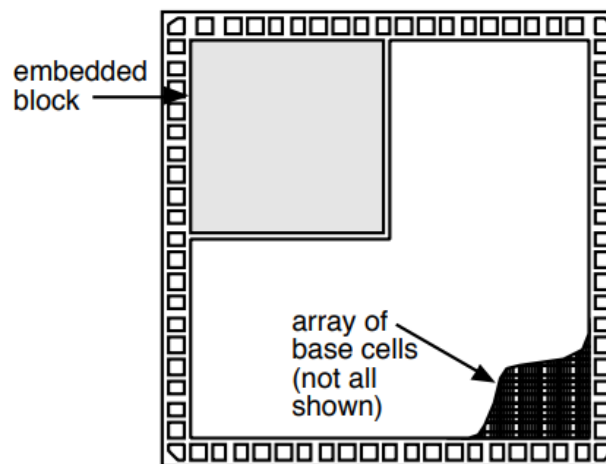
Non ci sono canali predefiniti. L'intera superficie del chip è coperta da un'alta densità di transistor (definita come un "mare" di transistor o "sea of gates"). Le interconnessioni vengono inserite sopra i transistor usando strati metallici.



### 1. Structured Gate Array

È un compromesso tra i

**Gate Array** tradizionali e i circuiti completamente custom. Comprende blocchi di funzionalità predefinite (come memorie, processori, o blocchi analogici), integrati insieme a blocchi programmabili di transistor.



## La Macchina di Turing

La macchina di Turing è un modello teorico di calcolo ideale che descrive un dispositivo a stati infiniti che manipola simboli su un **nastro di lunghezza infinita** secondo una serie di regole predefinite. Tale macchina è composta da

un nastro, suddiviso in celle contenenti un simbolo di un alfabeto finito, **da una testina di lettura/scrittura, da uno stato, e da una funzione di transizione che dato lo stato attuale e il simbolo letto il quale determina:** il nuovo stato, la direzione della testina e il simbolo da scrivere sulla cella corrente.

### **MICROPROCESSORE COME TURING MACHINE**

Un microprocessore può essere considerato una macchina di Turing **grazie alla sua dimensione:** sebbene abbia **memoria finita**, essa è sufficientemente grande da poter essere assimilabile al nastro infinito, la testina è assimilabile all'accesso alla RAM (in r/w) e l'unità di elaborazione corrisponde al ISA (Instruction Set Architecture).

**Di conseguenza, con un microprocessore possiamo risolvere qualunque processo computazionale che ammette una soluzione algoritmica.**

Quando un modello computazionale ha le capacità di soluzione di problemi pari ad una macchina di Turing diciamo che è **"Turing complete"**.

---

# Sintesi dei Circuiti Digitali

## Sintesi

Il processo di sintesi è spesso diviso per livelli di astrazione (**idea → formalizzazione dell'idea → struttura a blocchi → progetto esecutivo**) in cui implementeremo una progettazione di tipo **top-down**, che partirà quindi dal livello di astrazione più alto fino ad arrivare a quello più basso.

Nei linguaggi di programmazione esiste una corrispondenza biunivoca tra costruito sintattico e la sua semantica; cioè la semantica espressa da un determinato costrutto non è interpretabile, ma ha un significato ben definito.

**Il linguaggio hardware è la creazione di un algoritmo volto alla descrizione dell'hardware;**

## Sintesi hardware vs Sintesi Software

Nel caso della **sintesi software** quando scrivo un segmento di codice, **specifico il comportamento che un automa (ovvero il processore) deve seguire per ottenere il risultato.** In sostanza sto definendo il comportamento che l'automa stesso deve avere.

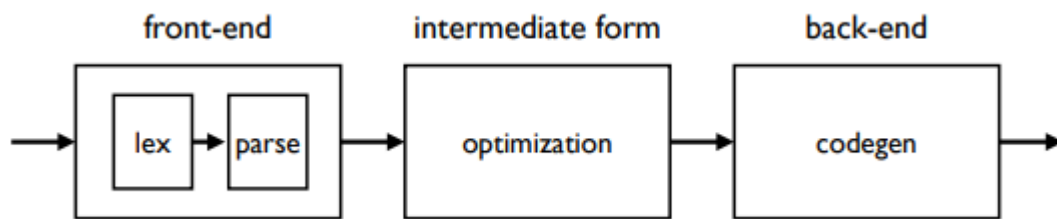
Dunque, l'interfaccia di comunicazione tra l'automa e il programmatore nonché tra hardware e software è il cosiddetto **instruction set** (cioè l'insieme di tutte le istruzioni che l'automa riesce ad eseguire). Esso rappresenta quel livello che sta tra hardware e software (è infatti l'implementazione hardware di primitive software) che consente al comportamento di diventare esecuzione.

Nel caso della sintesi software, le fasi che portano dall'idea all'esecuzione sono:

- **Front - End:** Fa l'analisi lessicale che corrisponde a verificare che, un dato alfabeto, dal punto di vista della grammatica sia corretto
- **Intermediate End:** Creazione di una struttura dati astratta che consente la creazione di un grafo di esecuzione;
- **Back - End:** "Trasforma" la descrizione (grafo) nell' instruction set

## DATAPATH

In quest'ottica definisco il Datapath come il cammino che i dati e le istruzioni devono seguire per essere lavorate;



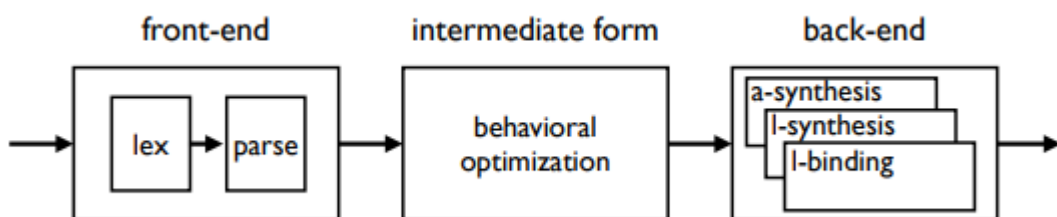
Anche nel caso della **sintesi hardware** specifico un comportamento. Qui però l'hardware non è dato, ma devo costruirlo specificando cosa voglio realizzare. La differenza rispetto alla sintesi software è dunque nel livello di astrazione più basso.

La sintesi hardware comporta, oltre al ricorso a dispositivi progettati ad hoc (eventualmente usando librerie di celle) anche la scelta di dispositivi a volte già esistenti (ad esempio dispositivi semi-custom) ed eventualmente quella dei particolari microprocessori da utilizzare.

La selezione dei diversi dispositivi influenza a sua volta la generazione del software, che dunque dipende dalle decisioni prese nelle fasi iniziali della sintesi hardware.

## PLATFORM

A differenza del datapath, qui non c'è un vero e proprio cammino dei dati. Infatti noi andiamo a "creare" la piattaforma che poi avrà la funzione di trasmettere i dati ad un livello software più elevato;



## Modello Computazionale

Nel processo di progetto ci sono diverse fasi, le quali hanno un certo numero di funzioni che devono essere assolate. Il processo di progetto prevede che:

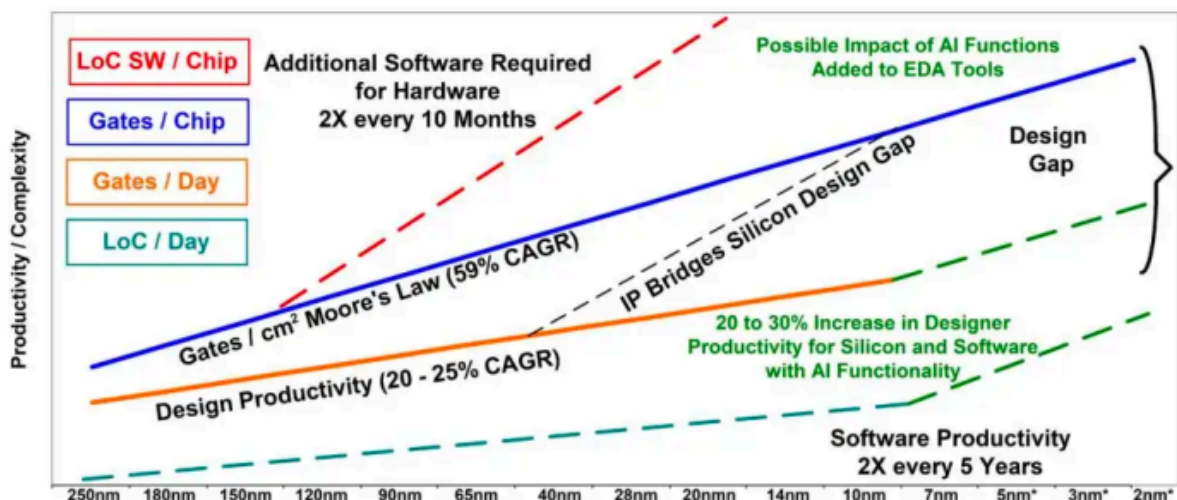
- **Deve essere possibile codificare il modello:** cioè devo avere un entry point in cui specifico questa funzionalità.
- **Valido e debuggo attraverso la simulazione al livello della codifica.**
- **Scelta delle opzioni architetturali:** scompongo la mia idea in blocchi funzionali;
- **Progetto esecutivo:** si parla di **sintesi o co-sintesi: la prima genera l'hardware che realizza quel funzionamento mentre la seconda è data dal fatto che posso decidere di realizzare porzioni in hardware e porzioni in software su un microprocessore dedicato.**

Il motivo per cui si è passati da un flusso di progetto hardware ad uno software è descritto dal cosiddetto *productivity gap*.

## PRODUCTIVITY GAP

**Il numero di transistor resi disponibili dalla tecnologia è di gran lunga superiore, e cresce più rapidamente, del numero di transistori che VHDL mi permette di utilizzare dato il suo livello di astrazione.**

Nascono così i software EDA (Electronic Design Automation) che consentono di catturare l'idea progettuale attraverso un modello HDL, di sintetizzare via via i livelli più bassi di astrazione e, infine, di ottimizzare il circuito e il parametro di progetto sulla base di una data metrica.



## Sintesi Hardware

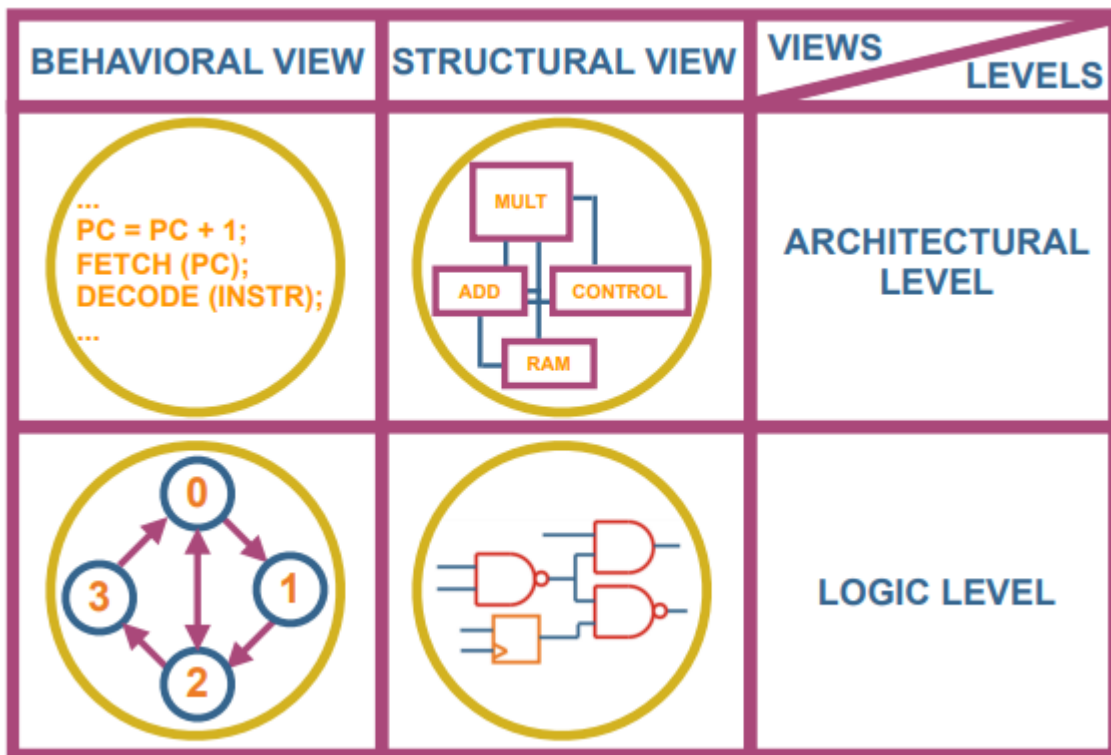
La **sintesi hardware** è il processo attraverso cui una specifica comportamentale di alto livello viene trasformata in hardware concreto. Questo processo prende una descrizione di **cosa** deve fare il circuito e la traduce in **come** deve essere realizzato fisicamente.

Si hanno 3 livelli di astrazione; dal **basso verso l'alto**:

- **Livello geometrico** (leggera astrazione del livello fisico)
- **Livello logico** (descrizione attraverso porte logiche): a questo livello di astrazione si minimizza l'area in presenza di vincoli sul ritardo di propagazione, inoltre si minimizza il ritardo di propagazione in presenza di vincoli sull'area.
- **Livello architetturale**: a questo livello di astrazione si determina il cycle-time, si minimizza l'area in presenza di vincoli sulla latenza e infine si minimizza la latenza in presenza di vincoli sull'area;

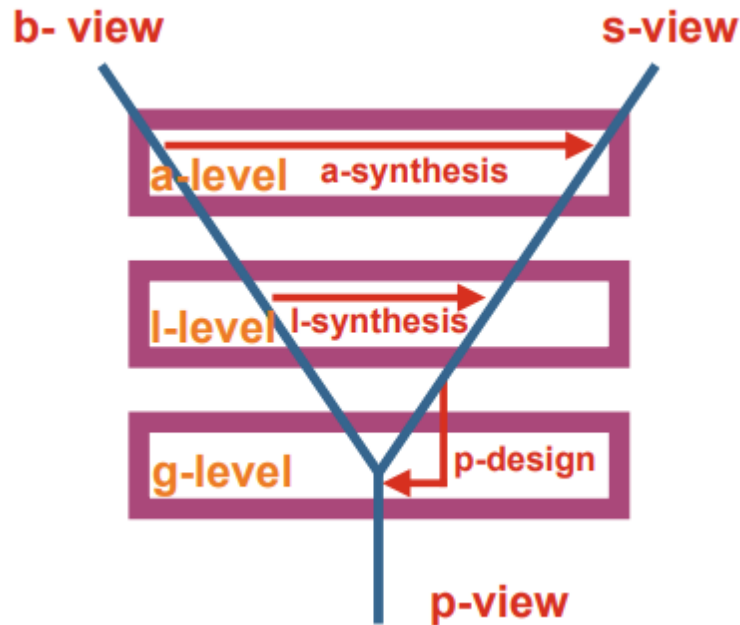
I livelli di astrazione possono essere visti in due modi:

- **Vista strutturale**
  - Per il livello logico è la mappatura di porte logiche;
  - Per il livello architetturale corrisponde ad uno schema a blocchi;
- **Vista comportamentale**
  - Per il livello logico è una macchina a stati finiti;
  - Per il livello architetturale corrisponde al codice sorgente;



Ad ogni livello di astrazione corrisponde un livello di progetto; in particolare, ad ogni passaggio da un livello di astrazione più elevato ad uno inferiore, **corrisponde una fase di sintesi**, cioè una fase di ottimizzazione della soluzione per ottenere vincoli richiesti. Per quanto detto si ha dunque:

- **Sintesi architetturale** : a partire da un comportamento architetturale, determina la struttura macroscopica in termini di macro-blocchi e loro interconnessioni.
- **Sintesi logica**: a partire da un comportamento logico, determina la struttura microscopica in termini di porte logiche e loro interconnessioni .
- **Sintesi geometrica**: determina il layout e quindi la definizione geometrica delle porte logiche, la loro posizione e le interconnessioni.



## Ottimizzazione della Sintesi

Si è detto che nella sintesi software abbiamo già data la macchina a stati mentre nella sintesi hardware la stiamo generando; questo processo di generazione dell'hardware corrispondente, prevede un incremento dell'informazione.

I processi così fatti si chiamano **processi di ottimizzazione**, cioè sono delle ricerche funzionali che mi rappresentano l'energia di una certa configurazione che devo minimizzare.

Questa minimizzazione avviene sulla base di certi vincoli, cioè di certe metriche ovvero variabili funzionali che mi interessano. Nei circuiti integrati queste sono principalmente:

- **Area occupata**: è una proprietà estensiva. Cioè un circuito di doppia funzionalità occupa due volte l'area.
- **Performance (resa di produzione)**: Indica quanto veloce posso andare. Questa non è una proprietà estensiva e non ha una definizione univoca, infatti:
  - Nei circuiti combinatori è il "ritardo di programmazione" e il "cycle-time";
  - Nei circuiti sequenziali è la "latenza" ovvero la distanza da quando i dati sono validi a quando le uscite recepiscono il cambiamento di stato;

- Nel caso dei circuiti "pipelined" è il "throughput", ovvero la velocità con cui il sistema può elaborare un certo numero di dati o operazioni in un determinato periodo di tempo. In pratica, è la misura del numero di risultati che la pipeline può produrre per unità di tempo;

*(Il termine Pipeline viene utilizzato per indicare un insieme componenti software collegati tra loro in cascata. In questo modo la latenza rimane sempre la stessa ma lo throughput cioè il rate di uscita aumenta; quindi la metrica non è più la latenza ma è lo throughput.)*

- Potenza dissipata
- Testabilità

## Metodologia della Sintesi

### ELEMENTI

- **Spazio di Progettazione (S):** Include tutte le implementazioni possibili che soddisfano il comportamento desiderato.
- **Spazio delle Funzioni di Valutazione (E):** È ottenuto applicando metriche di valutazione (come velocità, consumo energetico, area del circuito, ecc.) a ogni punto in S.
- **Metriche di Progettazione (N):** Sono i criteri o indicatori che utilizziamo per valutare le diverse implementazioni. Queste metriche determinano la dimensione dello Spazio E, che può essere misurato lungo più dimensioni (ad esempio, un progetto potrebbe essere valutato su tre dimensioni: prestazioni, costo e consumo energetico).

### PROCESSO

Tra tutti i possibili punti di S in E, la scelta non è univoca ma devo aggiungere delle informazioni attraverso la ricerca di un'implementazione ottima tra quelle funzionalmente corrette che corrisponde ad un ottimo (ovvero un minimo).

### COMPLESSITA'

Dal punto di vista della complessità algoritmica, il processo di ottimizzazione non è trattabile.

Il processo di progettazione diventa estremamente complesso e difficile da risolvere in modo ottimale attraverso algoritmi standard. Questo perché si tratta di un problema che coinvolge molte variabili e dimensioni (N dimensioni), il che lo rende **intrattabile** dal punto di vista algoritmico.

Per affrontare questa complessità, invece di cercare soluzioni perfette, si punta a trovare **soluzioni sub-ottimali**. Questo viene fatto **scomponendo il problema complesso in sotto-problemi più semplici** con meno variabili. In questo modo, si utilizzano approcci **euristici** per cercare soluzioni che, anche se non perfette, sono comunque efficaci e gestibili.

Il processo di ottimizzazione può essere affrontato a diversi livelli, ciascuno con i propri obiettivi:

### 1. Livello Architeturale:

- **Determinare il tempo di ciclo:** Assicurarsi che il tempo di esecuzione per ogni ciclo del sistema sia ottimizzato.
- **Minimizzare l'area rispettando i vincoli di latenza:** Cercare di ridurre al minimo l'area fisica del progetto (spazio occupato su un chip) senza superare i limiti imposti sulla latenza (tempo impiegato per completare un'operazione).
- **Minimizzare la latenza rispettando i vincoli di area:** Ridurre al minimo la latenza, assicurandosi che l'area fisica non superi una certa soglia.

### 2. Livello Logico:

- **Minimizzare l'area rispettando i vincoli di ritardo di propagazione:** Ridurre lo spazio fisico del circuito, garantendo che il ritardo di propagazione dei segnali (il tempo impiegato dai segnali per attraversare il circuito) rimanga nei limiti accettabili.
- **Minimizzare il ritardo di propagazione rispettando i vincoli di area:** Ridurre il tempo di propagazione dei segnali, mantenendo l'area fisica sotto controllo.

---

## Esempio di Sintesi

### ESEMPIO EQUAZIONE DIFFERENZIALE

Risoluzione di  $y'' + 3xy' + 3y = 0$

- Pongo le condizioni iniziali  $x(0) = 0, y(0) = 0, y'(0) = u$  e  $x \in [0, a]$ , allora ottengo:

$$dy'/dx + 3xy' + 3x = 0 \Rightarrow dy' + 3xy'dx + 3ydx = 0$$

- Risolvendola alle differenze finite si ottiene:

$$u = u_0 - 3xudx - 3ydx \Rightarrow dy/dx = u \Rightarrow y = y_0 + udx$$

La porto dunque a programma (xl ul, yl sono i nuovi valori di x, u, y)

```
diffeq
{
  read (x, y, u, dx, a);
  repeat
  {
    xl = x + dx;
    ul = u - (3 * x * u * dx) - (3 * y * dx);
    yl = y + u * dx;
    c = x < a;
    x = xl; u = ul; y = yl;
  } until (c);
  write (x, y);
}
```

## COMPONENTI

- Si può osservare che ci serviranno almeno 1 moltiplicatore, ed 1 ALU (perché ci servono sia sommatore che sottrattori) e 1 unità di controllo/memoria (per salvare i risultati parziali);

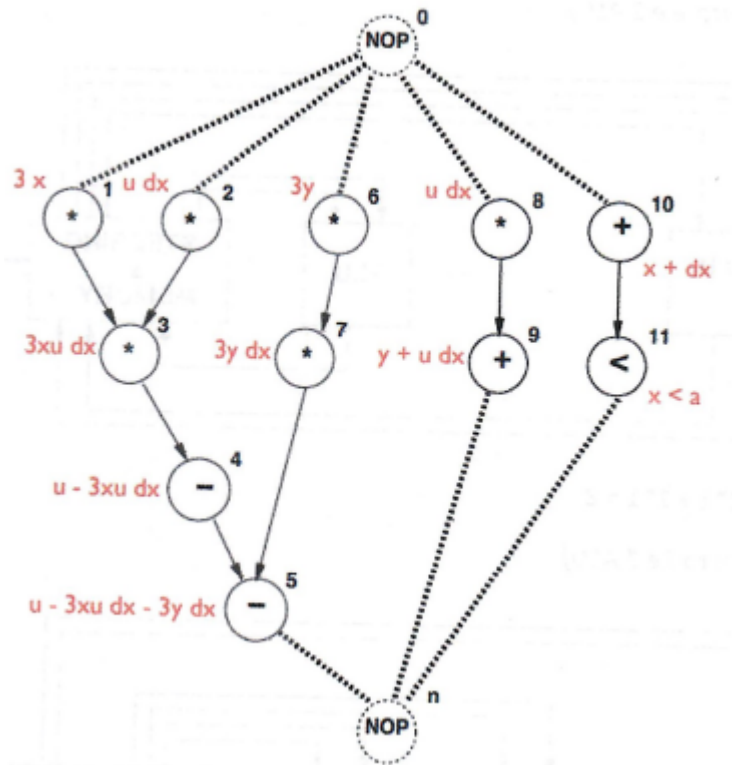
Congruentemente a quello che abbiamo visto essere il flusso di sintesi, dobbiamo passare da una rappresentazione testuale (sequenziale) ad una "funzionale". Inizieremo da una forma intermedia che di fatto è un grafo di esecuzione.

Da questa rappresentazione funzionale si vede il parallelismo e quindi anche il numero di componenti che mi servono

## DATA FLOW GRAPH

Questa serie di istruzioni che ciclerò N volte, mi definisce il cosiddetto direct acyclic graph (DAG) che ha un punto di partenza che è un "no istruzione" (NOP).

**Negli alberi, per passare al prossimo ramo, devo aver concluso tutte le operazioni su quel livello;**



### QUAL'E' IL NUMERO DI COMPONENTI OTTIMALI?

Dobbiamo trovare il costo di ogni comparatore in

**area e in latenza.** Per calcolare il **costo in latenza** dobbiamo capire l'ordine delle operazioni che dobbiamo fare e quante ne possiamo fare in parallelo. La **LATENZA** è l'intervallo di tempo che intercorre tra quando ci sono gli ingressi validi a un blocco, a quando ci sono le corrispondenti uscite valide;

Supponendo che dal punto di vista dei costi e delle performance le figure di merito siano:

- Un moltiplicatore costa 5 unità di area e ha una latenza di 1 unità di tempo;
- La ALU ha un costo di 1 unità di area e ha una latenza di 1 unità di tempo;
- L'unità di controllo + memoria costa 1 unità di area e non spreca tempo;

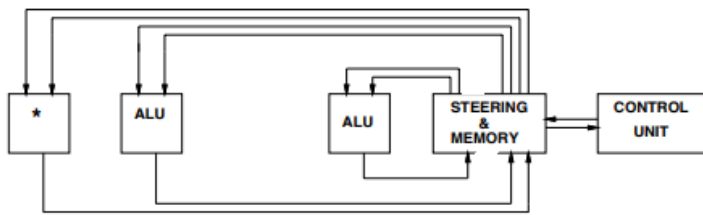
(nella realtà è diverso);

### Soluzione (1,1)

Studiamo il caso con 1 ALU + 1 Moltiplicatore + 1 unità di controllo;

#### AREA (proprietà estensiva)

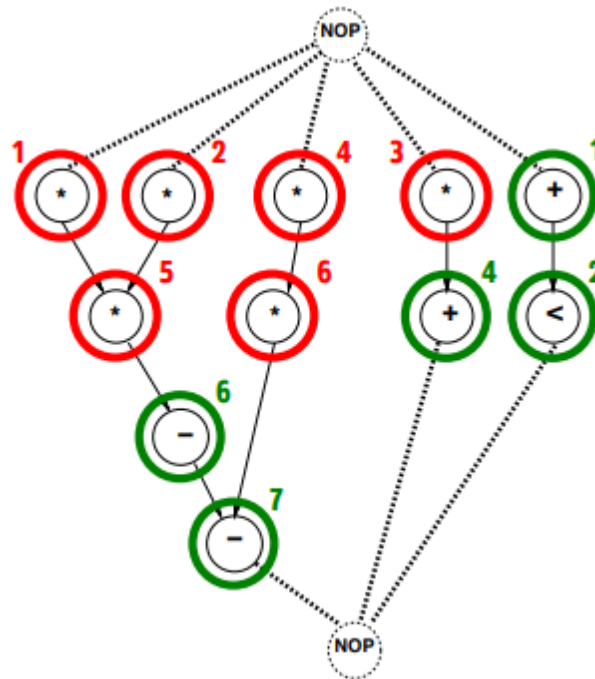
- Costo =  $1 * 5 + 1 * 1 + 1 * 1 = 7$



| 1 MUL; 1 ALU (1, 1) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 1 x 5 | 1       |
| ALU                 | 1 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 7     | 7       |

## LATENZA

Per capire la latenza, devo determinare **come posso eseguire le operazioni nei diversi casi**. Indico gli step temporali con i quali faccio fare una specifica operazione; ad ogni **colpo di clock scandito dalla macchina, gli stati possono fare una moltiplicazione ed una somma/sottrazione/confronto per volta**;



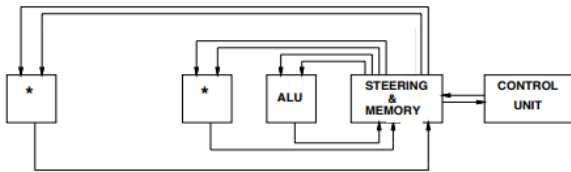
Si ottiene quindi una latenza pari a 7, per concludere tutte le operazioni.

## Soluzione (2,1)

Studiamo il caso con 2 Moltiplicatore + 1 ALU+ 1 unità di controllo;

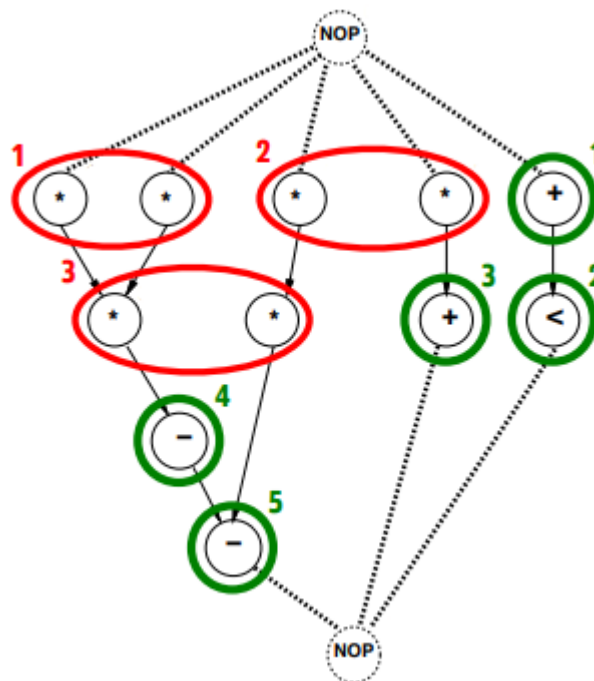
**AREA (proprietà estensiva)**

- Costo =  $2 * 5 + 1 * 1 + 1 * 1 = 12$



| 2 MUL; 1 ALU (2, 1) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 2 x 5 | 1       |
| ALU                 | 1 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 12    | 5       |

## LATENZA



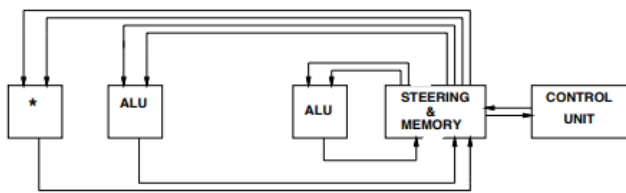
Si ottiene quindi una latenza pari a 5, per concludere tutte le operazioni.

## Soluzione (1,2)

Studiamo il caso con 1 Moltiplicatore + 2 Alu + 1 unità di controllo;

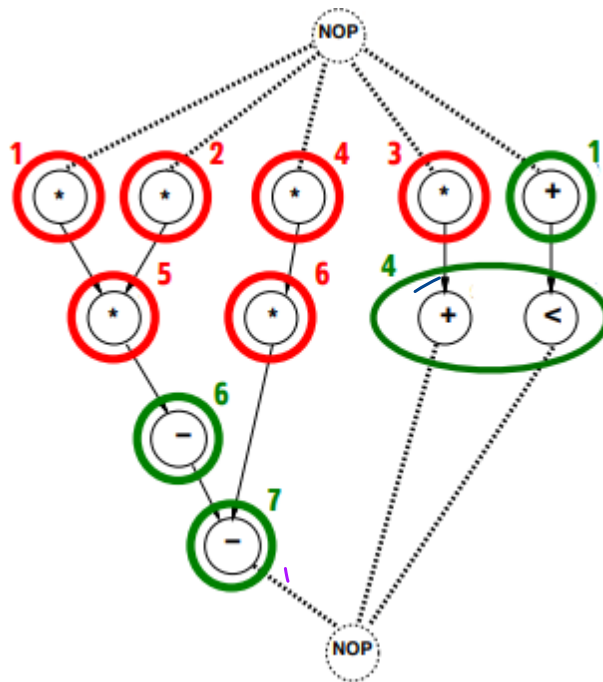
### AREA (proprietà estensiva)

- Costo =  $1 * 5 + 2 * 1 + 1 * 1 = 8$



| 1 MUL; 2 ALU (1, 2) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 1 x 5 | 1       |
| ALU                 | 2 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 8     | 7       |

## LATENZA

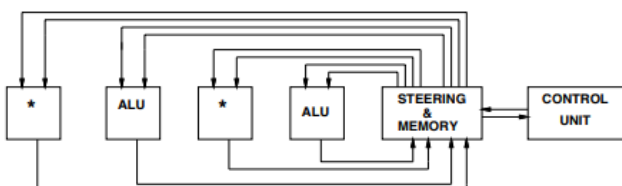


Si ottiene quindi una latenza pari a 7, per concludere tutte le operazioni.

## Soluzione (2,2)

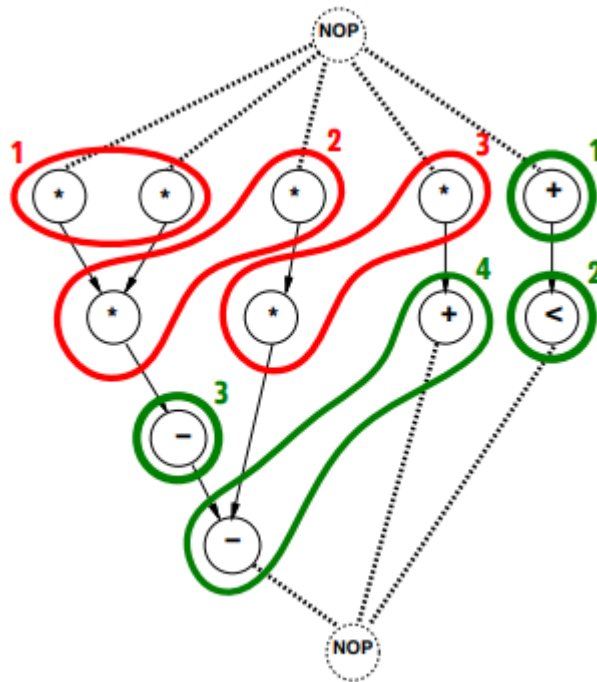
Studiamo il caso con 2 ALU + 2 Moltiplicatori + 1 unità di controllo;

- Costo =  $2 * 5 + 2 * 1 + 1 * 1 = 13$



| 2 MUL; 2 ALU (2, 2) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 2 x 5 | 1       |
| ALU                 | 2 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 13    | 4       |

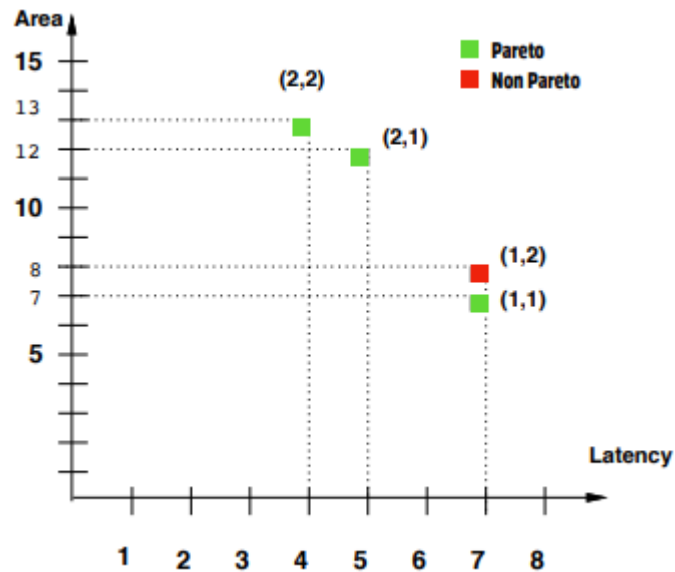
## LATENZA



Si ottiene quindi una latenza pari a 4, per concludere tutte le operazioni.

## Analisi Soluzioni

Mettendo in un grafico i risultati possiamo discriminare i punti che sono oggettivamente peggiori degli altri. Una volta rimossi tali punti possiamo ottenere la curva di Pareto che è l'insieme dei **punti di Pareto che sono i punti di un processo di ottimizzazione nei quali, per ciascuno, una metrica (latenza o l'area) è migliore di tutte le altre.**



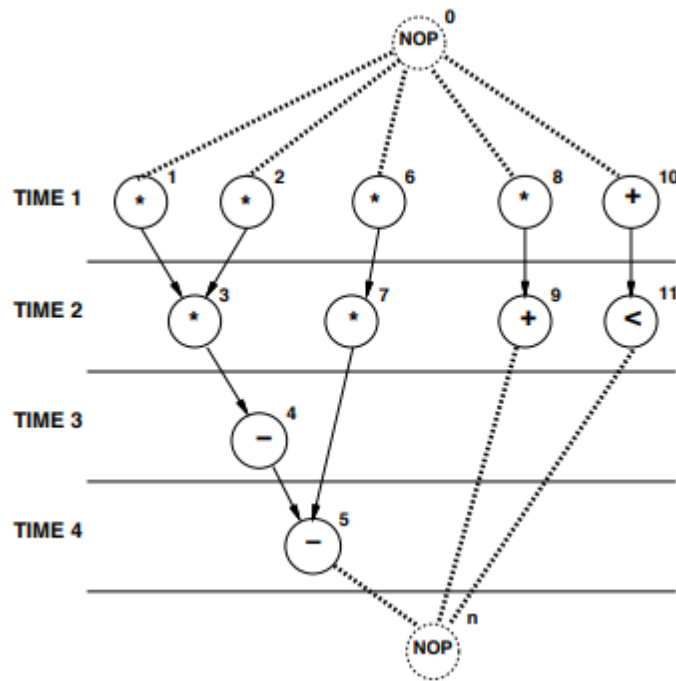
## Scheduling

Si dice **Scheduling**, determinare quando (cioè a quale istante di clock), dev'essere eseguita una determinata operazione. Può essere con o senza vincoli sul numero di risorse disponibili;

Esistono tre tipi di Scheduling:

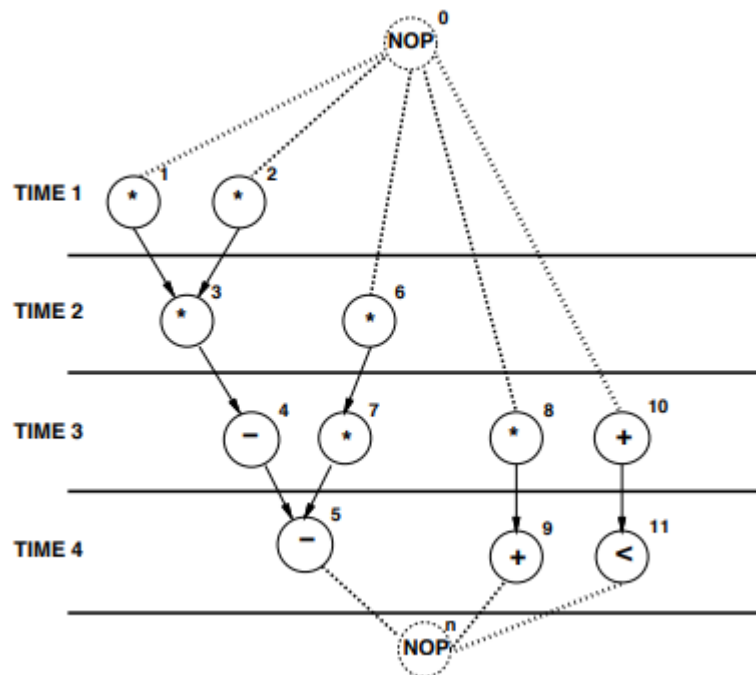
### ASAP Scheduling

Le operazioni sono eseguite il prima possibile, schedulando i vertici partendo dal primo, assegnato al vertice  $v_i$  il tempo di esecuzione come il **massimo tra quelli già schedulati + il suo ritardo di propagazione**;



### ALAP Scheduling

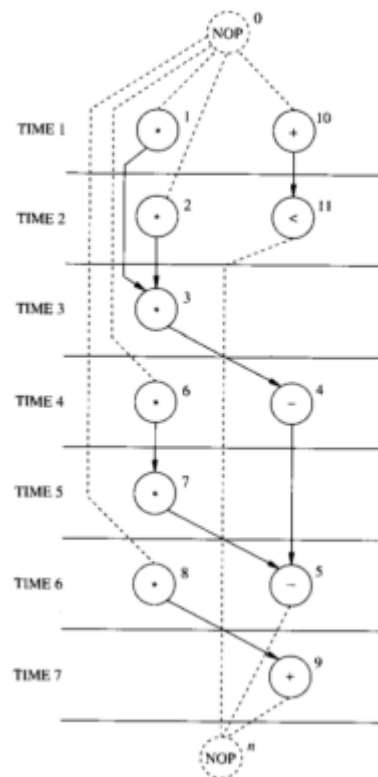
In questo caso parto dall'ultimo vertice e salgo **sottraendo i ritardi di propagazione**;



### Resource Constrain Scheduling

Viene fatto successivamente ad aver definito un ASAP o ALAP scheduling.  
Presi i limiti dei due modelli, **rialloco il grafo per allocare le risorse in modo da**

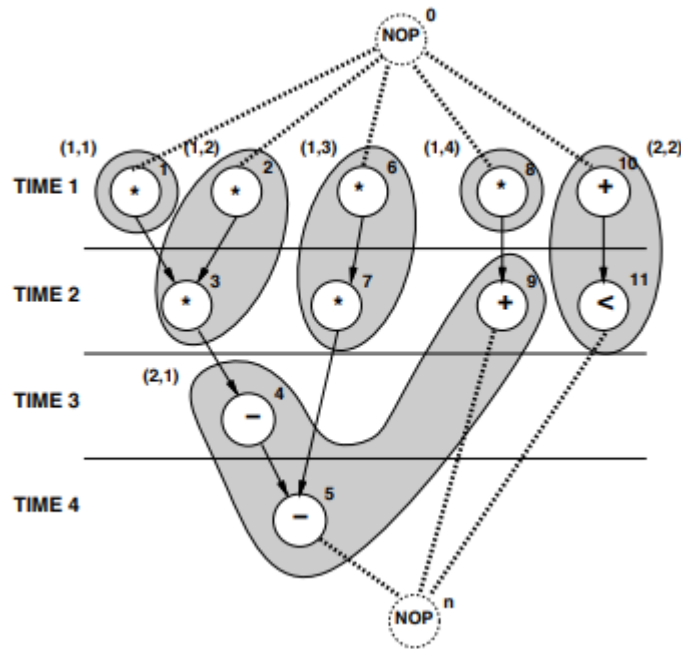
**non sovrapporre l'utilizzo.** Ad ogni intervallo di tempo quindi non esisterà mai la copresenza di due operazioni dello stesso tempo. In quel caso lo scheduling viene fatto con un numero di risorse limitato;



## Binding

### Resource Binding

Assegno a ciascuna risorsa (tramite macchina a stati) le operazioni da svolgere, ottenendo così una sintesi Hardware;



## Sintesi FPGA'S

La sintesi si compone dei seguenti passi:

- **Sintesi dai livelli di astrazione superiori a quelli più bassi:** Può avvenire manualmente o automaticamente.
- **Allocare le risorse** in cui si implementano delle ottimizzazioni;
- **Design Transformation**, cioè trasformazioni per soddisfare i vincoli;
- **Composizione/Decomposizione**, cioè il raggruppamento/suddivisione dei blocchi funzionali per corrispondere ai blocchi tecnologici disponibili
- **Scheduling**, per assegnare gli istanti di tempo ai quali svolgere le diverse operazioni. Questa esecuzione schedulata sarà poi mappata sui vari registri e sulle risorse e verranno instradati i vari percorsi in modo tale da realizzare una determinata operazione.
- **Binding:** cioè l'assegnazione delle operazioni alle risorse disponibili. Avrò quindi una ALU (se non la ho me la realizzo).

**Nei circuiti Custom, devo mappare una descrizione dell'hardware che voglio implementare sulle risorse disponibili.** Definisco quindi:

**ABSTRACT BEHAVOIR**

Descrive il comportamento del circuito tramite variabili lette e scritte, condizioni, valori temporanei e finali, senza specificare come sarà implementato il circuito.

### CONTROL FLOW BEHAVIOR

Descrive il comportamento del circuito in termini di registri, logica combinatoria e ordine di esecuzione delle operazioni.

### DATAPATH

Il

**Datapath** è la catena di risorse hardware necessarie per eseguire le operazioni richieste.

## Livelli di Astrazione

Ci sono diversi livelli di astrazione nel descrivere un circuito:

- **Gate Level:** Descrizione a livello di porte logiche;
- **Logic Level:** Simile al gate level, ma in termini di funzioni booleane;
- **Register- Transfer Level:** Descrive i trasferimenti di dati tra registri e le operazioni combinatorie. Questo livello include una parte comportamentale (come i dati si spostano) e una parte strutturale (come i registri sono connessi tra loro):
  - **Parte Comportamentale:** Descrive il comportamento del circuito a livello RTL. Vengono rappresentate solamente le transazioni visibili a questo livello. I segnali di controllo o sono dati per impliciti o espressi a un livello astratto;
  - **Parte Strutturale:** Descrive il circuito attraverso una descrizione strutturale a livello RT nella quale sono specificati i registri, gli operatori funzionali e le loro interconnessioni. Il controllo si suppone ricompreso implicitamente nella descrizione o specificato a parte;

## Obiettivi della Sintesi

Durante il processo di sintesi, si cerca di:

- Massimizzare la **velocità**.
- Minimizzare l'**area** occupata dalle risorse hardware.
- Minimizzare i consumi di **potenza**.

- Ridurre il **tempo di progettazione**.
- Massimizzare l'**affidabilità** e la **testabilità** del circuito.

## VINCOLI DELLA SINTESI

Il processo di sintesi deve rispettare diversi vincoli, tra cui:

- **Limiti tecnologici** (come l'assenza di tristati o memoria integrata).
- **Ritardi temporali** tra eventi.
- **Limiti di area** e numero di pin disponibili.
- **Tempi di esecuzione** massimi.
- **Affidabilità e testabilità** del circuito.

## TIPI DI SINTESI

- **Sintesi Comportamentale**: Traduce il comportamento astratto e algoritmico in una rappresentazione a flusso di dati.
  - **Sintesi RTL (Register-Transfer Level)**: Converte la rappresentazione a flusso di dati in una a livello di trasferimento tra registri.
  - **Sintesi Logica**: Converte la rappresentazione RTL in una logica basata su porte.
-

# Introduzione a VHDL

## Storia di VHDL

### VHDL: SIGNIFICATO

VHDL, acronimo di "Very High Speed Integrated Circuits HDL", è un linguaggio di descrizione hardware concepito attorno al 1980 per rispondere a specifiche esigenze del settore tecnologico. Questo linguaggio è nato con l'obiettivo di standardizzare i metodi di progettazione e unificare i vari dialetti HDL esistenti in un unico linguaggio, migliorando la portabilità dei progetti tra diversi strumenti di progettazione assistita da computer (EDA). Grazie a VHDL, il tempo di progettazione dei circuiti digitali è stato ridotto notevolmente: un processo che richiedeva da 6 a 18 mesi è stato compresso attraverso un approccio più efficiente.

La necessità degli **HDL (Hardware Description Language)** è emersa a causa del cosiddetto "Productivity Gap", ossia un divario tra la crescente complessità dei circuiti digitali e la capacità produttiva disponibile. Per superare questa limitazione, il settore ha adottato una nuova prospettiva, passando dalla progettazione a livello di porte logiche a livelli di astrazione più elevati. L'approccio VHDL permette così di descrivere circuiti in termini di linguaggio software, supportato da strumenti EDA e facilitando la transizione dalla progettazione manuale alla sintesi automatica.

### VHDL: NASCITA

Nel giugno del 1981, durante un workshop tenutosi a Woods Hole, Massachusetts, esponenti del governo statunitense e della comunità accademica definirono le caratteristiche dei Very High Speed Integrated Circuits (VHSIC). Nel luglio del 1983, DARPA (Defense Advanced Research Projects Agency), in collaborazione con Intermetrics, IBM e Texas Instruments, firmò un contratto per lo sviluppo di VHDL, con l'obiettivo di creare uno standard per la progettazione di circuiti ad alta velocità.

### VHDL: EVOLUZIONE

Nell'agosto del 1985 venne rilasciata la versione 7.2 di VHDL, e nel dicembre 1987 il linguaggio fu ufficialmente riconosciuto come standard IEEE. Da allora, sono state rilasciate diverse versioni successive: quella del 1987 fu seguita da

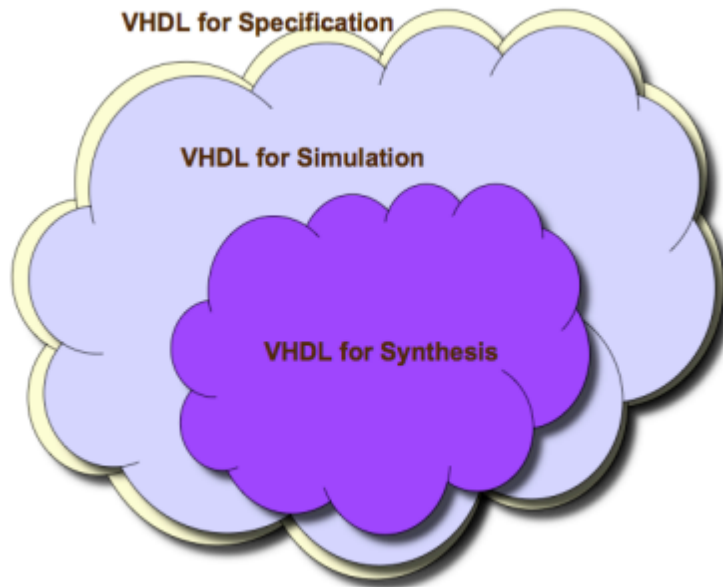
aggiornamenti rilevanti come le versioni del 1993 e del 2008, mentre altre, come quelle del 2000 e del 2002, hanno avuto un impatto minore.

## La Struttura di VHDL

### I livelli di VHDL

La struttura di VHDL si articola in **tre livelli principali**, ognuno dei quali risponde a esigenze specifiche nel processo di progettazione e verifica dei circuiti digitali:

1. **VHDL for Specification:** Questo livello di VHDL è dedicato alla descrizione generale del design per verificare il comportamento funzionale del circuito hardware. A questo livello, il codice permette di testare se il progetto risponde correttamente alle specifiche iniziali, concentrandosi quindi sugli aspetti logici e comportamentali del design.
2. **VHDL for Simulation:** In questo livello, il codice è scritto con l'obiettivo di consentire una simulazione accurata del circuito. VHDL for Simulation permette di testare e verificare come il circuito risponderà in diverse condizioni, attraverso una simulazione dettagliata che imita l'effettiva operatività del dispositivo. Questo livello consente di individuare eventuali errori prima della realizzazione fisica del circuito.
3. **VHDL for Synthesis:** Questo livello è pensato per la generazione del circuito fisico e il codice è ottimizzato per essere interpretato e convertito in hardware reale, tipicamente FPGA o ASIC. A differenza dei livelli precedenti, VHDL for Synthesis include solo istruzioni che possono essere tradotte in componenti hardware effettivi, eliminando elementi puramente comportamentali o di simulazione.



## ENTITY & ARCHITECTURE

In VHDL, la struttura di un design è effettivamente composta da due componenti fondamentali: **Entity** e **Architecture**, che insieme definiscono completamente il comportamento del circuito.

1. **Entity:** L'entity rappresenta l'interfaccia del blocco hardware, stabilendo le connessioni con l'esterno, cioè gli input e output del circuito. Qui vengono definite le porte (input e output) e i tipi di segnale che il circuito può ricevere o inviare. In altre parole, l'entity specifica cosa il circuito può fare, descrivendo la sua interfaccia, ma non come deve realizzarlo.
2. **Architecture:** L'architecture contiene la descrizione funzionale e strutturale di come il circuito implementa il comportamento definito nell'entity. È qui che vengono specificate le operazioni logiche, i processi e le relazioni tra i segnali interni, determinando in dettaglio le operazioni che il circuito deve eseguire. L'architecture può essere descritta a diversi livelli di astrazione, come logico o gate-level, oppure a un livello di astrazione più alto, come il livello RTL (Register Transfer Level).

Quindi, schematicamente, un design VHDL contiene:

- **Input e Output** definiti nell'entity.
- **Blocco hardware**, descritto attraverso l'architecture, che specifica come il circuito processa i segnali in ingresso per generare quelli in uscita.

Questo schema generale consente di separare chiaramente l'interfaccia del circuito (entity) dalla sua logica interna (architecture), agevolando la comprensione e la manutenzione del design.

## VHDL

### Generalità

#### CASE SENSIVITY

VHDL è

*case insensitive*, quindi non distingue tra maiuscole e minuscole. Ad esempio, un'etichetta come `databus` è considerata identica a `Databus`, `DataBus` o `DATABUS`. Tutte queste varianti fanno riferimento allo stesso simbolo e sono intercambiabili.

#### NOMI ED ETICHETTE

Per essere validi, i nomi e le etichette in VHDL devono rispettare alcune regole:

- Devono iniziare con un carattere alfabetico (A-Z o a-z).
- Possono includere caratteri alfabetici, numerici (0-9) e l'underscore `_`.
- Non sono consentiti underscore multipli consecutivi.
- Non possono includere caratteri di punteggiatura (`!?.&+`, ecc.), poiché questi sono riservati.
- Ogni nome o etichetta deve essere univoco all'interno della stessa entity o architecture.

#### FORMATTAZIONE DEL CODICE

In VHDL, non ci sono regole convenzionali obbligatorie per la formattazione del codice, ma è buona pratica essere ordinati e, idealmente, mantenere un file separato per ogni entity. La leggibilità è migliorata rispettando spaziature e indentazioni coerenti.

#### COMMENTI

I commenti in VHDL iniziano con

`--` e si estendono fino alla fine della riga. Non esistono commenti a blocco; i commenti devono essere brevi e utilizzati solo quando strettamente necessari per chiarire il codice. Alcuni esempi:

```
-- Questo è il sub-circuito principale  
Data_in <= Data_bus; -- lettura dal FIFO
```

## Esempio Programma VHDL

### NAND

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;
```

```
ENTITY nand_gate_nv1 IS  
    PORT ( a: IN BIT;  
           b: IN BIT;  
           z: OUT BIT );  
END nand_gate_nv1;
```

} COS'È

```
ARCHITECTURE behavior OF nand_gate_nv1 IS  
BEGIN  
    z <= a NAND b;  
END behavior;
```

} COSA FA

### SPIEGAZIONE DEL CODICE

#### 1. VHDL e Ritardi di Propagazione

VHDL è un linguaggio di descrizione hardware che non incorpora informazioni sui ritardi di propagazione. Ciò significa che, in questa rappresentazione, il linguaggio descrive solo il funzionamento logico del circuito e non tiene conto dei tempi di ritardo con cui il segnale si propaga nei circuiti fisici.

#### 2. Introduzione alle Librerie

La sezione iniziale del codice carica le librerie necessarie per la simulazione:

- `LIBRARY ieee;` specifica la libreria IEEE standard per il progetto.
- `USE ieee.std_logic_1164.ALL;` include tutti gli elementi della libreria `std_logic_1164`, utilizzata per operazioni logiche standard.

#### 3. Entity (Definizione Strutturale)

La porzione compresa tra `ENTITY` e `END` rappresenta la struttura del circuito, ovvero "cos'è" il circuito.

In questo esempio, l'`ENTITY` definisce i seguenti segnali:

- `a` e `b` come ingressi di tipo `BIT`
- `z` come uscita di tipo `BIT`

Questo blocco stabilisce i collegamenti di input e output per la porta NAND.

#### 4. Architecture (Descrizione Comportamentale)

La parte compresa tra `ARCHITECTURE` e `END behavior` rappresenta la descrizione comportamentale del circuito, ovvero "cosa fa" il circuito.

- La `ARCHITECTURE behavior OF nand_gate_nvl IS` contiene il corpo della logica.
- La linea `z <= a NAND b;` specifica che l'uscita `z` è il risultato dell'operazione NAND tra gli ingressi `a` e `b`.

Questo codice descrive quindi un circuito logico semplice, che calcola l'uscita `z` come il NAND degli ingressi `a` e `b`.

## Dichiarazioni

### LIBRARY DECLARATION

**Library Declaration** importa librerie esterne necessarie per il codice VHDL, fornendo accesso a tipi di dati e pacchetti standard, come `ieee.std_logic_1164`.

È posizionata all'inizio del file, prima della definizione dell'entity e dell'architecture.

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;
```

### ENTITY DECLARATION

**L'Entity Declaration** definisce l'interfaccia del componente hardware, specificando i segnali di ingresso e uscita.

Ogni entity è seguita da un'**Architecture** che ne descrive il comportamento.

```
ENTITY example_entity IS  
    PORT (  

```

```

        a: IN BIT;           -- Ingressi
        b: IN BIT;           -- Ingressi
        z: OUT BIT           -- Uscita
    );
END example_entity;

```

### INPUT MODE: IN

I segnali dichiarati con

**IN** sono ingressi per l'entity. Possono essere letti all'interno del circuito, ma non possono essere modificati.

```

a: IN BIT;

```

### OUTPUT MODE: OUT

I segnali dichiarati con

**OUT** sono uscite dall'entity. Non possono essere letti all'interno dell'entity stessa; possono solo essere scritti.

```

z: OUT BIT;

```

### OUTPUT MODE: OUT con un segnale extra

Se è necessario leggere il valore di un'uscita, si può utilizzare un segnale interno per memorizzare il valore che sarà poi assegnato all'output.

```

signal temp_z: BIT;
z <= temp_z; -- Assegnazione dell'uscita

```

### INPUT BUFFER: BUFFER

L'output dichiarato come

**BUFFER** può essere letto all'interno dell'entity e può anche essere utilizzato come un segnale intermedio per assegnare valori. Si differenzia da **OUT** perché permette la lettura.

```

z: BUFFER BIT;

```

### BIDIRECTIONAL MODE: INOUT

I segnali dichiarati come **INOUT** permettono di leggere e scrivere un valore. Questa modalità è utilizzata quando si desidera che il segnale possa operare

sia come ingresso che come uscita.

È utile, ad esempio, per segnali di dati che possono essere inviati e ricevuti.

```
data_bus: INOUT BIT;
```

## ARCHITECTURE DECLARATION

L'**Architecture Declaration** fornisce la descrizione del comportamento e della struttura interna di un'entity definita in VHDL. Può contenere dettagli riguardanti la logica, il flusso di dati e come i segnali sono gestiti all'interno dell'entity.

In VHDL, ci sono due tipi principali di architettura:

- **Comportamentale:** Descrive cosa fa il circuito, senza dettagli sulla sua implementazione fisica.
- **Strutturale:** Descrive come è composto il circuito, includendo le interconnessioni tra le entità.

```
ARCHITECTURE behavior OF example_entity IS  
BEGIN  
    -- Comportamento del circuito  
END behavior;
```

## std\_logic

Il tipo `BIT` è limitato a due valori logici: **'0' (basso)** e **'1' (alto)**. Tuttavia, nel contesto della progettazione digitale, ci sono situazioni in cui è necessario rappresentare condizioni più complesse che non possono essere catturate semplicemente usando questi due valori. Per questo motivo, si raccomanda di utilizzare `std_logic` per le porte delle entità in VHDL. Questo tipo di dato non solo permette di rappresentare i valori logici '0' e '1', ma offre anche una varietà di stati aggiuntivi, come indeterminatezza e alta impedenza, che sono fondamentali per descrivere comportamenti più complessi nei circuiti digitali.

### DIFFERENZA std\_logic & std\_ulogic

#### std\_logic:

È un tipo di dato che rappresenta un singolo bit e può assumere 9 stati distinti. È progettato per essere utilizzato in situazioni in cui più stati devono essere

rappresentati, come in circuiti complessi.

### **std\_ulogic:**

È simile a `std_logic`, ma rappresenta un singolo bit con una restrizione in più: può assumere solo uno stato alla volta. Non supporta la condizione di alta impedenza ('Z') e le condizioni indeterminate ('X'). È generalmente utilizzato in contesti dove è necessario un controllo più rigoroso sugli stati del segnale.

### **VALORI SPECIALI DI std\_logic**

- **"X" Value (Indeterminato):** Rappresenta uno stato non definito.
- **"Z" Value (Alta impedenza):** Indica che la linea non è pilotata (tri-state).
- **Valori Forti e Deboli:**
  - **H (High):** Alta resistenza.
  - **L (Low):** Bassa resistenza.
- **"-" Don't Care:** Il valore 'don't care' ('-') viene utilizzato nella sintesi logica per assegnare un'etichetta di irrilevanza al valore logico che la funzione da sintetizzare può assumere in corrispondenza a specifici valori di input. Questo valore è utile per ottimizzare il costo della sintesi in termini di utilizzo delle risorse, come ad esempio nella copertura delle mappe di Karnaugh.

### **WIRES AND BUS**

Nel contesto di VHDL, i **wires** (filamenti) e le **buses** (bus) rappresentano strutture di interconnessione fondamentali per la comunicazione tra vari elementi all'interno di un circuito.

- **Wires:** I fili sono utilizzati per trasmettere segnali singoli. Possono essere utilizzati per connettere porte o segnali all'interno di un'entità, facilitando il passaggio di un singolo valore logico, come un `std_logic` o un `std_logic_vector`.
- **Buses:** I bus, invece, sono utilizzati per trasmettere più segnali contemporaneamente. Rappresentano un gruppo di fili che possono trasmettere un insieme di valori logici attraverso un'unica connessione.

### **COSTANTI NUMERICHE**

Quando si assegnano valori costanti ai segnali in VHDL, è importante distinguere tra la notazione per i fili e quella per i bus:

- **Costanti Singole:** Per assegnare una costante a un wire di tipo `std_logic`, si utilizzano le **virgolette singole**. Ad esempio:

```
my_wire <= '1'; -- Assegna il valore logico alto al wire
```

- **Costanti Multiple:** Per assegnare costanti a un bus di tipo `std_logic_vector`, si utilizzano le **virgolette doppie**. Ad esempio:

```
my_bus <= "11001010"; -- Assegna un valore binario al bus
```

## std\_logic\_vector

Il tipo `std_logic_vector` è un array di elementi di tipo `std_logic` e permette di rappresentare un gruppo di bit, consentendo una gestione più efficiente e strutturata dei dati. Può essere utilizzato per rappresentare bus di dati, indirizzi o altri segnali che richiedono più di un singolo bit. Ad esempio, un `std_logic_vector(7 downto 0)` rappresenta un bus di 8 bit.

### DOWN vs TO

Questa distinzione si riferisce all'orientamento del vettore, ovvero quale bit è considerato il più significativo (MSB, Most Significant Bit) e quale è il meno significativo (LSB, Least Significant Bit).

- **Down to:** La sintassi `downto` viene utilizzata per definire un vettore in cui il bit più significativo si trova all'indice più alto. Ad esempio

```
signal my_vector : std_logic_vector(7 downto 0); -- a <= "00000001"; con 1 in posizione meno significativa, ovvero a = 1;
```

- **To:** La sintassi `to` è l'opposto, dove il bit più significativo si trova all'indice più basso. Ad esempio:

```
signal my_vector : std_logic_vector(0 to 7); --a <= "00000001"; con 1 in posizione più significativa, ovvero a = 128;
```

## OPERATORE CONCATENATO

Il **concatenation operator** in VHDL è rappresentato dall'operatore `&`. Viene utilizzato per unire due o più segnali o vettori in un unico vettore. Questo è particolarmente utile quando si desidera combinare dati provenienti da diverse fonti in un'unica rappresentazione.

Ecco un esempio di utilizzo dell'operatore di concatenazione:

```
signal a : std_logic_vector(3 downto 0); -- 4 bit
signal b : std_logic_vector(3 downto 0); -- 4 bit
signal c : std_logic_vector(7 downto 0); -- 8 bit

c <= a & b; -- Unisce i vettori a e b in c
```

## Signal vs Variable

### VARIABLE

Le variabili in VHDL hanno una semantica simile a quella dei linguaggi di programmazione come il C e servono a memorizzare valori utilizzati per l'elaborazione. **È importante notare che non producono hardware;** le variabili mantengono informazioni su cui è possibile scrivere e leggere valori, ma non generano un circuito fisico.

### SIGNAL

I segnali in VHDL, al contrario, trasmettono informazioni circuitali e producono hardware. **I segnali creano un registro fisico che conserva informazioni e generano circuiti reali.** Questa distinzione è fondamentale nella progettazione VHDL, poiché le variabili e i segnali vengono utilizzati in modi diversi per rappresentare e gestire le informazioni nel design hardware

## Sequential and Concurrent Statements

### SEQUENTIAL STATEMENTS

Le istruzioni sequenziali specificano l'ordine in cui devono essere eseguiti i passaggi dell'algoritmo per ottenere un'elaborazione specifica. Queste istruzioni funzionano come in linguaggi di programmazione come il C, dove l'ordine delle operazioni è cruciale. Le istruzioni sequenziali possono trovarsi solo all'interno di un processo, che è un blocco di codice che definisce un algoritmo sequenziale. Le operazioni all'interno di un processo vengono eseguite una dopo l'altra, producendo un risultato finale.

## ESEMPIO

```
PROCESS (a, b, c)
```

```
VARIABLE aandb, orc: BIT;
```

```
BEGIN
```

```
    aandb := a AND b;
```

```
    orc := aandb OR c;
```

```
    d <= orc;
```

```
END PROCESS;
```

```
e <= d AND a;
```

VARABILI x ELABORAZIONI, non

SONO SEGNALI

→ VARABILI MANUTENUTE

è e d non vanno modificati, d è

un segnale dell'entity, E è il risultato

In questo esempio, i valori prodotti dal processo vengono annotati e messi in uscita alla successiva esecuzione. Dal punto di vista dell'hardware, un processo è una dichiarazione che descrive il comportamento dell'hardware da realizzare.

Definisco inoltre,

**sensitivity list : "(a, b, c)"** ovvero, determina quali segnali devono attivare la riesecuzione del processo. Ad esempio, se un segnale nella sensitivity list cambia, il processo verrà rieseguito, aggiornando i valori in base alle nuove condizioni.

## CONCURRENT STATEMENTS

Le istruzioni concorrenti, invece, descrivono la struttura di una porzione di circuito e specificano elaborazioni hardware che evolvono simultaneamente. Queste operazioni vengono descritte in modo tale da non richiedere un ordine specifico di esecuzione, a differenza delle istruzioni sequenziali. In altre parole, i segnali e le connessioni tra i vari componenti vengono aggiornati in modo concorrente

# Sintesi in VHDL

## VHDL per sistemi di Sintesi

In VHDL, non tutte le parti di un progetto ammettono sintesi, ovvero la conversione del codice VHDL in hardware reale. La sintesi è limitata a un sottoinsieme specifico dei tipi e delle strutture del linguaggio, che permette la creazione di circuiti digitali fisici basati sul design descritto.

## Tipologie di Sintesi

### TIPI CHE AMMETTONO SINTESI

I tipi seguenti sono compatibili con la sintesi e possono essere utilizzati nella progettazione di circuiti hardware:

#### 1. Tipi Enumerati:

- **bit**: rappresenta due valori logici, '0' e '1'.
- **boolean**: rappresenta valori logici come true e false.
- **std\_logic** e **std\_ulogic**: rappresentano più valori logici e stati come '0', '1', 'Z' (alta impedenza) e 'X' (non definito).
- **character**: per rappresentare caratteri ASCII (usato raramente in sintesi, ma supportato).

#### 2. Tipi Numerici:

- **integer**: supporta numeri interi.
- **natural**: rappresenta numeri interi non negativi.
- **positive**: include solo valori interi positivi.

#### 3. Array:

- Gli array ammettono sintesi se hanno confini statici definiti, ovvero lunghezza fissa. Ad esempio, un vettore definito come `std_logic_vector(7 downto 0)`.

#### 4. Sottotipi:

- I sottotipi sono ammessi se il loro range è definito come una sottoinsieme di valori di tipo enumerato, come `bit` o `std_logic`. Ad esempio,

```
subtype my_bit is std_logic range '0' to '1';.
```

## TIPI CHE NON AMMETTONO SINTESI

Non tutti i tipi in VHDL possono essere tradotti in hardware. Ad esempio, Access Type, ovvero puntatori a memoria, e File, adatti solo alla simulazione.

## Statement Concorrenziali

I *concurrent statements* in VHDL descrivono operazioni che avvengono simultaneamente, anziché seguire una sequenza temporale come accade negli statement sequenziali. Questa caratteristica rende i statement concorrentiali particolarmente adatti per descrivere comportamenti hardware in cui più segnali possono cambiare contemporaneamente.

## Conditional Statement

I *conditional statements* vengono utilizzati per assegnare valori ai segnali in base a determinate condizioni. Si basano su espressioni logiche che controllano il flusso delle assegnazioni e consentono di implementare, ad esempio, multiplexer e altre funzioni condizionali. I conditional statements vengono spesso scritti con la sintassi `WHEN...ELSE`.

### ESEMPIO MUX41

```
library IEEE;
use IEEE.std_logic_1164.all;
ENTITY mux2 is
  PORT(a    : IN std_logic;
        b    : IN std_logic;
        sel  : IN std_logic;
        y    : OUT std_logic);
END mux2;
ARCHITECTURE behavior OF mux2 IS
  BEGIN
    y <= a WHEN (sel = '0') ELSE
        b WHEN (sel = '1') ELSE
        'X';
  END behavior;
```

## Selection Statements

I *selection statements* sono simili ai conditional statements, ma si usano quando si devono gestire più condizioni (o "casi") per un singolo segnale di controllo. La sintassi più comune è `WITH...SELECT`, che funziona come un multiplexer a più vie. Qui ogni condizione è associata a un valore specifico, come accade per una selezione di input in un circuito multiplexe

## ESEMPIO MUX21

```
library IEEE;
use IEEE.std_logic_1164.all;
ENTITY mux4 is
  PORT(a    : IN std_logic;
        b    : IN std_logic;
        c    : IN std_logic;
        d    : IN std_logic;
        sel  : IN std_logic_vector(1 DOWNTO 0);
        y    : OUT std_logic);
END mux4;
ARCHITECTURE behavior OF mux4 IS
  BEGIN
    WITH sel SELECT
      y <= a WHEN "00",
          b WHEN "01",
          c WHEN "10",
          d WHEN "11",
      'X' W
```

## Operatori

### ARITMETICI

Quando si includono le librerie numeriche ( `numeric_std` è la più comune), VHDL supporta vari operatori aritmetici che consentono di eseguire operazioni come somma, sottrazione, moltiplicazione, divisione e altro. Gli operatori principali sono:

- `abs` (valore assoluto),
- `+` (somma),
- (sottrazione),

- `*` (moltiplicazione),
- `/` (divisione),
- `rem` (resto della divisione),
- `mod` (modulo).

### Esempio: Divisione tra numeri

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY divider IS
    PORT (
        divisor   : IN unsigned(1 DOWNTO 0);
        dividend  : IN unsigned(1 DOWNTO 0);
        quotient   : OUT unsigned(1 DOWNTO 0)
    );
END divider;

ARCHITECTURE behavior OF divider IS
BEGIN
    quotient <= dividend / divisor;
END behavior;
```

In questo esempio, la porta `quotient` riceve il valore risultante dalla divisione di `dividend` per `divisor`.

### RELAZIONALI

Gli operatori di relazione permettono il confronto tra segnali numerici e includono:

- `>`, `<`, `>=`, `<=` per confronti di maggiore, minore, maggiore o uguale, minore o uguale;
- `=`, `/=` per verificare l'uguaglianza o disuguaglianza.

### Esempio: Confronto tra numeri

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY compare IS
  PORT (
    a      : IN unsigned(3 DOWNT0 0);
    b      : IN unsigned(3 DOWNT0 0);
    aleb   : OUT boolean
  );
END compare;

ARCHITECTURE behavior OF compare IS
BEGIN
  aleb <= (a <= b);
END behavior;

```

In questo esempio, `aleb` sarà `true` se `a` è minore o uguale a `b`, e `false` altrimenti.

## SHIFT & CONVERSIONE

Gli operatori di shift consentono di spostare i bit a sinistra o a destra in un vettore. Alcuni di questi operatori sono:

- `shift_left`, `shift_right` per spostamenti a sinistra o a destra,
- `rotate_left`, `rotate_right` per rotazioni a sinistra o a destra,
- `resize` per ridimensionare un vettore,
- `to_integer`, `to_unsigned` per convertire tra tipi.

### Esempio: Shift a sinistra

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY shift_4 IS
  PORT (
    a : IN unsigned(3 DOWNT0 0);

```

```

    b : IN unsigned(1 DOWNTO 0);
    y : OUT unsigned(3 DOWNTO 0)
);
END shift_4;

ARCHITECTURE behavior OF shift_4 IS
BEGIN
    y <= shift_left(a, to_integer(b));
END behavior;

```

## Statement Sequenziali

Gli statement sequenziali specificano il comportamento algoritmico che l'hardware deve implementare e sono usati per definire una sequenza ordinata di operazioni. In un contesto di programmazione, sono simili a quelli di linguaggi come il C, dove l'ordine delle istruzioni è cruciale per ottenere il risultato corretto.

In VHDL, gli statement sequenziali:

- Possono essere inclusi solo all'interno di un costrutto **PROCESS**, il quale ne controlla l'esecuzione.
- Definiscono il flusso di esecuzione sequenziale delle operazioni per produrre un risultato.

## Il Process

Un **PROCESS** rappresenta un blocco che esegue una sequenza di operazioni, implementando un comportamento algoritmico nel progetto hardware. Il Process deve includere una delle seguenti due opzioni:

1. **Lista di Sensibilità**: Definisce i segnali che, cambiando stato, attiveranno la riesecuzione del Process.
2. **Statement WAIT**: Specifica una condizione per attendere che un evento (ad esempio un segnale) si verifichi.

Solo alcuni statement **WAIT** ammettono sintesi. In particolare, **WAIT UNTIL** è utilizzato per implementare condizioni che si riferiscono a specifici cambiamenti di segnale, come ad esempio:

- `WAIT UNTIL input1 = '1';` – attende che il segnale `input1` assuma il valore logico `1`.
- `WAIT UNTIL clock'EVENT AND clock = '1';` – attende un evento di fronte (rising edge) sul segnale di clock, quindi `clock` passa da `0` a `1`.

Nota: `clock'EVENT` e `clock = '1'` sono utilizzati insieme per garantire che le operazioni vengano eseguite solo sul fronte di salita del segnale `clock`, importante per la sincronizzazione in molti circuiti digitali.

### ESEMPIO DI PROCESS CON LISTA DI SENSIBILITA'

Nel seguente esempio, `sig1` e `sig2` sono segnali intermedi che vengono calcolati e usati per definire il valore di `y` all'interno del `PROCESS`.

```
library IEEE;
use IEEE.std_logic_1164.all;

ENTITY aoi_process IS
  PORT(
    a : IN std_logic;
    b : IN std_logic;
    c : IN std_logic;
    y : OUT std_logic
  );
END aoi_process;

ARCHITECTURE behavior OF aoi_process IS
  SIGNAL sig1, sig2 : std_logic;
BEGIN
  comb : PROCESS(a, b, c, sig1, sig2) -- Lista di Sensibilit 
  BEGIN
    sig1 <= a AND b;                -- Primo statement sequenziale
    sig2 <= c OR sig1;              -- Secondo statement sequenziale
    y <= NOT sig2;                  -- Terzo statement sequenziale
  END PROCESS comb;
END behavior;
```

## ESEMPIO DI PROCESS CON LISTA DI SENSIBILITA' INCOMPLETA

Una *Incomplete Sensitivity List* si verifica quando la lista di sensibilità di un `PROCESS` non include tutti i segnali che influenzano i valori assegnati all'interno del `PROCESS`.

```
library IEEE;
use IEEE.std_logic_1164.all;
ENTITY aoi_process is
  PORT(a : IN std_logic;
        b : IN std_logic;
        c : IN std_logic;
        y : OUT std_logic);
END aoi_process;
ARCHITECTURE behavior OF aoi_process IS
  SIGNAL sig1 : std_logic;
  BEGIN
    comb : PROCESS(a, b, c) -- Lista di sensibilità incompleta
    BEGIN
      sig1 <= a AND b;
      y <= not(sig1 or c);
    END PROCESS comb;
  END behavior;
```

Nel `PROCESS comb`, la lista di sensibilità include solo i segnali `a`, `b`, e `c`, ma non `sig1`. Tuttavia, il valore di `y` dipende dal valore di `sig1`, che viene calcolato all'interno del `PROCESS` come `a AND b`. Questo significa che, affinché `y` si aggiorni correttamente ogni volta che `sig1` cambia, il `PROCESS` dovrebbe attivarsi anche al cambiamento di `sig1`.

Tuttavia:

- `sig1` viene aggiornato all'interno del `PROCESS` stesso, quindi tecnicamente non può essere aggiunto alla lista di sensibilità.
- La lista di sensibilità deve comunque includere tutti i segnali di input, quindi `a`, `b`, e `c` dovrebbero essere sufficienti in questo caso, se usati correttamente.

## Assegnazioni nei Process

All'interno dei costrutti `PROCESS` di VHDL, le assegnazioni sequenziali consentono la definizione del comportamento algoritmico che verrà implementato in hardware. Tra le strutture di controllo del flusso, sono ammesse le seguenti costruzioni:

- **IF:**

Il costrutto

`IF` permette di specificare blocchi di codice condizionali. In VHDL, si utilizza per descrivere decisioni logiche che influenzano i segnali di uscita.

```
library IEEE;
use IEEE.std_logic_1164.all;

ENTITY xor_process IS
    PORT(a : IN std_logic;
          b : IN std_logic;
          y : OUT std_logic);
END xor_process;

ARCHITECTURE behavior OF xor_process IS
BEGIN
    comb : PROCESS(a, b)                                -- Esempio XOR
    BEGIN
        IF ((a = '1' AND b = '0') OR
            (a = '0' AND b = '1')) THEN
            y <= '1';
        ELSE
            y <= '0';
        END IF;
    END PROCESS comb;
END behavior;
```

- **CASE:**

Il costrutto

`CASE` è utile per selezionare tra diverse alternative sulla base del valore di una variabile. Ogni caso viene gestito singolarmente e questo tipo di costrutto si dimostra particolarmente efficiente per rappresentare selettori, come nel caso di un multiplexer.

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY mux4_process IS
    PORT(a    : IN std_logic;
          b    : IN std_logic;
          c    : IN std_logic;
          d    : IN std_logic;
          sel  : IN std_logic_vector(1 DOWNTO 0);
          y    : OUT std_logic);
END mux4_process;

ARCHITECTURE behavior OF mux4_process IS
BEGIN
    comb : PROCESS(a, b, c, d, sel)
    BEGIN
        CASE sel IS
            WHEN "00" => y <= a;
            WHEN "01" => y <= b;
            WHEN "10" => y <= c;
            WHEN "11" => y <= d;
            WHEN OTHERS => y <= 'X';
        END CASE;
    END PROCESS comb;
END behavior;

```

-- Esempio MUX 4:

- **FOR:**

Il ciclo

**FOR** consente di iterare su un intervallo di valori. In VHDL, i limiti devono essere definiti staticamente per consentire la sintesi hardware. È comunemente utilizzato per operazioni come spostamenti o somma di serie di valori.

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY shift4 IS
    PORT(mode      : IN std_logic;

```

```

        shift_in  : IN std_logic;
        a         : IN std_logic_vector(4 DOWNTO 1);
        y         : OUT std_logic_vector(4 DOWNTO 1);
        shift_out  : OUT std_logic);
END shift4;

ARCHITECTURE behavior OF shift4 IS
    SIGNAL in_temp : std_logic_vector(5 DOWNTO 0);
    SIGNAL out_temp : std_logic_vector(5 DOWNTO 1);
BEGIN
    in_temp(0) <= shift_in;
    in_temp(4 DOWNTO 1) <= a;
    in_temp(5) <= '0';

    comb : PROCESS(mode, in_temp, a)
    BEGIN
        -- Operazione di Shift
        FOR i IN 1 TO 5 LOOP
            IF (mode = '0') THEN
                out_temp(i) <= in_temp(i-1);
            ELSE
                out_temp(i) <= in_temp(i);
            END IF;
        END LOOP;
    END PROCESS comb;

    y <= out_temp(4 DOWNTO 1);
    shift_out <= out_temp(5);
END behavior;

```

Nota: Nei costrutti **CASE**, le clausole **WHEN** non possono includere valori metalogici come **'X'**, in quanto VHDL richiede che ogni possibile valore sia esplicitamente gestito o che **WHEN OTHERS** gestisca tutte le altre condizioni non previste.

## Funzioni e Dichiarazioni

Le dichiarazioni di *Funzioni* e *Procedure* in VHDL consentono di creare moduli di codice riutilizzabili e sintetizzabili, usati per definire e implementare comportamenti specifici. Questi possono essere definiti in un `PACKAGE` o direttamente nella sezione dichiarativa di un' `ARCHITECTURE`, ma devono contenere esclusivamente codice sintetizzabile per essere adatti alla sintesi hardware.

### FUNZIONI

Le funzioni vengono utilizzate per eseguire operazioni e restituire un singolo valore. Accettano variabili come argomenti e devono essere dichiarate con un tipo di ritorno. Ad esempio:

```
FUNCTION majority(in1, in2, in3 : std_logic) RETURN std_logic
    VARIABLE result : std_logic;
BEGIN
    IF((in1 = '1' and in2 = '1') or (in2 = '1' and in3 = '1'))
        result := '1';
    ELSE
        result := '0';
    END IF;
    RETURN result;
END majority;
```

In questo caso, la funzione

`majority` calcola un valore di maggioranza per tre segnali `std_logic`.

### PROCEDURE VHDL

Le procedure eseguono operazioni che non richiedono il ritorno di un singolo valore ma possono modificare variabili e segnali passati tramite parametri `IN` e `OUT`. Le procedure sono spesso utilizzate per configurazioni più complesse o per impostare stati multipli nei segnali. Ad esempio:

```
PROCEDURE decode(SIGNAL input : IN std_logic_vector(1 DOWNTOW 0)
    SIGNAL output : OUT std_logic_vector(3 DOWNTOW 0)
BEGIN
    CASE input IS
        WHEN "00" => output <= "0001";
        WHEN "01" => output <= "0010";
```

```

        WHEN "10" => output <= "0100";
        WHEN "11" => output <= "1000";
        WHEN OTHERS => output <= "XXXX";
    END CASE;
END decode;

```

## DIFFERENZE TRA FUNZIONI E PROCEDURE

- **Valore di Ritorno:** Le funzioni restituiscono sempre un singolo valore, mentre le procedure possono aggiornare diversi segnali di output senza restituire un valore diretto.
- **Sintesi:** Entrambe le strutture devono essere sintetizzabili. Tuttavia, le procedure consentono un controllo maggiore su più segnali contemporaneamente.
- **Usi Tipici:** Le funzioni vengono usate per calcoli o logiche che producono un singolo risultato; le procedure, invece, sono usate per configurare più valori o per rappresentare un blocco di logica più complesso.

## Tri-state Logic e Don't Care Logic

La logica **tri-state** si applica quando è necessario "disabilitare" un segnale, portandolo in stato di alta impedenza ('Z') per evitare conflitti sulla linea di segnale, ad esempio nei bus di dati. Questo si ottiene utilizzando la condizione metalogica 'Z'. Di seguito un esempio di implementazione in VHDL:

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY tri_state4 IS
    PORT(enable : IN std_logic;
          a      : IN std_logic_vector(3 DOWNT0 0);
          y      : OUT std_logic_vector(3 DOWNT0 0));
END tri_state4;

ARCHITECTURE behavior OF tri_state4 IS
BEGIN
    y <= a WHEN (enable = '1') ELSE "ZZZZ"; -- Attiva l'uscita
END behavior;

```

La logica **Don't Care** si applica quando alcune combinazioni di input non sono rilevanti o non influenzano il risultato. In VHDL, i valori metalogici '-' (don't care) e 'X' (unknown) vengono utilizzati per specificare queste condizioni

```
library IEEE;
use IEEE.std_logic_1164.all;

ENTITY not_xor IS
  PORT(a : IN std_logic;
        b : IN std_logic;
        y : OUT std_logic);
END not_xor;

ARCHITECTURE behavior OF not_xor IS
BEGIN
  comb : PROCESS(a, b)
  BEGIN
    IF((a = '1' AND b = '0') OR
       (a = '0' AND b = '1')) THEN
      y <= '1'; -- y è '1' se gli input sono opposti
    ELSE
      y <= 'X'; -- y assume valore indeterminato se gli
    END IF;
  END PROCESS comb;
END behavior;
```

## Esempi di logiche sequenziali

Gli esempi di logiche sequenziali in VHDL mostrano vari tipi di latch e flip-flop per la memorizzazione e la sincronizzazione dei segnali. Ogni componente ha uno specifico comportamento e tempistica di attivazione, che dipende dalla tipologia di segnale di controllo (come `clk`, `pre`, e `clr`)

### D-LATCH

Un **D-Latch** è un circuito che memorizza il valore di `d` (dato) solo quando `clk` (clock) è alto. Il valore di `q` rappresenta l'uscita del latch, mentre `qn` è l'uscita complementare.

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY d_latch IS
  PORT(d    : IN std_logic;
        clk : IN std_logic;
        q    : OUT std_logic;
        qn   : OUT std_logic);
END d_latch;

ARCHITECTURE behavior OF d_latch IS
BEGIN
  seq : PROCESS(d, clk)
  BEGIN
    IF(clk = '1') THEN -- Memorizza 'd' solo quando 'clk
      q <= d;
    END IF;
  END PROCESS seq;
  qn <= NOT q; -- Uscita complementare
END behavior;

```

## MASTER- SLAVE D-LATCH

Il Master-Slave D-Latch è un circuito di memorizzazione composto da due latch in cascata. Il **Master** cattura il valore dell'ingresso **d** quando il clock (**clk**) è alto, ma non lo trasmette immediatamente. Quando il clock si abbassa, il **Slave** acquisisce il valore memorizzato nel Master e lo trasferisce all'uscita **q**. Questo meccanismo riduce i glitch e assicura che l'uscita cambi solo in momenti precisi del ciclo di clock, migliorando la sincronizzazione dei dati.

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY ms_latch IS
  PORT(d    : IN std_logic;
        clk : IN std_logic;
        q    : INOUT std_logic;
        qn   : OUT std_logic);

```

```

END ms_latch;

ARCHITECTURE behavior OF ms_latch IS
    SIGNAL q_int : std_logic; -- Segnale intermedio per la sincronizzazione
BEGIN
    seq1 : PROCESS(d, clk)
    BEGIN
        IF(clk = '1') THEN -- Il primo latch memorizza 'd' quando clk = 1
            q_int <= d;
        END IF;
    END PROCESS seq1;

    seq2 : PROCESS(q_int, clk)
    BEGIN
        IF(clk = '0') THEN -- Il secondo latch trasferisce 'q_int' a 'q' quando clk = 0
            q <= q_int;
        END IF;
    END PROCESS seq2;

    qn <= NOT q; -- Uscita complementare
END behavior;

```

## D-FLIP FLOP CON TRIGGER SUL FRONTE (EDGE-TRIGGERED)

Il D-Flip-Flop con Trigger sul Fronte è un circuito di memorizzazione che cattura il valore dell'ingresso **d** solo durante una transizione specifica del clock, tipicamente sul **fronte di salita** (rising edge). Questo significa che l'uscita **q** cambia solo in corrispondenza del fronte di clock, garantendo un aggiornamento sincrono e riducendo il rischio di glitch.

- Versione con **rising\_edge**

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY d_ff1 IS
    PORT(d      : IN std_logic;
         clk     : IN std_logic;
         q       : INOUT std_logic;
         qn      : OUT std_logic);

```

```

END d_ff1;

ARCHITECTURE behavior OF d_ff1 IS
BEGIN
    seq : PROCESS(clk)
    BEGIN
        IF(rising_edge(clk)) THEN -- Memorizza 'd' solo sul
            q <= d;
        END IF;
    END PROCESS seq;
    qn <= NOT q; -- Uscita complementare
END behavior;

```

- Versione con `clk'event AND clk = 1`

```

ENTITY d_ff2 IS
PORT(d    : IN std_logic;
      clk : IN std_logic;
      q    : INOUT std_logic;
      qn   : OUT std_logic);
END d_ff2;

ARCHITECTURE behavior OF d_ff2 IS
BEGIN
    seq : PROCESS(clk)
    BEGIN
        IF(clk'EVENT AND clk = '1') THEN -- Memorizza 'd' su
            q <= d;
        END IF;
    END PROCESS seq;
    qn <= NOT q; -- Uscita complementare
END behavior;

```

## D-FLIP FLOP CON PRESET E CLEAR ASINCRONI

Il D-Flip-Flop con **Preset** e **Clear Asincroni** è un flip-flop che può impostare (`preset`) o resettare (`clear`) l'uscita `q` in modo indipendente dal clock. Quando i segnali di preset o clear sono attivi, il flip-flop modifica immediatamente l'uscita

senza attendere un fronte del clock, rendendolo ideale per condizioni in cui è necessario forzare rapidamente l'uscita a uno stato definito.

```
ENTITY d_ff_pc IS
  PORT(d      : IN std_logic;
        clk   : IN std_logic;
        pre    : IN std_logic;
        clr    : IN std_logic;
        q      : INOUT std_logic;
        qn     : OUT std_logic);
END d_ff_pc;

ARCHITECTURE behavior OF d_ff_pc IS
BEGIN
  seq : PROCESS(clk, pre, clr)
  BEGIN
    IF(pre = '0') THEN    -- Preset asincrono: imposta 'q'
      q <= '1';
    ELSIF(clr = '0') THEN -- Clear asincrono: resetta 'q'
      q <= '0';
    ELSIF(rising_edge(clk)) THEN -- Altrimenti, memorizza 'd'
      q <= d;
    END IF;
  END PROCESS seq;
  qn <= NOT q; -- Uscita complementare
END behavior;
```

## D-FLIP FLOP CON PRESET E CLEAR SINCRONI

Il D-Flip-Flop con **Preset** e **Clear Sincroni** permette di impostare ( **preset** ) o resettare ( **clear** ) l'uscita **q** solo in corrispondenza del fronte di clock. Questo significa che le operazioni di preset e clear avvengono solo quando si verifica una transizione di clock (tipicamente il fronte di salita), mantenendo così il flip-flop sincronizzato con il segnale di clock e limitando i cambiamenti dell'uscita agli istanti previsti dal clock.

```
ENTITY d_ff_spc IS
  PORT(d      : IN std_logic;
        clk   : IN std_logic;
```

```

    pre : IN std_logic;
    clr : IN std_logic;
    q   : INOUT std_logic;
    qn  : OUT std_logic);
END d_ff_spc;

ARCHITECTURE behavior OF d_ff_spc IS
BEGIN
    seq : PROCESS(clk)
    BEGIN
        IF(rising_edge(clk)) THEN -- Esegue le operazioni so
            IF(pre = '0') THEN -- Preset sincrono: impost
                q <= '1';
            ELSIF(clr = '0') THEN -- Clear sincrono: resetta
                q <= '0';
            ELSE
                q <= d; -- Altrimenti, memorizza '
            END IF;
        END IF;
    END PROCESS seq;
    qn <= NOT q; -- Uscita complementare
END behavior;

```

## Fallacies

Quando si progetta logica combinatoria, l'omissione di alcuni casi nei costrutti `IF` o `CASE` può causare l'inferenza di latch, cioè elementi di memoria non voluti, anche se si intendeva creare solo una logica combinatoria.

### Caso Incompleto

Consideriamo un multiplexer a 3 ingressi (`mux3_seq`). In questo esempio, il segnale `y` è assegnato solo per alcune condizioni del selettore `sel`, mentre la condizione `WHEN OTHERS` è lasciata vuota. Ciò porta all'inferenza di un latch per `y`, poiché il compilatore non sa come gestire tutti i casi di `sel`:

```

library IEEE;
use IEEE.std_logic_1164.all;

```

```

ENTITY mux3_seq IS
  PORT(a    : IN std_logic;
        b    : IN std_logic;
        c    : IN std_logic;
        sel  : IN std_logic_vector(1 DOWNTO 0);
        y    : OUT std_logic);
END mux3_seq;

ARCHITECTURE behavior OF mux3_seq IS
BEGIN
  comb : PROCESS(a, b, c, sel)
  BEGIN
    CASE sel IS
      WHEN "00" => y <= a;
      WHEN "01" => y <= b;
      WHEN "10" => y <= c;
      WHEN OTHERS => -- Caso vuoto
    END CASE;
  END PROCESS comb;
END behavior;

```

Per evitare l'inferenza del latch, è necessario specificare una condizione per il caso `WHEN OTHERS`, utilizzando valori predefiniti (`'-'` o `'X'`), che indicano al compilatore come gestire i casi rimanenti:

```

library IEEE;
use IEEE.std_logic_1164.all;

ENTITY mux3 IS
  PORT(a    : IN std_logic;
        b    : IN std_logic;
        c    : IN std_logic;
        sel  : IN std_logic_vector(1 DOWNTO 0);
        y    : OUT std_logic);
END mux3;

ARCHITECTURE behavior OF mux3 IS
BEGIN

```

```

    comb : PROCESS(a, b, c, sel)
    BEGIN
        CASE sel IS
            WHEN "00" => y <= a;
            WHEN "01" => y <= b;
            WHEN "10" => y <= c;
            WHEN OTHERS => y <= 'X'; -- Evita latch specificand
        END CASE;
    END PROCESS comb;
END behavior;

```

## Contatore con Interferenza Latch

In un altro esempio, un contatore ( `pc_comb1` ) aggiorna il segnale `pc` solo in alcune condizioni di controllo ( `cntrl` ). Senza un'istruzione per tutte le combinazioni possibili del controllo, può verificarsi la generazione di latch, in quanto il segnale `pc` non viene aggiornato per tutti i casi di `cntrl` .

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY pc_comb1 IS
    PORT(data_in : IN unsigned(3 DOWNTO 0);
          cntrl   : IN unsigned(1 DOWNTO 0);
          data_out : OUT unsigned(3 DOWNTO 0));
END pc_comb1;

ARCHITECTURE rtl OF pc_comb1 IS
    SIGNAL pc : unsigned(3 DOWNTO 0);
BEGIN
    one : PROCESS(data_in, cntrl, pc)
    BEGIN
        CASE cntrl IS
            WHEN "01" => pc <= (pc + "0001");
            WHEN "10" => pc <= pc;
            WHEN OTHERS => pc <= data_in; -- Specifica tutti i
        END CASE;
    END PROCESS one;

```

```
data_out <= pc;
END rtl;
```

## Inferenza di Latch nelle Macchine a Stati Finiti Asincrone (FSM)

Quando si definiscono macchine a stati finiti asincrone, l'assenza di specifiche per tutti i possibili stati del segnale di controllo ( `cntrl` ) può comportare la generazione non intenzionale di latch. Qui, specificando il caso `WHEN OTHERS` con un valore come `'X'`, si evita l'inferenza di latch e si assicura che tutti gli stati siano gestiti.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY pc_comb2 IS
  PORT(data_in  : IN unsigned(3 DOWNTO 0);
        cntrl    : IN std_logic;
        data_out : OUT unsigned(3 DOWNTO 0));
END pc_comb2;

ARCHITECTURE rtl OF pc_comb2 IS
  SIGNAL pc : unsigned(3 DOWNTO 0);
BEGIN
  one : PROCESS(data_in, cntrl, pc)
  BEGIN
    CASE cntrl IS
      WHEN '1' => pc <= (pc + "0001");
      WHEN '0' => pc <= data_in;
      WHEN OTHERS => pc <= "XXXX"; -- Gestisce tutti i ca
    END CASE;
  END PROCESS one;
  data_out <= pc;
END rtl;
```

Per evitare l'inferenza non intenzionale di latch nelle logiche combinatorie in VHDL:

- Utilizzare `WHEN OTHERS` per gestire tutti i casi non specificati.

- Assegnare valori predefiniti come 'X' o '0' nei casi non specifici, garantendo così la completezza del codice.
- Assicurarsi che tutte le possibili combinazioni dei segnali di controllo siano coperte.

## Macchine a Stati Finiti

Le **Finite State Machines** (FSM) sono modelli logici che combinano elementi sequenziali e combinatoriali per rappresentare sistemi con un numero finito di stati. Le FSM possono essere implementate attraverso linguaggi di descrizione dell'hardware (HDL) come VHDL, utilizzando tool di sintesi che permettono di concentrarsi sul comportamento della FSM piuttosto che sulla struttura. Questo consente di descrivere FSM in modo più intuitivo e standardizzato. Esistono due principali modelli di FSM: il modello **Mealy** e il modello **Moore**.

### STRUTTURA FSM

Le FSM sono composte da:

- **Stato Presente (Present State):** rappresenta l'attuale condizione della macchina.
- **Input Primari (Primary Inputs):** rappresentano i segnali in ingresso che influenzano il comportamento della FSM.
- **Stato Futuro (Future State):** è il prossimo stato della FSM, determinato dal combinatore di transizione.
- **Uscite Primarie (Primary Outputs):** rappresentano l'output della macchina in base allo stato presente e al tipo di FSM.

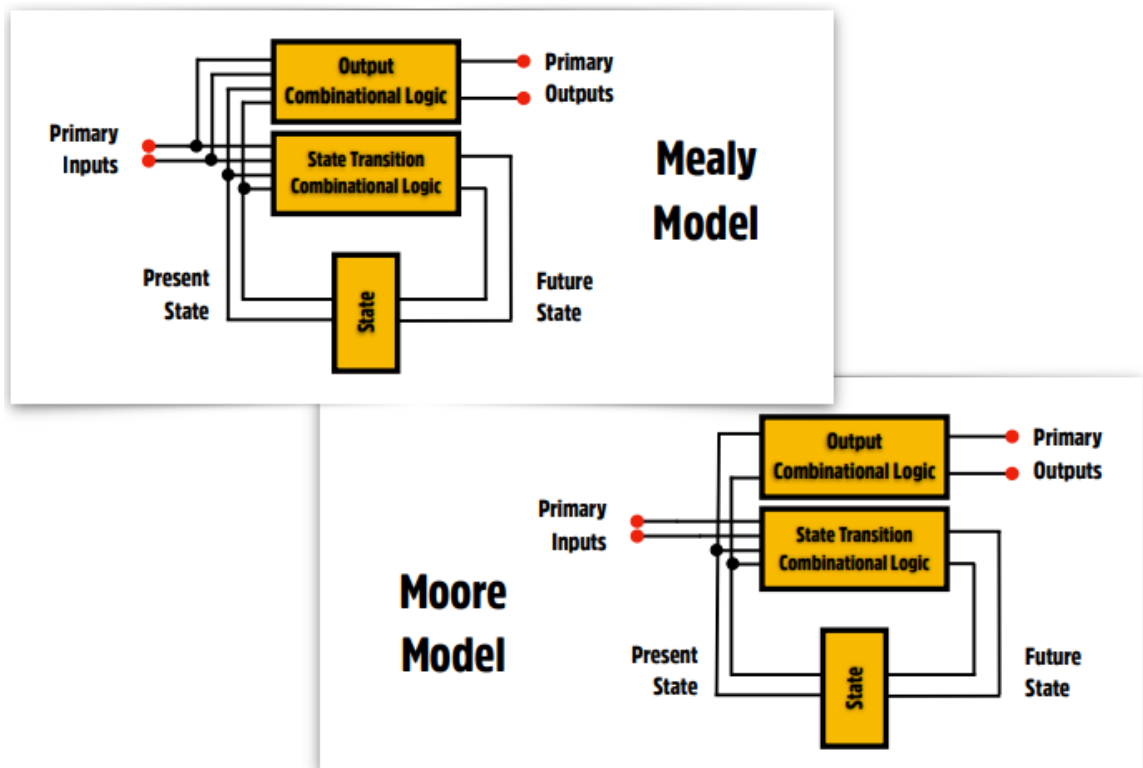
## Modelli di FSM: Mealy e Moore

### 1. Moore FSM:

- In un modello di Moore, le uscite della macchina dipendono solo dallo **stato presente** e non dai **segnali di ingresso**.
- Le uscite cambiano solo quando la FSM passa a un nuovo stato, rendendo il modello stabile e predicibile, utile per implementazioni dove è richiesta una sincronizzazione precisa con il clock.

### 2. Mealy FSM:

- In un modello di Mealy, le uscite dipendono sia dallo **stato presente** che dai **segnali di ingresso**.
- Questo permette alle uscite di cambiare immediatamente con l'input, rendendo la FSM più reattiva ma anche più soggetta a variazioni temporanee (glitches) nei segnali di uscita.



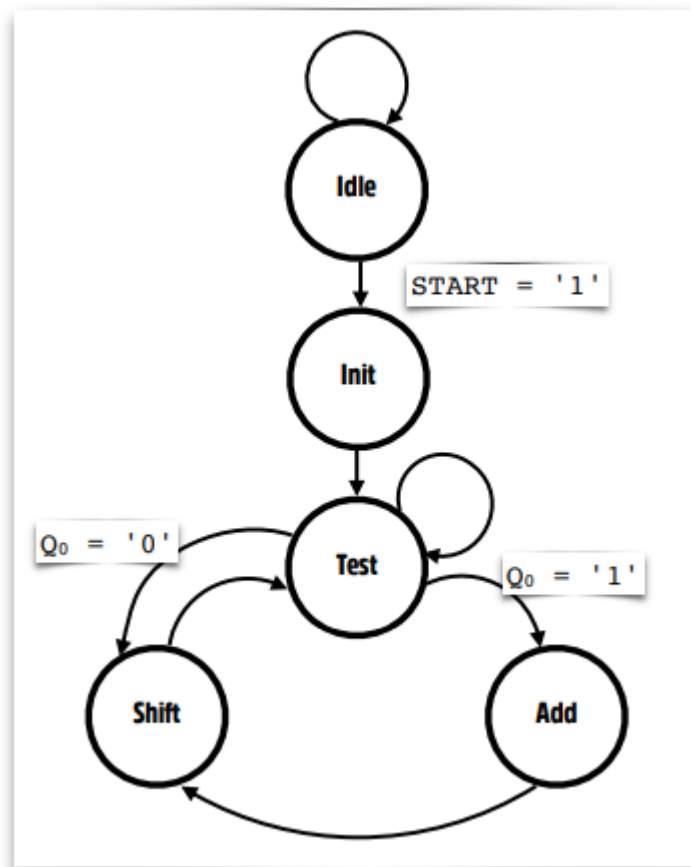
## Codifica degli Stati

La codifica degli stati può essere ottimizzata usando diversi metodi di codifica, in base alla complessità della FSM e alle caratteristiche del tool di sintesi usato. La codifica degli stati può essere indicata direttamente con l'attributo `enum_encoding`, che assegna un codice specifico a ciascuno stato della FSM.

### ESEMPIO:

```
ATTRIBUTE enum_encoding: STRING;
TYPE state_type IS (idle, init, test, add, shift);
ATTRIBUTE enum_encoding OF state_type: TYPE IS "000 100 110 0
```

## Esempio di FSM



## STATO DI RESET

Ogni FSM deve essere resettata al momento dell'avvio. Il reset può essere sincrono o asincrono. Le transizioni tra stati devono avvenire a uno dei due fronti del clock.

```

clocked : PROCESS(clk, reset)
BEGIN
  IF (reset = '0') THEN -- Se il reset è attivo (livello basso)
    present_state <= idle; -- Imposta lo stato iniziale
  ELSIF (clk'EVENT AND clk = '1') THEN -- Al fronte di salita
    present_state <= next_state; -- Passa allo stato successivo
  END IF;
END PROCESS clocked;

```

## TRANSIZIONE DI STATO

Il processo di transizione di stato specifica come la FSM passa da uno stato all'altro in base agli input. Qui il costrutto

**CASE** facilita la descrizione delle condizioni di transizione per ogni stato, il che aiuta gli strumenti di sintesi a ottimizzare il codice.

```
nextstate : PROCESS(present_state, start, q0)
BEGIN
    CASE present_state IS
        WHEN idle =>
            IF(start='1') THEN
                next_state <= init; -- Transizione allo stato "init"
            ELSE
                next_state <= idle; -- Rimane nello stato "idle"
            END IF;

        WHEN init =>
            next_state <= test; -- Transizione automatica allo stato "test"

        WHEN test =>
            IF(q0='1') THEN
                next_state <= add; -- Passa allo stato "add"
            ELSIF(q0='0') THEN
                next_state <= shift; -- Passa allo stato "shift"
            ELSE
                next_state <= test; -- Rimane nello stato "test"
            END IF;

        WHEN add =>
            next_state <= shift; -- Transizione automatica allo stato "shift"

        WHEN shift =>
            next_state <= test; -- Ritorna allo stato "test"
    END CASE;
END PROCESS nextstate;
```

## PROCESSO DI OUTPUT

Il processo di output descrive come i segnali di uscita della FSM sono generati. Nel **modello Moore**, questi dipendono solo dallo stato attuale, mentre nel **modello Mealy** possono dipendere anche dagli input. In questo esempio, si

suggerisce il costrutto `CASE` per chiarezza sintattica, ma è possibile utilizzare anche `IF THEN ELSE`.

```
output : PROCESS(present_state, start, q0)
BEGIN
    -- Imposta valori di default per gli output
    a_enable <= '0' AFTER delay;
    a_mode <= '0' AFTER delay;
    c_enable <= '0' AFTER delay;
    m_enable <= '0' AFTER delay;

    CASE present_state IS
        WHEN init =>
            a_enable <= '1' AFTER delay; -- Abilita `a` nell
            c_enable <= '0' AFTER delay;
            m_enable <= '1' AFTER delay; -- Abilita `m` nell

        WHEN add =>
            a_enable <= '0' AFTER delay;
            c_enable <= '1' AFTER delay; -- Abilita `c` nell
            m_enable <= '0' AFTER delay;

        WHEN shift =>
            a_enable <= '0' AFTER delay;
            a_mode <= '0' AFTER delay;
            m_enable <= '1' AFTER delay; -- Abilita `m` nell

        WHEN OTHERS =>
            NULL; -- Nessun output per gli altri stati
    END CASE;
END PROCESS output;
```

## RIASSUNTO

- **Reset:** Inizializza lo stato della FSM a `idle` sia con reset sincrono che asincrono.
- **Transizione di stato:** Utilizza il costrutto `CASE` per passare da uno stato all'altro in base agli input, garantendo chiarezza e sintesi efficiente.

- **Processo di output:** Definisce i segnali di uscita per ogni stato, rendendo possibile configurare la FSM come Moore (uscite dipendono dallo stato) o Mealy (uscite dipendono da stato e input).

## Esempio FSM con più di una variabile di stato

Nella prima FSM, lo stato era tracciato unicamente dalla variabile `present_state`, che identificava gli stati principali della macchina (`idle`, `init`, `test`, `add`, `shift`). In questa versione, oltre a `present_state`, è presente una seconda variabile, `present_count`, che tiene traccia del numero di cicli completati in uno specifico stato.

Questa seconda variabile viene utilizzata per controllare meglio le transizioni tra stati e per implementare condizioni di uscita più complesse. In particolare,

`present_count` permette di:

- Contare i cicli di clock in uno stato specifico (ad esempio in `test`) fino a un valore predefinito.
- Aggiungere maggiore controllo nelle transizioni, come riportato nello stato `test`, dove `present_count` viene usato per stabilire se la FSM deve ritornare allo stato `idle`.

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY control_unit2 IS
    PORT(clk      : IN std_logic;
          q0      : IN std_logic;
          reset    : IN std_logic;
          start    : IN std_logic;
          a_enable : OUT std_logic;
          a_mode   : OUT std_logic;
          c_enable : OUT std_logic;
          m_enable : OUT std_logic);
END control_unit2;

ARCHITECTURE fsm OF control_unit2 IS
    CONSTANT delay : time := 5 ns;
    TYPE state_type IS (idle, init, test, add, shift);
```

```

SUBTYPE count_type IS integer RANGE 15 DOWNT0 0; -- Defi

SIGNAL present_state, next_state : state_type;
SIGNAL present_count, next_count : count_type; -- Variab

BEGIN

-- Processo clocked con reset e aggiornamento di `present
clocked : PROCESS(clk, reset)
BEGIN
    IF (reset = '0') THEN
        present_state <= idle;
        present_count <= 0; -- Inizializzazione del cont
    ELSIF (clk'EVENT AND clk = '1') THEN
        present_state <= next_state;
        present_count <= next_count; -- Aggiornamento de
    END IF;
END PROCESS clocked;

-- Processo per le transizioni di stato (FSM)
nextstate : PROCESS(present_state, present_count, start,
BEGIN
    next_count <= present_count; -- Mantiene il contatore
    CASE present_state IS
        WHEN idle =>
            IF (start = '1') THEN
                next_state <= init;
            ELSE
                next_state <= idle;
            END IF;

        WHEN init =>
            next_state <= test;

        WHEN test =>
            IF (present_count < 8 AND q0 = '1') THEN
                next_state <= add;
            ELSIF (present_count < 8 AND q0 = '0') THEN

```

```

        next_state <= shift;
    ELSIF (present_count = 7) THEN
        next_state <= idle;  -- Usa il contatore
    ELSE
        next_state <= test;
    END IF;

    WHEN add =>
        next_state <= shift;

    WHEN shift =>
        next_state <= test;
        next_count <= present_count + 1;  -- Incrementa

    END CASE;
END PROCESS nextstate;

-- Processo per definire gli output (FSM)
output : PROCESS(present_state, start, q0)
BEGIN
    a_enable <= '0' AFTER delay;
    a_mode <= '0' AFTER delay;
    c_enable <= '0' AFTER delay;
    m_enable <= '0' AFTER delay;

    CASE present_state IS
        WHEN init =>
            a_enable <= '1' AFTER delay;
            c_enable <= '0' AFTER delay;
            m_enable <= '1' AFTER delay;

        WHEN add =>
            a_enable <= '0' AFTER delay;
            c_enable <= '1' AFTER delay;
            m_enable <= '0' AFTER delay;

        WHEN shift =>
            a_enable <= '0' AFTER delay;

```

```

        a_mode    <= '0' AFTER delay;
        m_enable  <= '1' AFTER delay;

        WHEN OTHERS =>
            NULL;
        END CASE;
    END PROCESS output;

END fsm;

```

## VHDL Strutturato

In VHDL strutturato, la progettazione viene gestita attraverso l'interconnessione di componenti più semplici, come porte logiche o altri blocchi logici definiti in entità separate, per formare un sistema più complesso. Questa tecnica consente un'implementazione modulare e rende il codice più leggibile e facilmente manutenibile, favorendo il riutilizzo di componenti già progettati.

### Struttura del Codice in VHDL Strutturato

#### 1. DEFINIZIONE ENTITA' ELEMENTARI

- **AND2:** Un'entità che rappresenta una porta AND a due ingressi.
- **OR2:** Un'entità che rappresenta una porta OR a due ingressi.
- **INV:** Un'entità che rappresenta un inverter (porta NOT).

Ogni entità ha una propria architettura definita in modo comportamentale. Di seguito sono riportati gli esempi di codice per queste entità:

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY and2 IS
    PORT(a : IN std_logic;
         b : IN std_logic;
         c : OUT std_logic);
END and2;

ARCHITECTURE behav OF and2 IS

```

```
BEGIN
    c <= a AND b;
END behav;
```

Lo stesso principio viene seguito per `or2` e `inv`, dove vengono utilizzate rispettivamente le operazioni `OR` e `NOT` per l'output `c` o `b`.

## 2. PROGETTAZIONE STRUTTURATA DELL'AND-OR-INVERTER (AOI)

- L'obiettivo è realizzare un circuito combinatorio che implementa una funzione `AOI` utilizzando le componenti `and2`, `or2`, e `inv` precedentemente definite.
- Per fare questo, si definisce una nuova entità `aoi1_str` con un'architettura `structural` che collega questi componenti in un circuito più complesso.

## 3. UTILIZZO DI COMPONENTI INTERNI

### MAPPING PER POSIZIONE

Qui viene utilizzata una mappatura per posizione, quindi gli ingressi e uscite di ciascun componente (`a`, `b`, `c`) sono associati direttamente agli `in` e `out` nella stessa posizione.

- In `aoi1_str`, ogni componente (`and2`, `or2`, `inv`) viene dichiarato e inserito nella struttura tramite l'uso di `COMPONENT`.
- La sintassi `FOR ALL` specifica come e dove utilizzare l'architettura `behav` di ciascuna entità (`work.and2`, `work.or2`, `work.inv`).

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY aoi1_str IS
    PORT(a_in : IN std_logic;
         b_in : IN std_logic;
         c_in : IN std_logic;
         d_out : OUT std_logic);
END aoi1_str;

ARCHITECTURE structural OF aoi1_str IS
    COMPONENT and2
```

```

        PORT(a : IN std_logic;
              b : IN std_logic;
              c : OUT std_logic);
    END COMPONENT;
    COMPONENT or2
        PORT(a : IN std_logic;
              b : IN std_logic;
              c : OUT std_logic);
    END COMPONENT;
    COMPONENT inv
        PORT(a : IN std_logic;
              b : OUT std_logic);
    END COMPONENT;

    FOR ALL : and2 USE ENTITY work.and2(behav);
    FOR ALL : or2 USE ENTITY work.or2 (behav);
    FOR ALL : inv USE ENTITY work.inv (behav);

    SIGNAL and_out : std_logic;
    SIGNAL or_out : std_logic;
BEGIN
    AND_1 : and2 PORT MAP (a_in, b_in, and_out);
    OR_1 : or2 PORT MAP (and_out, c_in, or_out);
    INV_1 : inv PORT MAP (or_out, d_out);
END structural;

```

## MAPPING PER NOME

Un altro metodo di collegamento è il **port mapping per nome**, illustrato in `aoi2_str`. Qui, ogni porta viene collegata specificando i nomi, il che rende il codice più esplicito e aiuta a evitare errori di posizionamento.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY aoi2_str IS
    PORT(a_in : IN std_logic;
          b_in : IN std_logic;
          c_in : IN std_logic;

```

```

        d_out : OUT std_logic);
END aoi2_str;

ARCHITECTURE structural OF aoi2_str IS
    COMPONENT and2
        PORT(a : IN std_logic;
              b : IN std_logic;
              c : OUT std_logic);
    END COMPONENT;
    COMPONENT or2
        PORT(a : IN std_logic;
              b : IN std_logic;
              c : OUT std_logic);
    END COMPONENT;
    COMPONENT inv
        PORT(a : IN std_logic;
              b : OUT std_logic);
    END COMPONENT;

    FOR ALL : and2 USE ENTITY work.and2(behav);
    FOR ALL : or2 USE ENTITY work.or2 (behav);
    FOR ALL : inv USE ENTITY work.inv (behav);

    SIGNAL and_out : std_logic;
    SIGNAL or_out : std_logic;
BEGIN
    INV_1 : inv PORT MAP (b => d_out, a => or_out);
    OR_1 : or2 PORT MAP (c => or_out, a => and_out, b => c);
    AND_1 : and2 PORT MAP (c => and_out, a => a_in, b => b_in);
END structural;

```

## Flattering nella Progettazione VHDL

Il **flattering** è una tecnica in cui il codice viene appiattito, ovvero rappresentato a un livello gerarchico inferiore. In pratica, le istanze dei componenti vengono "espanse" in modo che il design possa essere rappresentato senza una struttura gerarchica interna, migliorando l'ottimizzazione durante la sintesi del circuito.

# Test Bench

Il **Test Bench** in VHDL è uno strumento utilizzato per verificare la funzionalità e la correttezza di un circuito digitale simulato. In pratica, un test bench simula l'ambiente esterno in cui opererà il circuito, generando i segnali di ingresso e monitorando le risposte in uscita per verificare che il circuito si comporti come previsto.

## Struttura del Test Bench

Nel contesto del test bench, si incontrano alcuni componenti chiave:

1. **Unit Under Test (UUT):** È il circuito o la componente che si desidera testare, in questo caso, l'entity `aoi2_str`.
2. **Test Vector Generator (TVG):** Un modulo che genera una sequenza di segnali di test, simulando i vari casi che il circuito dovrà gestire. In questo esempio, `tvb_bhv` genera vari vettori di ingresso per testare `aoi2_str`.

## ESEMPIO DI TEST BECH

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- Entità del test bench (non ha porte di input/output perché
ENTITY testbench IS
END testbench;

ARCHITECTURE structural OF testbench IS
    -- Componenti utilizzati per la simulazione
    COMPONENT aoi2_str IS
        PORT (a_in : IN std_logic;
              b_in : IN std_logic;
              c_in : IN std_logic;
              d_out : OUT std_logic);
    END COMPONENT;

    COMPONENT tvb_bhv IS
        PORT (a_tv : OUT std_logic;
              b_tv : OUT std_logic;
              c_tv : OUT std_logic;
```

```

        d_tv : IN std_logic);
    END COMPONENT;

    -- Segnali interni per collegare il generatore di vettori
    SIGNAL a, b, c, d : std_logic;

BEGIN
    -- Mappatura dei componenti del test bench
    TVG : tvg_bhv PORT MAP (d_tv => d, b_tv => b, c_tv => c,
    UUT : aoi2_str PORT MAP (d_out => d, b_in => b, c_in => c
    END structural;

```

## Codice del Test Vector Generator (TVG)

Il modulo `tvg_bhv` è responsabile di generare diversi set di ingressi (`a_tv`, `b_tv`, `c_tv`) e di confrontare l'uscita con i risultati attesi. In caso di errore, viene segnalata un'anomalia con un messaggio.

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY tvg_bhv IS
    PORT (a_tv : OUT std_logic;      -- Uscita del test vector
          b_tv : OUT std_logic;      -- Uscita del test vector
          c_tv : OUT std_logic;      -- Uscita del test vector
          d_tv : IN std_logic);      -- Ingresso del risultato
END tvg_bhv;

ARCHITECTURE behavioral OF tvg_bhv IS
    SIGNAL vector : std_logic_vector(2 DOWNTO 0); -- Vettore
    SIGNAL result : std_logic;                    -- Risultato

BEGIN
    -- Associazione dei segnali di test agli ingressi
    a_tv <= vector(0);
    b_tv <= vector(1);
    c_tv <= vector(2);

    -- Processo per generare i vettori di test e controllare

```

```

stimula : PROCESS
BEGIN
    -- Applicazione di vari vettori di test
    vector <= "000"; WAIT FOR 10 ns;
    IF (d_tv /= '1') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "001"; WAIT FOR 10 ns;
    IF (d_tv /= '1') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "010"; WAIT FOR 10 ns;
    IF (d_tv /= '1') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "011"; WAIT FOR 10 ns;
    IF (d_tv /= '0') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "100"; WAIT FOR 10 ns;
    IF (d_tv /= '0') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "101"; WAIT FOR 10 ns;
    IF (d_tv /= '0') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "110"; WAIT FOR 10 ns;
    IF (d_tv /= '0') THEN ASSERT FALSE REPORT "Errore: ri

    vector <= "111"; WAIT FOR 10 ns;
    IF (d_tv /= '0') THEN ASSERT FALSE REPORT "Errore: ri

    WAIT; -- Mantiene la simulazione attiva
END PROCESS stimula;
END behavioral;

```

In questo processo:

- **Sequenza di Vettori di Test:** Il codice imposta vari ingressi binari nel segnale `vector`, rappresentando i diversi stati di ingresso (`a`, `b`, `c`). Ogni combinazione è applicata per un intervallo di 10 ns.
- **Controllo dell'Uscita:** Dopo ogni vettore, il codice verifica se l'uscita `d_tv` è corretta. Se l'uscita differisce dal valore atteso, viene segnalato un errore con un messaggio (`ASSERT`).

- **Aspetto della Tempistica:** Ogni test aspetta 10 ns per permettere al circuito di aggiornare l'uscita prima di verificare la condizione.

## Riassunto test bench

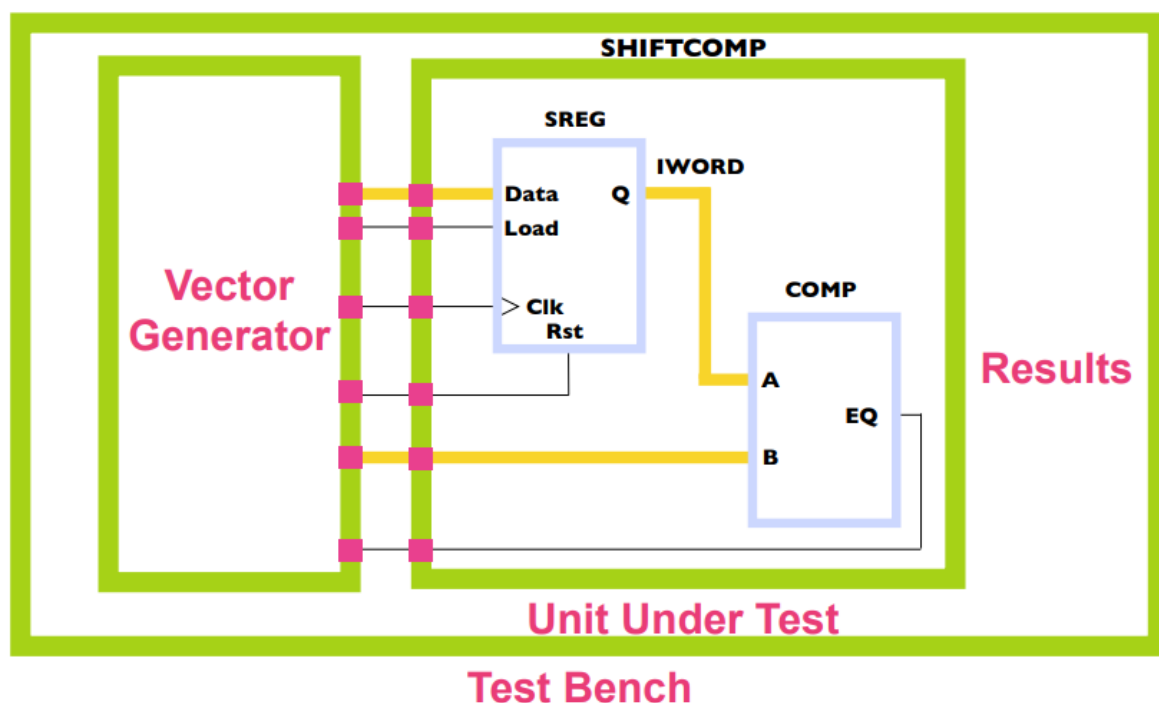
Il test bench permette di:

1. **Verificare la Logica del Circuito:** Testando l'UUT con vari scenari di ingresso per verificarne il comportamento.
2. **Automatizzare il Confronto con i Risultati Attesi:** Grazie alle istruzioni `ASSERT`, ogni errore è segnalato automaticamente.
3. **Validare le Condizioni di Margine:** Testando tutti i possibili casi, compresi quelli che potrebbero essere considerati estremi o rari.

## Esempio Generale

### SHIFTCOMP

Il modulo **ShiftAndCompare** è un sistema progettato per gestire un registro di rotazione a 8 bit. Questo sistema ruota il contenuto del registro ad ogni fronte di salita del segnale di clock (Clk) e verifica se il contenuto corrente del registro coincide con un valore di confronto. Se i valori sono uguali, viene attivato un segnale di uscita (`Limit`) per un singolo ciclo di clock.



## Architettura del Sistema

Il sistema è composto da tre moduli principali:

1. **Comparator**: Confronta due vettori a 8 bit e restituisce un segnale di uguaglianza ( **EQ** ).
2. **ShiftRegister**: Implementa un registro a 8 bit con funzionalità di caricamento e rotazione.
3. **ShiftAndCompare**: Collega il **ShiftRegister** e il **Comparator**, gestendo il flusso di dati e la logica di controllo.

### COMPARATOR

Il modulo **Comparator** verifica se due segnali a 8 bit sono uguali.

```
library ieee;
use ieee.std_logic_1164.all;

entity Comparator is
    port (
        A, B: in std_logic_vector (7 downto 0); -- Ingressi
        EQ: out std_logic                        -- Segnale
    );
end Comparator;

architecture Comparator_1 of Comparator is
begin
    EQ <= '1' when (A = B) else '0'; -- Uscita 1 se A e B so
end Comparator_1;
```

### SHIFT-REGISTER

Il modulo **ShiftRegister** implementa un registro a 8 bit che può essere caricato con un nuovo valore o ruotato ad ogni ciclo di clock.

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftRegister is
```

```

    port (
        Clk, Rst, Load: in std_logic;           -- R
        Data: in std_logic_vector (7 downto 0); -- D
        Q: out std_logic_vector (7 downto 0)     -- I
    );
end ShiftRegister;

architecture ShiftRegister_1 of ShiftRegister is
    signal Qreg: std_logic_vector (7 downto 0); -- R
begin
    reg: process (Rst, Clk)
    begin
        if Rst = '1' then -- R
            Qreg <= "00000000"; -- I
        elsif (Clk = '1' and Clk'event) then -- Fr
            if (Load = '1') then -- C
                Qreg <= Data; -- Ca
            else -- A
                Qreg <= Qreg (6 downto 0) & Qreg (7); -- Ru
            end if;
        end if;
    end process;

    Q <= Qreg; -- As
end ShiftRegister_1;

```

## SHIFT-AND-COMPARE

Il modulo **ShiftAndCompare** collega il registro di rotazione e il comparatore, gestendo il ciclo di clock e la logica di confronto.

```

library ieee;
use ieee.std_logic_1164.all;

entity ShiftAndCompare is
    port (
        Clk, Rst, Load: in std_logic;           --

```

```

        Init: in std_logic_vector (7 downto 0);           -- I
        Test: in std_logic_vector (7 downto 0);          -- T
        Limit: out std_logic                               -- L
    );
end ShiftAndCompare;

architecture ShiftAndCompare_1 of ShiftAndCompare is
    component Comparator
        port (
            A, B: in std_logic_vector (7 downto 0);
            EQ: out std_logic
        );
    end component;

    component ShiftRegister
        port (
            Clk, Rst, Load: in std_logic;
            Data: in std_logic_vector (7 downto 0);
            Q: out std_logic_vector (7 downto 0)
        );
    end component;

    signal Q: std_logic_vector (7 downto 0);              -- S

begin
    COMP1: Comparator port map (A => Q, B => Test, EQ => Limit);
    SHFT1: ShiftRegister port map (Clk => Clk, Rst => Rst, Load => Load, Data => Data);
end ShiftAndCompare_1;

```

## TEST-BENCH

Il **Test Bench** simula il comportamento dell'intero sistema. Non ha porte di ingresso o uscita, poiché serve solo a generare stimoli e monitorare il comportamento del sistema.

```

library ieee;
use ieee.std_logic_1164.all;

entity TestBench is

```

```

end TestBench;

architecture TestBench_1 of TestBench is
    component ShiftAndCompare
        port (
            Clk, Rst, Load: in std_logic;
            Init: in std_logic_vector (7 downto 0);
            Test: in std_logic_vector (7 downto 0);
            Limit: out std_logic
        );
    end component;

    signal Clk, Rst, Load: std_logic;
    signal Init: std_logic_vector (7 downto 0);
    signal Test: std_logic_vector (7 downto 0);
    signal Limit: std_logic;

begin
    UUT: ShiftAndCompare port map (Clk, Rst, Load, Init, Test, Limit);

    -- Processo per generare il segnale di clock
    clock: process
        variable clktmp: std_logic := '0';
    begin
        clktmp := not clktmp;
        Clk <= clktmp;
        wait for 50 ns;
    end process;

    -- Processo per applicare gli stimoli
    stimulus: process
    begin
        Rst <= '0';
        Load <= '1';
        Init <= "00001111";
        Test <= "11110000";
        wait for 100 ns;
        Load <= '0';
    end process;

```

```
        wait for 600 ns;                -- A
        wait;                          -- M
    end process;
end TestBench_1;
```

## Riepilogo

Il sistema **ShiftAndCompare** è progettato per gestire una rotazione a 8 bit e confrontare il risultato con un valore specifico. Ogni modulo è responsabile di una parte del processo, e il test bench fornisce un ambiente controllato per testare l'intero sistema, assicurando che le funzionalità di caricamento, rotazione e confronto funzionino correttamente.

---

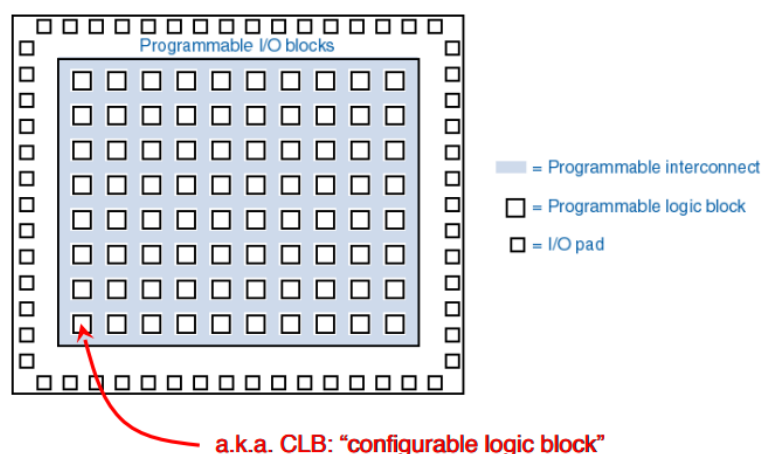
# FPGA nel Dettaglio

## Logiche Programmabili

In questo ambito, con logiche programmabili, intendiamo non macchine di Turing, bensì logiche la cui configurazione circuitale è programmabile, infatti, queste logiche non possono eseguire dei programmi, ma la loro struttura circuitale è completamente programmabile.

## Architettura Sistema

L'Architettura del sistema che viene utilizzata è sempre la stessa che abbiamo visto fin ora. **Alla periferia del chip ci sono dei BLOCCHI di I/O (input/output)** che acquisiscono un segnale analogico che influenza il segnale finale. Questi blocchi possono fare sia da ingresso che da uscita. La parte centrale del chip è invece realizzata tramite un array regolare di logiche programmabili che quando vengono interconnesse generano un certo andamento funzionale complessivo.



**NOTA:** *i segnali logici sono una semplice astrazione, gli unici segnali che realmente esistono sono le forme d'onda analogiche che poi opportunatamente interpretate "diventano" segnali logici digitali.*

Gli **elementi costitutivi** sono come abbiamo già detto gli I/O BLOCKS e i BLOCCHI LOGICI che sono interconnettibili attraverso degli switch di interconnessione.

# Programmabilità Fisica

## Evoluzione delle memorie a stato solido

Le **ROM (read-only-memory)** sono memorie a sola lettura, e fanno da contraltare alle (**RAM random-access memory**), che sono nate come evoluzione della memoria magnetica: un tempo le memorie erano seriali, poi con le memorie a semiconduttore l'**accesso è diventato random** ovvero, **possiamo andare a selezionare il bit che ci interessa senza dover per forza scorrere sequenzialmente tutta la catena. Le ROM sono più o a meno la stessa cosa però non sono riprogrammabili né scrivibili come le RAM.**

In origine, infatti, le interconnessioni non erano riprogrammabili, perché create con processi irreversibili (es. bruciare un fusibile) (**PROM-programmable read only memory**) significa che una volta scritto il dato con questo processo "distruttivo" non si poteva più tornare indietro. Inseguito, venne considerata un'alternativa reversibile e non volatile, realizzata con un mosfet a doppio gate (**EPROM erasable programmable read only memory**), in questo caso abbiamo delle memorie cancellabili (attraverso una esposizione a luce ultravioletta). Se nel gate intermedio non c'è carica, il mosfet funziona normalmente; viceversa se il gate intermedio ha carica sufficiente per bilanciare quella del gate superiore, non si crea il canale d' inversione e perciò il mosfet non funziona mai.

NOTA: ci sono anche le EEPROM (**electric erasable programmable read only memory**): sono come le EPROM ma in questo caso possono essere cancellate elettricamente senza dover far uso dei raggi UV.

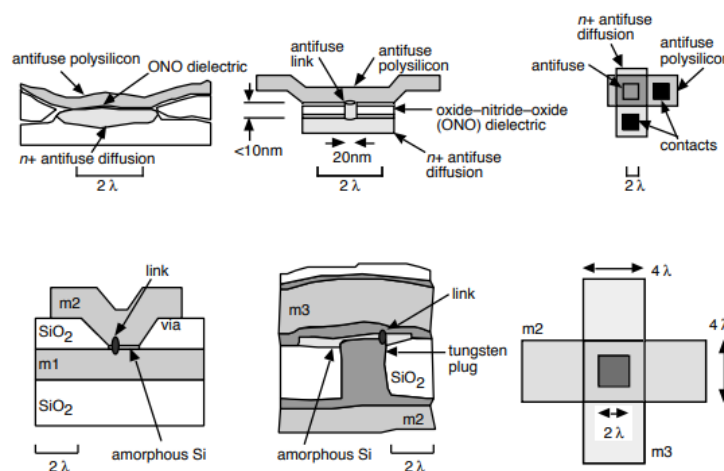
Un PLD contiene componenti sia di logica che di memoria (contenenti le informazioni di configurazione); quest'ultime possono essere di tipo:

- **Antifusibili al silicio**
- **SRAM**
- **Flash**
- **Celle EPROM**

## Antifusibili al Silicio (Normalmente OFF)

**Originariamente la programmazione dei dispositivi una volta realizzata non era più riconfigurabile** (ovvero non era un evento reversibile). La procedura più utilizzata era l'Antifuse, **dove era presente un plug resistivo (di norma un**

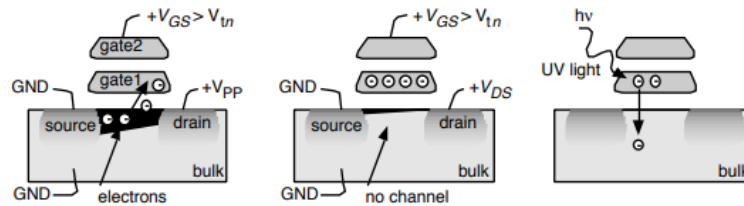
**fusibile che veniva bruciato) che poteva essere fuso attraverso l'applicazione di una data tensione, che in tal modo, metteva definitivamente in contatto due piste, realizzando così, una connessione permanente.** Con questo processo non si può più tornare indietro perché è appunto un processo distruttivo. I circuiti integrati che utilizzano la tecnologia "antifuse", quindi impiegano una barriera sottile di materiale dielettrico di silicio amorfo (circa 9 nanometri di nitrato di silicio) tra due conduttori metallici. Quando viene applicata una tensione sufficientemente alta (mediante un breve impulso di circa 1 millisecondo dell'ampiezza di circa 16 volt), tutto il silicio amorfo si trasforma in una lega policristallina silicio-metallo con una bassa resistenza, che è conduttiva .



## Non Volatile Control EPROM ed EEPROM

Si tratta in questo caso di una **programmazione non distruttiva ma riprogrammabile e recuperabile**. Fa uso di un transistor MOS che possiamo paragonare come ben sappiamo a un "rubinetto". Applicando una tensione di soglia sul gate si crea un canale di elettroni che si muovono dal drain al source o viceversa . In questo caso prendiamo il gate e lo isoliamo con un ossido (gate isolato) il MOS diventa un condensatore con un'armatura isolata completamente.

**Sopra il gate viene posto il gate 2 (che è flottante), applicando una tensione che è maggiore a quella di soglia sul gate 2 (di controllo) "la carica si sposta dal canale di elettroni al gate 1 (non c'è più canale di elettroni), è quindi in questo modo che si crea/elimina la connessione programmando di conseguenza il circuito.** La programmazione in questo caso è reversibile perché il processo non è distruttivo, per ripristinare le condizioni iniziali basterà esporre il circuito a luce UV.

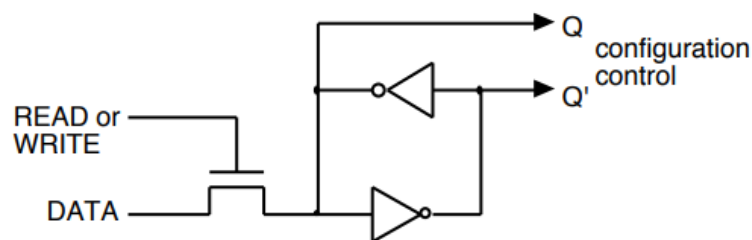


## Static RAM Control

Utilizzando questa tecnologia nel momento in cui si spegne il circuito (non si fa passare più corrente) la memoria "scompare" (si tratta di un flip-flop) ovvero una cella di memoria RAM statica.

**Read/Write** :comanda un pass transistor che abilita la lettura, i due inverter retroazionati costituiscono il latch di memoria.

I due inverter retroazionati fungono da **Flip-Flop che consiste in una memoria statica**.



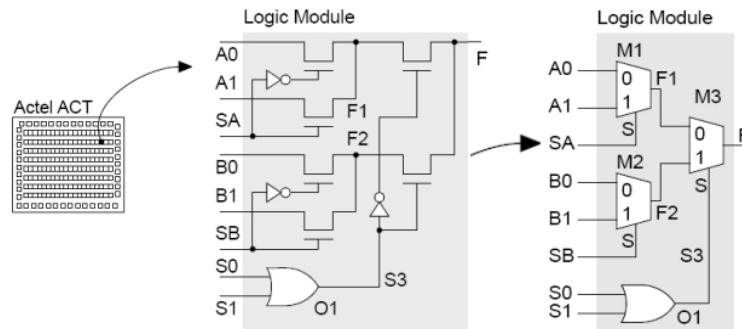
## Programmabilità Logica

La **programmabilità logica** si riferisce alla capacità di configurare dinamicamente il comportamento di un circuito logico, modificando le connessioni tra i blocchi logici interni per implementare diverse funzioni. Questa funzionalità rende possibile la realizzazione di dispositivi versatili e adattabili, utili in applicazioni che richiedono variazioni rapide senza la necessità di riprogettare il circuito fisico.

### Esempio Actel ACT 1

L'**Actel ACT 1** rappresenta un caso prototipico di dispositivo a logica programmabile, una delle prime implementazioni che ha sfruttato blocchi logici configurabili e un'architettura a matrice regolare. Questa struttura ha introdotto una versatilità notevole, rendendo possibile la configurazione dinamica delle connessioni per creare una gamma di funzioni logiche senza bisogno di ridefinire il circuito fisico. I multiplexer, utilizzati come blocchi di configurazione

fondamentali, offrono la possibilità di selezionare diverse combinazioni di input e simulare comportamenti di memoria ROM, rendendo il sistema adattabile a varie applicazioni logiche.



## The Motivation of The Cell Structure

La progettazione delle celle logiche programmabili mira a realizzare funzioni complesse attraverso una struttura modulare. Questa struttura permette di suddividere funzioni logiche avanzate in blocchi più semplici, rendendo le configurazioni più flessibili e risparmiando risorse. Ogni cella è costruita per essere riprogrammabile e utilizzabile in diverse combinazioni, garantendo al dispositivo la capacità di adattarsi a nuove configurazioni in modo efficiente.

## Mappatura di una Singola Cella

Consideriamo, ad esempio, la funzione logica:

$$F = AB + B'C + D$$

Questa funzione può essere riformulata utilizzando la logica booleana per semplificarne la configurazione, come segue:

- $F = B(A+D) + B'(C+D) = B \cdot F2 + B' \cdot F1;$

Dove:

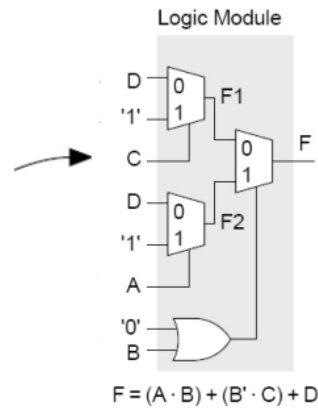
- $F1 = C+D = C \cdot 1 + C' \cdot D;$
- $F2 = A+D = A \cdot 1 + A' \cdot D;$

Questa formulazione consente di implementare F suddividendo la funzione in due sotto funzioni, F1 e F2 che possono essere configurate su blocchi logici più semplici e poi interconnesse.

$$F = AB + B'C + D = B'F_1 + BF_2$$

$$F_1 = C + D = C \cdot 1 + C'D$$

$$F_2 = A + D = A \cdot 1 + A'D$$



## Quante funzioni di due variabili binarie esistono?

La versatilità della programmabilità logica può essere compresa esaminando il numero di funzioni che è possibile creare con due variabili binarie. Con due ingressi, ci sono  $2^2 = 4$  configurazioni di input possibili, il che implica la possibilità di creare  $2^4 = 16$  funzioni logiche diverse. Tra queste troviamo:

- Una funzione sempre zero
- Quattro funzioni con un solo "1" e tre "0"
- Quattro funzioni con un solo "0" e tre "1"
- Sei funzioni con due "1" e due "0"
- Una funzione sempre uno

Questa varietà di combinazioni consente ai dispositivi programmabili di adattarsi a numerose esigenze circuitali, rappresentando una soluzione flessibile per le applicazioni logiche. Alcuni esempi di queste funzioni includono:

- **F0 = 0: sempre zero**
- **F4=AB: AND di A e B**
- **F5=A+B: OR di A e B**
- **F14=A·B'+A'·B: XOR tra A e B**

Questa gamma completa di possibilità rende le logiche programmabili strumenti particolarmente potenti per implementare qualsiasi operazione logica di base.

For  $n = 2$ , the functions are 16:

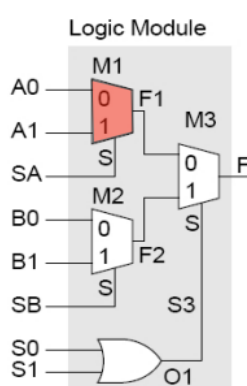
|                                |
|--------------------------------|
| 1 x tutti "zero"               |
| 4 x un solo "uno" e tre "zero" |
| 4 x un solo "zero" e tre "uno" |
| 6 x due "uno" e due "zero"     |
| 1 x tutti "uno"                |

|                |                        |
|----------------|------------------------|
| $F_0 = "0"$    | $F_8 = A' + B'$        |
| $F_1 = A' B'$  | $F_9 = A'$             |
| $F_2 = A' B$   | $F_{10} = A$           |
| $F_3 = A B'$   | $F_{11} = B'$          |
| $F_4 = A B$    | $F_{12} = B$           |
| $F_5 = A + B$  | $F_{13} = A' B' + A B$ |
| $F_6 = A + B'$ | $F_{14} = A B' + A' B$ |
| $F_7 = A' + B$ | $F_{15} = "1"$         |

## Funzioni svolte dal MUX

Un singolo multiplexer (MUX) può essere utilizzato per realizzare molte delle funzioni logiche fondamentali. Grazie alla sua capacità di selezionare tra vari ingressi, un MUX può essere configurato come una sorta di generatore di funzioni logiche. In una configurazione programmabile, il MUX seleziona le combinazioni di input desiderate per ottenere l'uscita logica richiesta.

Ad esempio, collegando opportunamente i dati in ingresso di un MUX, è possibile rappresentare tutte le combinazioni di funzione logica basate su due variabili. La configurazione del MUX rappresenta una forma semplificata di programmabilità logica, che offre un modo versatile per creare numerose operazioni a partire da un singolo componente logico.



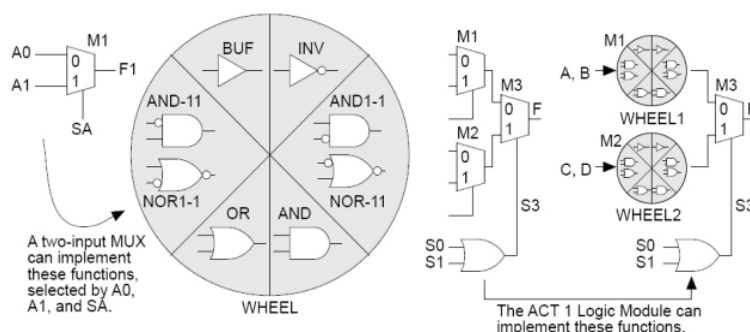
| A0 | A1 | SA | F     | F <sub>n</sub>  |
|----|----|----|-------|-----------------|
| 0  | —  | 0  | "0"   | F <sub>0</sub>  |
| —  | 1  | 1  | "1"   | F <sub>15</sub> |
| 0  | A  | B  | AB    | F <sub>4</sub>  |
| A  | 0  | B  | AB'   | F <sub>3</sub>  |
| B  | 0  | A  | A'B   | F <sub>2</sub>  |
| —  | A  | 1  | A     | F <sub>10</sub> |
| —  | B  | 1  | B     | F <sub>12</sub> |
| 1  | 0  | B  | B'    | F <sub>11</sub> |
| 1  | 0  | A  | A'    | F <sub>9</sub>  |
| A  | 1  | B  | A + B | F <sub>5</sub>  |

## Wheel of Fortune

Nella programmabilità logica, il concetto di "Wheel of Fortune" rappresenta un modo per visualizzare le diverse funzioni ottenibili con i blocchi logici

programmabili. Come una ruota che può ruotare per selezionare una diversa funzione a seconda della posizione, i dispositivi programmabili consentono di passare da una configurazione logica all'altra in modo dinamico.

Questa capacità è fondamentale per applicazioni che richiedono aggiornamenti o cambiamenti frequenti delle funzioni logiche implementate. La "ruota" delle funzioni rappresenta dunque la flessibilità intrinseca dei circuiti programmabili, che possono essere configurati per implementare funzioni diverse in modo semplice e rapido.

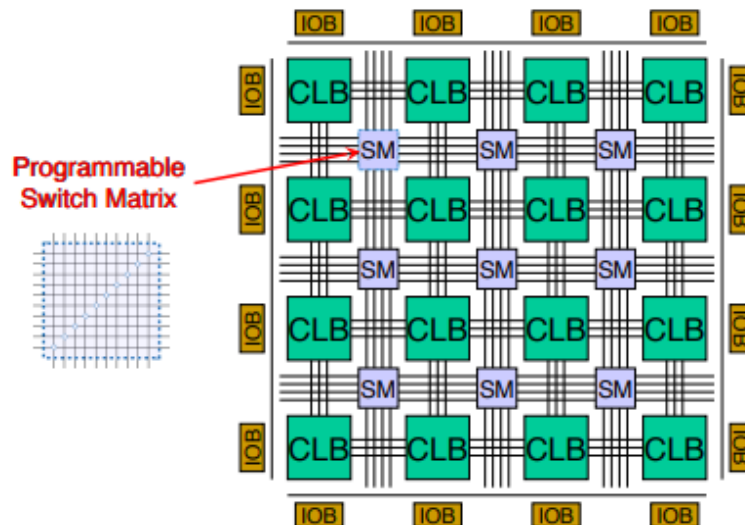


## Blocchi Logici Configurabili

Il **Configurable Logic Block (CLB)** rappresenta il componente fondamentale nelle FPGA (Field-Programmable Gate Array). Ogni CLB contiene elementi logici programmabili e risorse di memoria, che possono essere configurati per implementare funzioni combinatorie o sequenziali. I CLB sono organizzati in una matrice regolare e interconnessi da una rete di commutazione programmabile, che permette la configurazione personalizzata del comportamento logico complessivo.

All'interno di ogni CLB sono tipicamente presenti:

- **Blocchi logici combinatori**, spesso realizzati con Look-Up Table (LUT), per implementare funzioni logiche.
- **Flip-flop** per la memorizzazione di valori logici e la gestione di operazioni sequenziali.
- **Multiplexer** per selezionare gli input o combinare le uscite.



## ACTEL ACT 1

Le **celle logiche ACTEL ACT1** utilizzano un'architettura basata su antifusibili per interconnessioni permanenti. Questo approccio garantisce alte prestazioni e un basso consumo energetico, ma rende la configurazione irreversibile.

Ogni cella ACTEL ACT1 include:

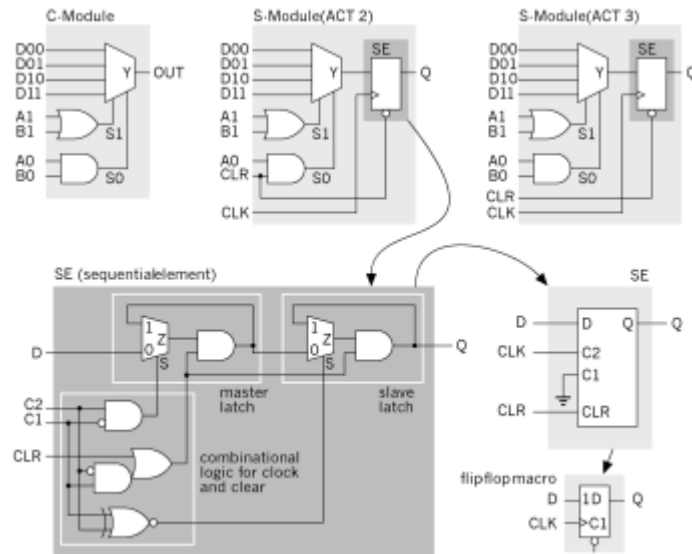
- **Elementi combinatori** per eseguire funzioni logiche di base.
- **Flip-flop** per supportare operazioni sequenziali.
- Una struttura di interconnessione semplice e stabile, ottimizzata per applicazioni a bassa latenza.

## MODELLO DEI RITARDI DELLA LOGICA

modello di temporizzazione per le celle logiche ACTEL ACT1 è definito dai seguenti parametri:

- **Critical Path:**  $t_{PD} + t_{SUD} + t_{CO}$
- $t_{PD}$  : Tempo di propagazione, dipendente dalla funzione combinatoria implementata.
- $t_{SUD}$  : Tempo di setup.
- $t_{CO}$  : Tempo di clock-to-output, influenzato dal fan-out.
- $t_H$  : Tempo di hold del flip-flop.

La temporizzazione reale dipende dalla logica implementata e dalle interconnessioni del blocco.

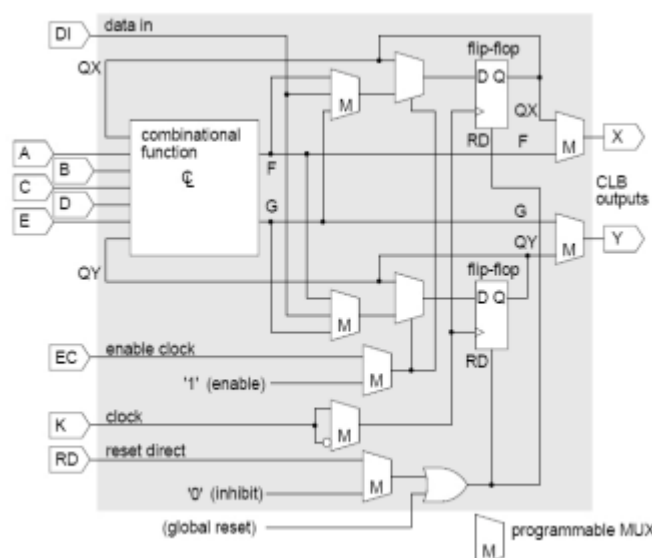


## Xilinx XC3000

Le celle logiche Xilinx XC3000 utilizzano un'architettura basata su Look-Up Table (LUT) per implementare funzioni logiche combinatorie. Ogni cella logica comprende:

- Una **LUT a 3 ingressi** per realizzare funzioni logiche di base.
- Un **flip-flop** per operazioni sequenziali.
- Un'unità di controllo configurabile per gestire il comportamento della cella.

Questo design consente una maggiore flessibilità rispetto agli antifusibili, rendendo le celle riprogrammabili.



## Xilinx XC4000

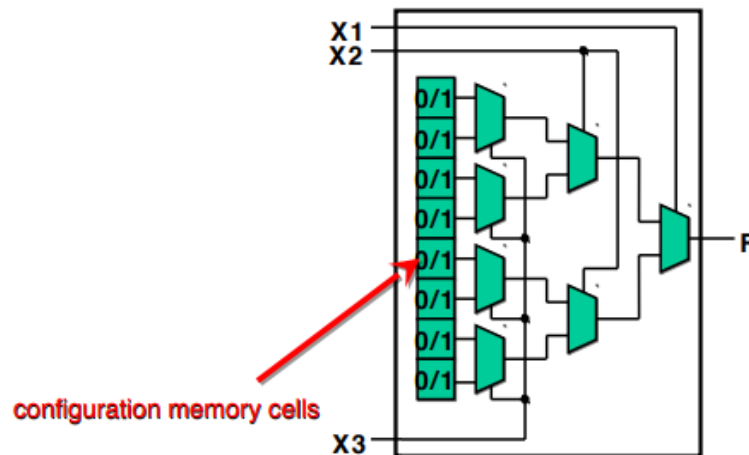
- Una **LUT a 4 ingressi**, che consente di implementare funzioni logiche più complesse.
- Una struttura di **interconnessione avanzata** per supportare applicazioni ad alta densità.
- **Flip-flop integrati** per il supporto delle operazioni sequenziali.

Le **Look-Up Table (LUT)** sono il cuore dell'implementazione logica nelle FPGA. Una LUT è essenzialmente una piccola memoria che memorizza i valori di output per tutte le combinazioni possibili di input.

Le LUT permettono di implementare qualsiasi funzione logica combinatoria in modo efficiente. Rispetto ai circuiti tradizionali:

- La struttura di una LUT prevede:

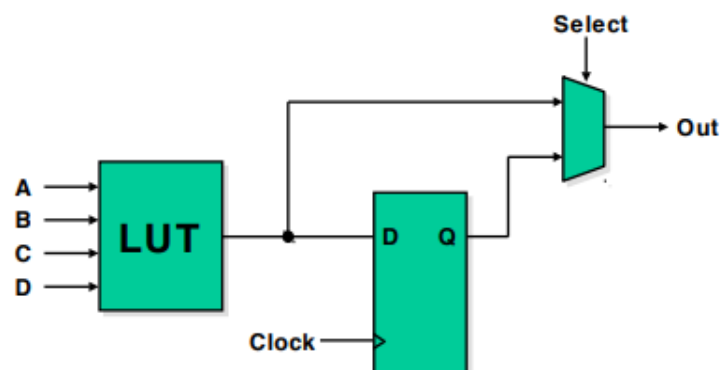
- **Ingressi logici** che selezionano l'indirizzo nella memoria.
- **Uscite logiche** che corrispondono ai valori memorizzati per l'indirizzo selezionato.
- Una configurazione programmabile per personalizzare le funzioni logiche.



## CLB

Il **Configurable Logic Block (CLB)** è composto principalmente da una combinazione di LUT, flip-flop e risorse di routing. Ogni CLB è progettato per:

- Eseguire funzioni logiche combinatorie e sequenziali.
- Supportare configurazioni personalizzate per ottimizzare il circuito logico.

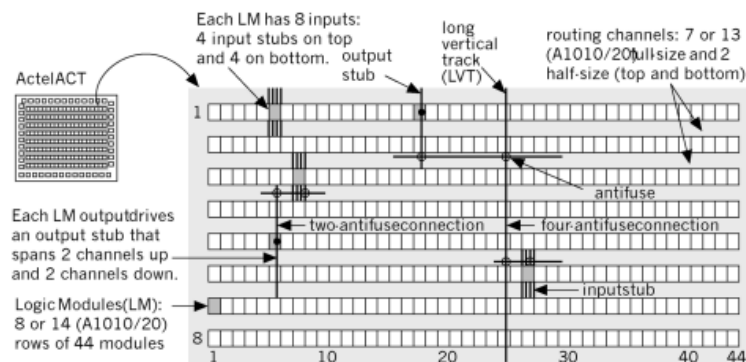


## Interconnessioni

Le **interconnessioni** rappresentano una componente cruciale nelle FPGA, poiché determinano la capacità di collegare tra loro i blocchi logici configurabili

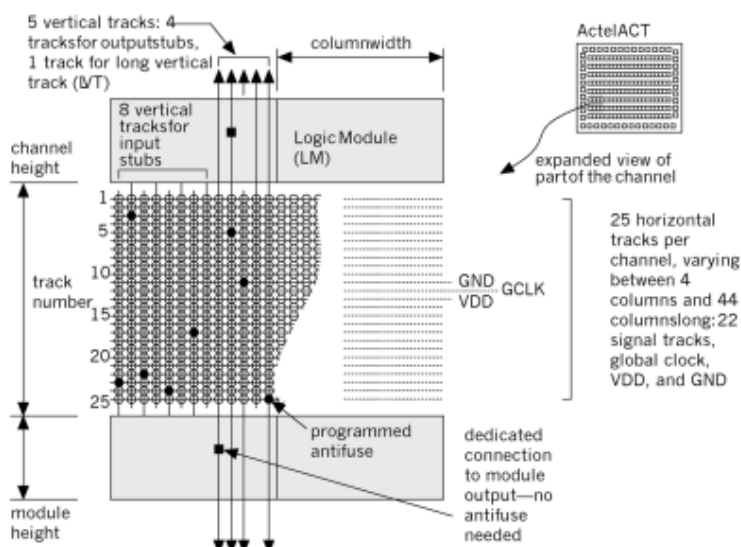
(CLB) e di definire il comportamento complessivo del circuito. Tuttavia, richiedono un elevato utilizzo di risorse, in particolare moduli RAM, per gestirne la programmazione. Inoltre, costituiscono la principale causa di ritardo nel circuito, influenzando in modo significativo le prestazioni generali.

## ACTEL ACT 1 Architettura

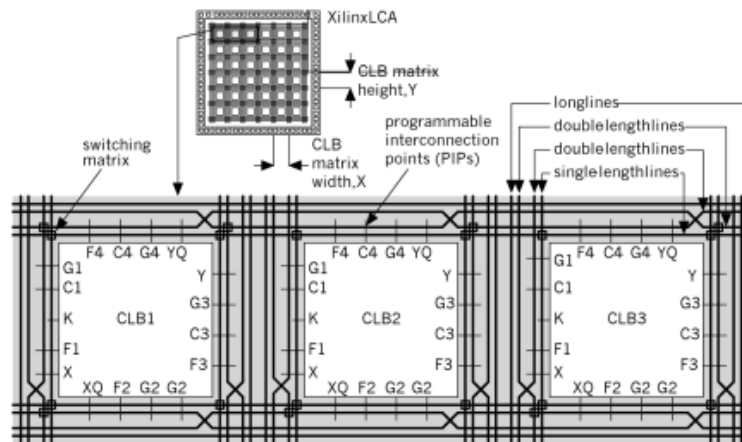


## Zooming the Channel

Il **channel** rappresenta l'area dedicata alle linee di interconnessione tra i CLB. Ogni channel è composto da più piste e punti di intersezione che permettono la programmazione delle connessioni. L'ottimizzazione di questa rete è essenziale per minimizzare i ritardi e migliorare le prestazioni.

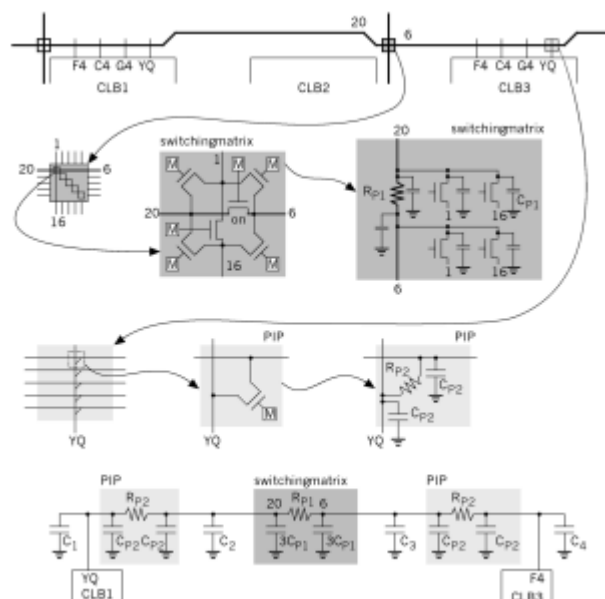


## Xilinx Architettura



## Interconnessione a Matrice

La matrice di interconnessione utilizza un **pass-transistor** tra ogni possibile coppia di linee. Attivando selettivamente i pass-transistor, è possibile stabilire quali collegamenti sono attivi. Questo modello può essere tradotto in un modello elettrico, dal quale si calcola facilmente il **ritardo dovuto all'interconnessione**.



## Stima del Critical Path - Il ritardo di Elmore

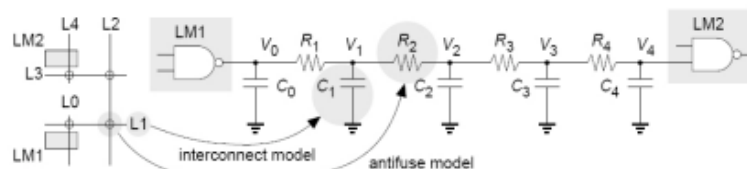
La **Critical Path Estimation** è un'analisi fondamentale per verificare che i ritardi di propagazione siano compatibili con i requisiti temporali dell'applicazione. Ogni applicazione ha requisiti caratteristici, e il percorso critico dipende sia dalla funzione implementata sia dalla sua disposizione fisica (layout). Pertanto,

è necessaria una verifica **post-layout (post place & route)** per confermare la compatibilità temporale.

Il **ritardo di Elmore** è un modello utilizzato per stimare i tempi di propagazione nelle interconnessioni. Questo modello è basato sull'analisi della resistenza e della capacità delle linee di connessione, fornendo un'approssimazione del ritardo introdotto dalle interconnessioni all'interno del circuito.

### ESEMPIO NELL'ACTEL

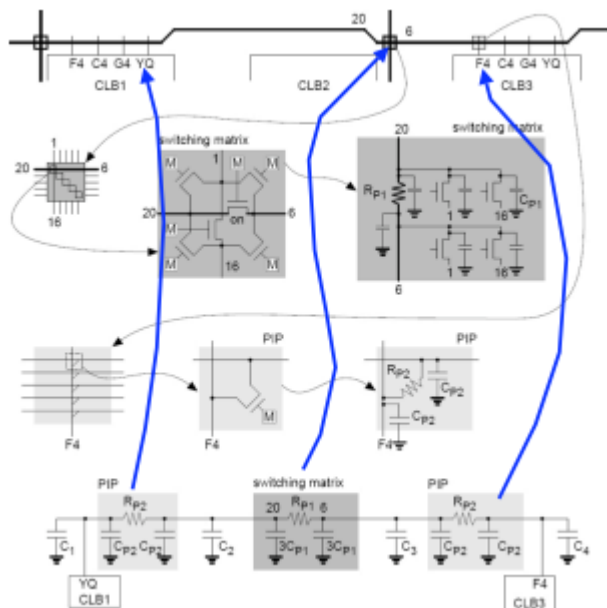
Nel caso delle FPGA **Actel**, il ritardo di propagazione è fortemente influenzato dalla semplicità dell'architettura basata su antifusibili. La struttura delle interconnessioni permanenti riduce il carico parassita e, di conseguenza, i ritardi complessivi. Questo rende le Actel particolarmente adatte per applicazioni a bassa latenza.



$$\begin{aligned}
 V_4 &= - \sum_k R_{k4} C_k (dV_4/dt) \\
 \tau_{D4} &= \sum_k R_{k4} C_k = R_{14} C_1 + R_{24} C_2 + R_{34} C_3 + R_{44} C_4 = \\
 &= R_1 C_1 + (R_1 + R_2) C_2 + (R_1 + R_2 + R_3) C_3 + \\
 &\quad + (R_1 + R_2 + R_3 + R_4) C_4 \\
 \text{if } R_i &= R_j = R \text{ and } C_i = C_j = C \\
 \tau_{D4} &= 10 RC
 \end{aligned}$$

### ESEMPIO NELL'XILINX

L'architettura **Xilinx** si basa su una **LUT a 5 ingressi**, riconfigurabile come due LUT a 4 ingressi (purché non si utilizzino più di 5 segnali distinti). Questa configurazione consente una migliore ottimizzazione delle risorse quando la funzione combinatoria ha una complessità ridotta. Tuttavia, la maggiore flessibilità dell'architettura LUT comporta un aumento del ritardo di propagazione rispetto alle interconnessioni basate su antifusibili.



## Implementazione

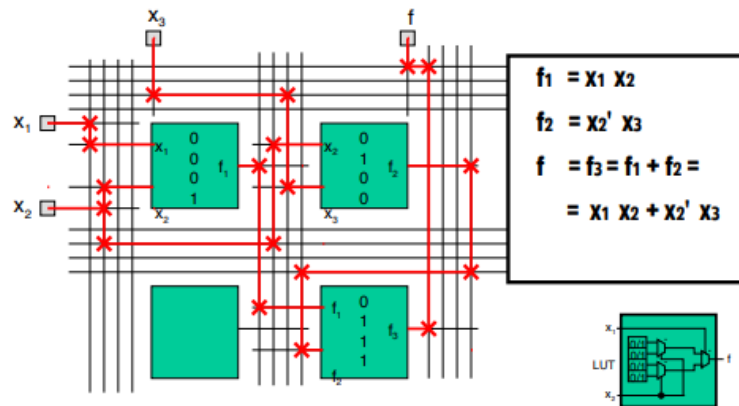
Per comprendere il funzionamento delle FPGA, si può immaginare un esempio pratico in cui una funzione logica è suddivisa tra diversi CLB. Gli ingressi vengono instradati attraverso la rete di interconnessione, i blocchi eseguono le funzioni logiche assegnate e i risultati vengono combinati per produrre l'output desiderato.

### Vantaggi delle FPGA:

- **Prestazioni superiori** rispetto ai processori tradizionali in applicazioni che richiedono operazioni logiche intensive.
- **Consumi energetici ottimizzati** per unità di area rispetto ad altre tecnologie.

### Svantaggi delle FPGA:

- **Complessità di programmazione**, che richiede competenze specifiche per sfruttarne appieno il potenziale.



## Condizionamento di Segnale

Il **condizionamento del segnale** è un processo fondamentale nel trattamento dei segnali analogici per consentire la loro corretta interpretazione come segnali digitali. Poiché i segnali analogici possono avere andamenti variabili e non sempre interpretabili univocamente, è necessario normalizzarli e filtrarli. Questo garantisce che possano essere rappresentati come simboli digitali "0" o "1" senza ambiguità.

Nel campo dell'acquisizione dati, il condizionamento del segnale è cruciale: i segnali provenienti dai sensori devono essere adattati per rientrare nei parametri di funzionamento dei circuiti interni di un dispositivo. I blocchi di input/output (I/O) svolgono questa funzione, introducendo e normalizzando i segnali esterni all'interno del chip.

## BLOCCHI INPUT/OUTPUT

I blocchi di I/O gestiscono l'interazione tra l'esterno del chip e i circuiti interni, svolgendo diverse funzioni essenziali:

- **Condizionamento dei segnali esterni.**
- **Protezione contro scariche elettrostatiche.**
- **Fornitura di alimentazione e riferimenti di tensione.**

## REQUISITI FUNZIONALI DEI BLOCCHI I/O

I blocchi I/O possono gestire diverse tipologie di ingressi e uscite, come:

- **Ingressi di potenza:** per alimentare il dispositivo.
- **Segnali di clock:** utilizzati per sincronizzare i circuiti digitali.

- **Ingressi/Uscite in corrente continua (DC):** per pilotare LED, relè o altri piccoli carichi resistivi.
- **Ingressi/Uscite in corrente alternata (AC):** per segnali ad alta frequenza, logiche veloci, linee seriali e bus dati.

Dal punto di vista funzionale, ogni dispositivo che pilota una linea esterna è considerato un buffer.

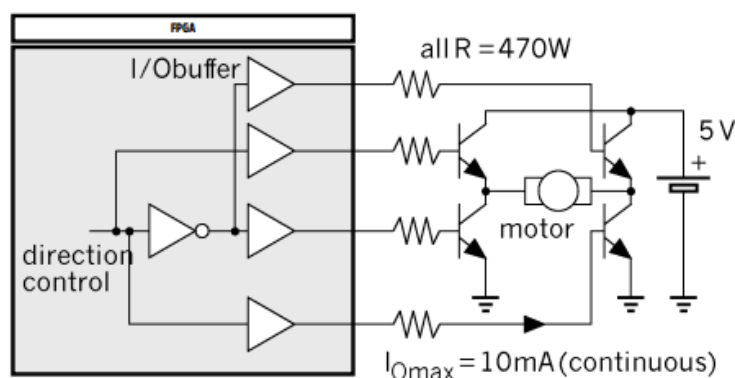
## BLOCCHI OUTPUT (50-200mA)

Il **buffer di uscita** consente di pilotare carichi capacitivi significativi. Questo avviene caricando o scaricando capacità esterne con tempi di propagazione adeguati. I buffer sono spesso configurabili come ingressi o uscite.

### ESEMPIO: CONTROLLO MOTORI

Nel caso di un controllo motore mediante FPGA:

- I buffer di uscita non pilotano direttamente il motore.
- È necessario interporre uno stadio di potenza, ad esempio un ponte di transistori, per gestire le correnti richieste.



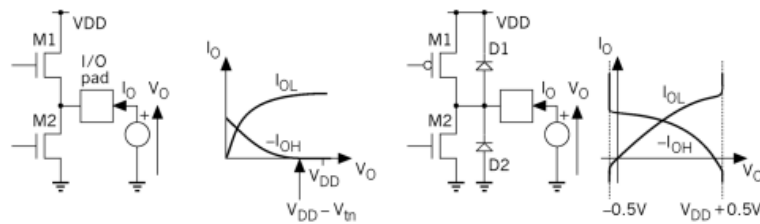
## TOTEM-POLE OUTPUT

La configurazione **totem-pole** è uno stadio attivo in cui due transistor lavorano in opposizione di fase (uno acceso, l'altro spento). Questa configurazione include diodi di protezione (o diodi di clamping) che proteggono da sovratensioni o sottotensioni dovute a carichi induttivi.

### PROTEZIONE CONTRO CARICHI INDUTTIVI

Se una bobina, per esempio, accumula energia induttiva e il generatore viene spento, la bobina tende a richiamare corrente, causando un aumento di

tensione pericoloso. I diodi di protezione evitano che il dielettrico tra drain e gate dei transistor venga danneggiato.

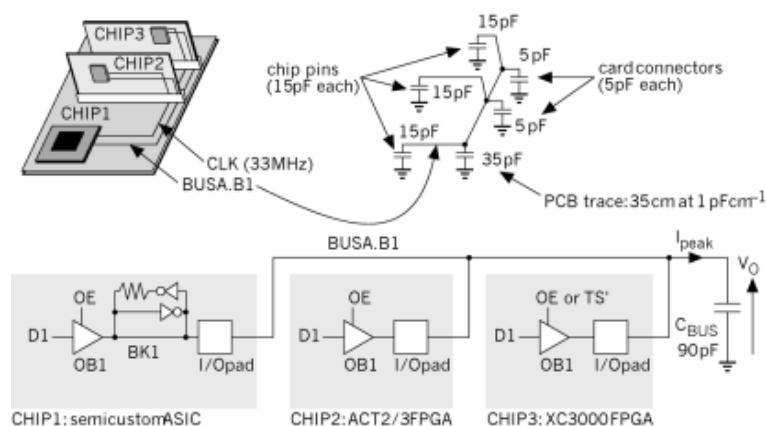


## TRI-STATE

Un **buffer tri-state** consente tre stati distinti:

- Stato "0": transistor PD acceso, PU spento.
- Stato "1": transistor PU acceso, PD spento.
- Stato ad alta impedenza: entrambi i transistor spenti, senza conduzione.

Questa configurazione è essenziale per condividere bus tra più dispositivi, garantendo che solo uno alla volta possa trasmettere. Per evitare conflitti, il buffer viene disabilitato (ad alta impedenza) quando non è necessario.



## LINEE DI TRASMISSIONE

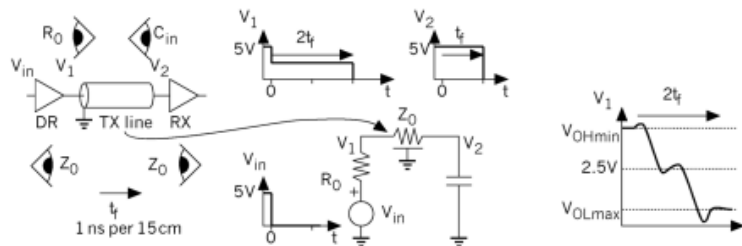
In presenza di **commutazioni rapide rispetto alle impedenze coinvolte**, si deve considerare il **tempo di propagazione**, noto come **tempo di volo(  $t_f$ )**.

Tipicamente, è dell'ordine di **1 ns ogni 30 cm di linea di trasmissione** (circa metà della velocità della luce nel vuoto).

- L'onda si propaga fino alla fine della linea e viene riflessa.

- Gli effetti di propagazione diventano significativi quando la **commutazione all'uscita** avviene in un tempo minore di **2 volte il tempo di volo**.

Questi fenomeni possono generare **fluttuazioni indesiderate** nel segnale, causando **malfunzionamenti** nei circuiti logici.

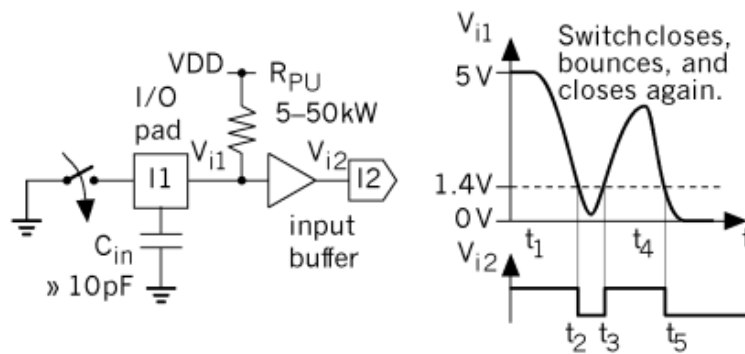


Per mitigare tali problemi, si utilizzano le **terminazioni di adattamento**, che includono:

1. **Circuito aperto:** sfrutta l'impedenza d'ingresso del ricevitore.
2. **Resistenza in parallelo:** riduce il rischio di riflessi, ma aumenta il consumo di potenza in corrente continua.
3. **Terminazione di Thévenin:** offre un consumo di potenza in corrente continua ridotto.
4. **Adattamento alla sorgente:** assicura la corrispondenza di impedenza tra sorgente e linea.
5. **Adattamento in parallelo con condensatore in serie:** combina vantaggi di resistenza e capacità per migliorare il bilanciamento del segnale.

## INPUT BOUNCING

Quando un segnale digitale presenta **rimbalzi** all'ingresso, possono verificarsi **interpretazioni errate** di stati logici ("0" o "1"). Per prevenire tali errori, si ricorre a tecniche di **debouncing**.



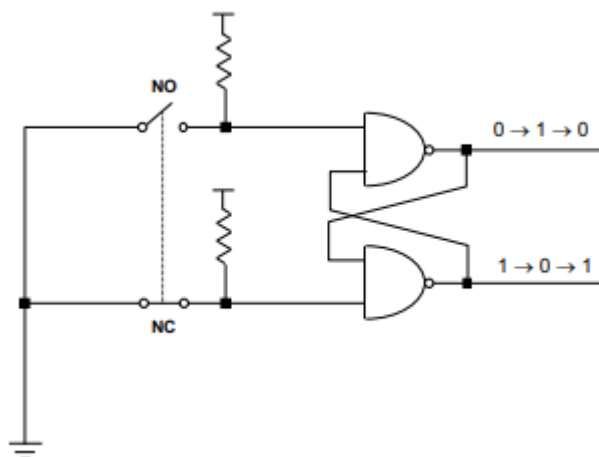
## 1. Debouncing con Flip-Flop SR

Un circuito anti-rimbalzo può essere implementato tramite **porte logiche**, come NAND o NOR, per creare un **flip-flop Set-Reset (SR)**.

- **Funzionamento:**

- Il flip-flop **memorizza lo stato dell'uscita**, ignorando gli impulsi di disturbo sull'ingresso.
- Per cambiare lo stato dell'uscita, è necessario applicare un impulso su **due ingressi distinti** (Set e Reset).

- **Vantaggi:** elevata immunità al disturbo.



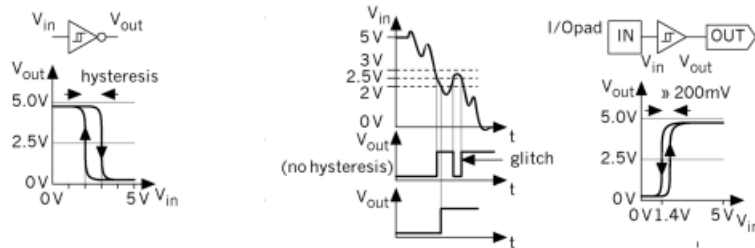
## 2. Debouncing con Trigger di Schmitt

I dispositivi di **Trigger di Schmitt** vengono utilizzati per il **condizionamento del segnale**, rimuovendo **rumore** e **rimbalzi** nei contatti degli interruttori.

- **Funzionamento:**

- Il Trigger di Schmitt introduce una **isteresi**, che trasforma un segnale analogico in uno digitale.

- L'uscita varia tra due valori di tensione predefiniti, a seconda che l'ingresso superi una soglia superiore o scenda sotto una soglia inferiore.
- **Vantaggi:** consente uno **squadramento** del segnale, garantendo una maggiore stabilità e precisione.



## CLOCK INPUTS

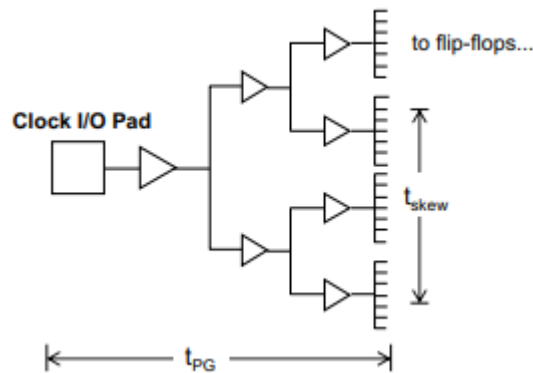
Alcuni ingressi nelle FPGA sono dedicati esclusivamente ai segnali di **clock**, che costituiscono il riferimento per l'evoluzione temporale dell'intera rete sincrona. Il clock deve essere distribuito a tutti i dispositivi in modo sincronizzato e con **bassa latenza** e **basso skew**.

La rete di distribuzione del clock adotta una struttura ad **albero bilanciato**, che garantisce una propagazione uniforme del segnale. Questa configurazione riduce al minimo lo skew, definito come la **differenza temporale tra i fronti del clock** ricevuti dai vari dispositivi.

- **Schema del Ritardo:**
  - Tra l'oscillatore al quarzo, che genera il segnale di clock, e i singoli flip-flop, esistono cammini di propagazione identici. Questo assicura che il **ritardo di propagazione** sia presente, ma lo skew sia pressoché nullo.

## Vantaggi e Considerazioni

- Le FPGA sincrone sfruttano la rete di clock per garantire il funzionamento coordinato di tutti i dispositivi.
- Sebbene le **logiche asincrone** consumino meno potenza rispetto a quelle sincrone, le FPGA sincrone permettono di gestire complessità circuitali che sarebbero difficilmente raggiungibili con logiche asincrone.



## POWER INPUTS

Tutti i dispositivi richiedono ingressi di alimentazione dedicati, tipicamente:

- **VDD** e **GND** per il funzionamento normale.
- **VPP** per la fase di programmazione (se necessaria).

Nei dispositivi di grandi dimensioni, è necessario prevedere **più pin di alimentazione e massa** per mantenere i riferimenti di tensione:

- **Stabili** e insensibili ai transienti di corrente.
- In grado di gestire elevate richieste di corrente senza compromettere la stabilità del sistema.

L'aumento del numero di pin dedicati all'alimentazione riduce la disponibilità di **pin digitali I/O**, limitando il numero di connessioni utilizzabili per scopi di elaborazione o comunicazione.

## ESEMPIO CON XILINX 4000: I/O E CLOCK

Nel caso in cui sia necessario utilizzare **più segnali di clock**, si può impiegare un pin generico come sorgente del clock. Tuttavia, è fondamentale considerare alcune accortezze per garantire la sincronizzazione:

### 1. Ritardi Programmabili:

- Quando il clock non utilizza la rete di distribuzione dedicata, è necessario introdurre ritardi programmabili.
- Questi ritardi permettono di adattare il segnale in modo tale che arrivi a più celle in maniera **sincrona**, eliminando eventuali disallineamenti temporali.

### 2. Reti Dedicare per il Clock:

Le FPGA Xilinx 4000 dispongono di logiche dedicate per la gestione del clock, che permettono di:

- Ridurre lo **skew** (de-skew), ovvero la dispersione tra i fronti del segnale.
- Eseguire operazioni avanzate sul clock, come:
  - **Shift di fase** per sincronizzare circuiti con esigenze temporali differenti.
  - **Divisione o moltiplicazione** della frequenza del clock.
  - **Sintesi di frequenze**, utile per applicazioni che richiedono segnali di clock personalizzati.

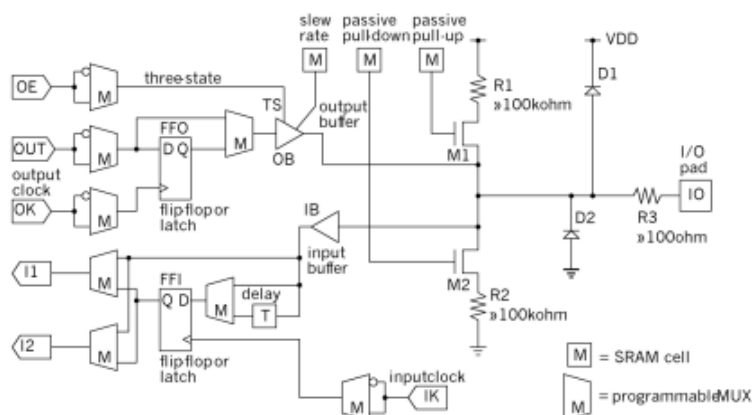
Negli ingressi/uscite, la gestione dei segnali richiede particolare attenzione per evitare conflitti elettrici:

### 1. Conflitti Elettrici:

- Se un PU (Pull-Up) e un PD (Pull-Down) fossero attivi contemporaneamente, potrebbe verificarsi un cortocircuito o un sovraccarico.
- Per evitare ciò, si utilizza un **PU resistivo**, il quale limita la corrente e riduce la dissipazione di potenza, mantenendo il sistema operativo stabile.

### 2. Configurazioni Open-Drain e Open-Source:

- Queste configurazioni evitano conflitti utilizzando:
  - **PU attivo** combinato con **PD passivo**, oppure viceversa.
- La resistenza collegata a VDD garantisce livelli di corrente e tensione compatibili con le caratteristiche del MOSFET, evitando sovraccarichi.



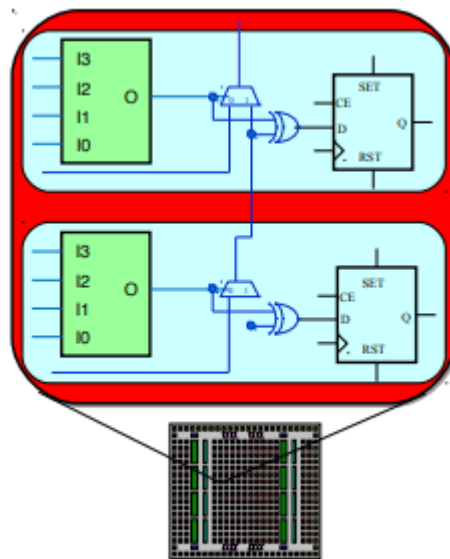
## LOGIC FABRIC

La **Logic Fabric** delle FPGA comprende:

- **Logic Cell:** Include LUT (Lookup Table), Flip-Flop, Carry Logic e MUX.
- **Slice:** Due Logic Cell.

### Esempio:

le FPGA Spartan-3E contengono da 2.000 a 33.000 Logic Cell.



## MEMORY BLOCK

I blocchi di memoria possono essere configurati come RAM o ROM, con porte di lettura e scrittura indipendenti e configurabili.

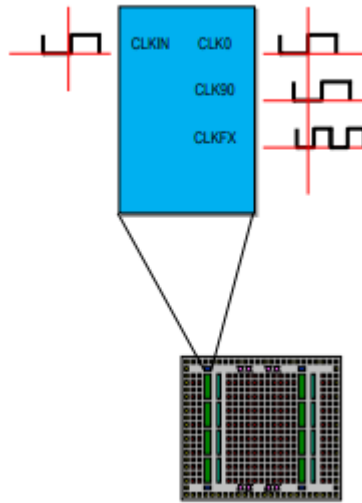
| Block RAM Configurations |       |           |             |
|--------------------------|-------|-----------|-------------|
| Configuration            | Depth | Data bits | Parity bits |
| 16K x 1                  | 16Kb  | 1         | 0           |
| 8K x 2                   | 8Kb   | 2         | 0           |
| 4K x 4                   | 4Kb   | 4         | 0           |
| 2K x 9                   | 2Kb   | 8         | 1           |
| 1K x 18                  | 1Kb   | 16        | 2           |
| 512 x 36                 | 512   | 32        | 4           |

## CLOCK MANAGEMENT BLOCK

Il Clock Manager offre:

- **De-skew del clock.**
- **Shift di fase.**

- **Moltiplicatori e divisori di clock.**
- **Sintesi di frequenze.**



# Microprocessori e Microcontrollori

## Il teorema di Godel

All'inizio del 1900, i matematici, guidati da Hilbert, cercarono di formalizzare la matematica in un sistema assiomatico rigoroso, nel quale tutte le branche derivassero da un insieme di assiomi fondamentali. Tuttavia, Kurt Gödel, con il suo **Teorema di Incompletezza**, dimostrò che:

1. **Esistenza di proposizioni indecidibili:** In ogni sistema matematico assiomatico sufficientemente potente da contenere l'aritmetica elementare, esistono proposizioni che non possono essere né dimostrate né confutate all'interno del sistema stesso, pur essendo vere o false indipendentemente.
2. **Limiti della consistenza:** La consistenza di un sistema matematico  $F$  non può essere dimostrata all'interno dello stesso sistema  $F$ . Ciò implica che non è possibile garantire la totale affidabilità del sistema basandosi esclusivamente sugli assiomi e le regole interne.

Questo risultato ha avuto un profondo impatto sulla logica, la matematica e la filosofia, dimostrando che la matematica non può essere completamente ridotta a un insieme di regole meccaniche.

## La macchina di Turing

Alan Turing affrontò il problema della formalizzazione della computabilità, elaborando la **Macchina di Turing**, un modello teorico capace di rappresentare qualsiasi processo algoritmico.

## Concetti fondamentali

- **Congettura di Church-Turing:** Qualunque problema computazionale che ammette una soluzione algoritmica può essere risolto da una macchina automatica, ossia una macchina di Turing.
- **Effettiva calcolabilità:** Una funzione è definita "effettivamente calcolabile" se i suoi valori possono essere determinati attraverso un processo

puramente meccanico, come quello implementato da una macchina di Turing.

## Struttura della macchina di Turing

### 1. Il nastro:

- Suddiviso in celle discrete, ciascuna delle quali può contenere un simbolo appartenente a un alfabeto finito.
- Il nastro è considerato infinito sia a destra che a sinistra.

### 2. La testina di lettura/scrittura (TLS):

- Può leggere i simboli presenti in una cella del nastro.
- Può scrivere nuovi simboli nella cella.
- Può muoversi lungo il nastro in entrambe le direzioni (destra o sinistra).

### 3. La macchina di controllo:

- È definita da una **quintupla di elementi**:
  - $s$ : lo stato attuale della macchina.
  - $i$ : il simbolo letto dal nastro.
  - $S(s,i)$ : lo stato successivo della macchina.
  - $I(s,i)$ : il simbolo che verrà scritto sul nastro.
  - $V(s,i)$ : la direzione del movimento della testina (destra o sinistra).

## Funzionamento

La macchina opera su intervalli discreti di tempo: ad ogni istante, il suo stato attuale e le azioni future dipendono dallo stato precedente e dal simbolo letto. Questo modello, sebbene teorico, è sufficiente a risolvere qualsiasi problema computazionale che possa essere espresso in termini algoritmici.

## Esempio: Verifica di una sequenza di parentesi

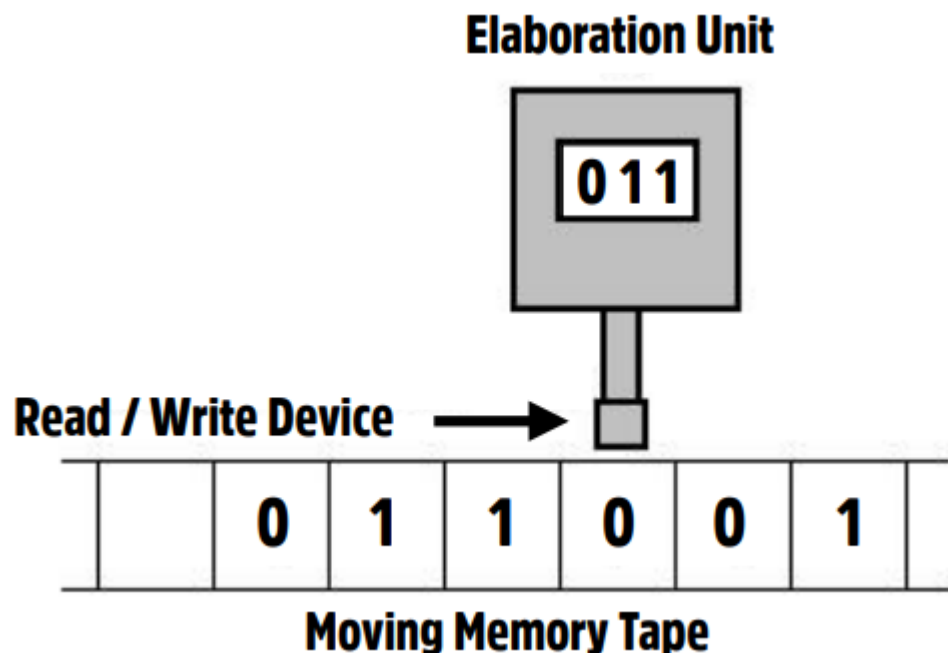
Un esempio pratico dell'applicazione di una macchina di Turing è il controllo della corretta apertura e chiusura di una sequenza di parentesi.

1. **Nastro inizializzato**: La sequenza di parentesi è scritta su un nastro, delimitata da due caratteri speciali, come  all'inizio e alla fine.
2. **Algoritmo**:

- La testina si muove verso destra fino a trovare una parentesi chiusa **)**. La parentesi viene sostituita con un simbolo speciale, ad esempio **x**.
- La testina si muove a sinistra fino a trovare la parentesi aperta **(** corrispondente, che viene anch'essa sostituita con **x**.
- Questo processo si ripete fino a quando si raggiunge il carattere di terminazione **#** su entrambi i lati.

### 3. Esito:

- Se tutti i caratteri sono stati trasformati in **x** e non vi sono parentesi rimanenti, la sequenza è corretta.
- Se restano ancora parentesi aperte o chiuse, la sequenza non è bilanciata.



## I Microprocessori

### Definizione e Modello Computazionale

I microprocessori possono essere considerati come versioni altamente evolute della **macchina di Turing**. Sebbene il numero di stati sia limitato (contrariamente al modello teorico di Turing che prevede stati infiniti), la quantità di stati disponibili nei microprocessori moderni è così grande da poter essere considerata **praticamente infinita**.

Di conseguenza:

- Con un microprocessore e il programma opportuno è possibile risolvere **qualsunque problema di tipo algoritmico**.
- Questa proprietà li rende **Turing-completi**, ossia in grado di esprimere la stessa classe di problemi risolvibili da una macchina di Turing classica.

## Turing Complete e Modelli di Computazione

Un sistema o modello computazionale è definito **Turing-completo** quando è capace di:

1. Simulare qualsiasi altro sistema computazionale equivalente.
2. Risolvere problemi algoritmici esprimibili in termini della macchina di Turing.

### Esempio pratico:

Un modello computazionale basato su

**porte logiche NAND** è Turing-completo, poiché può rappresentare qualsiasi funzione logica e, combinato con la memoria e un controllo sequenziale, risolvere problemi computabili

## La Complessità

### La Complessità Computazionale

La **complessità computazionale** si occupa di analizzare il tempo e le risorse necessarie per risolvere un problema mediante un algoritmo. Per descrivere il tempo richiesto, si utilizza una funzione in base a un parametro caratteristico  $n$ , che rappresenta la "dimensione" del problema (ad esempio, il numero di dati in ingresso).

La complessità di un algoritmo è indicata tramite la **notazione asintotica** (ad esempio,  $O(n)$ ,  $O(n^2)$ ,  $(2^n)$ ) che esprime come il tempo di esecuzione cresce al variare di  $n$ .

### Problemi Polinomiali e Non Polinomiali

#### 1. Problemi Polinomiali (P):

- La complessità cresce con una funzione polinomiale di  $n$  (ad esempio,  $O(n)$ ,  $O(n^2)$ ,  $O(n \log n)$ )
- Gli algoritmi in questa classe sono considerati **efficienti**, in quanto risolvibili in tempi ragionevoli con risorse limitate.

- Esempi: ordinamento con Merge Sort  $O(\log n)$ , ricerca binaria  $O(\log n)$

## 2. Problemi Non Polinomiali (NP):

- Richiedono una crescita esponenziale  $O(n)$ ,  $O(n!)$  o superiore rispetto a  $n$ .
- Questi problemi possono diventare rapidamente **impraticabili** per valori elevati di  $n$ , in quanto il tempo di calcolo aumenta drasticamente.
- Un sottoinsieme di questi problemi è detto **NP-completo**.

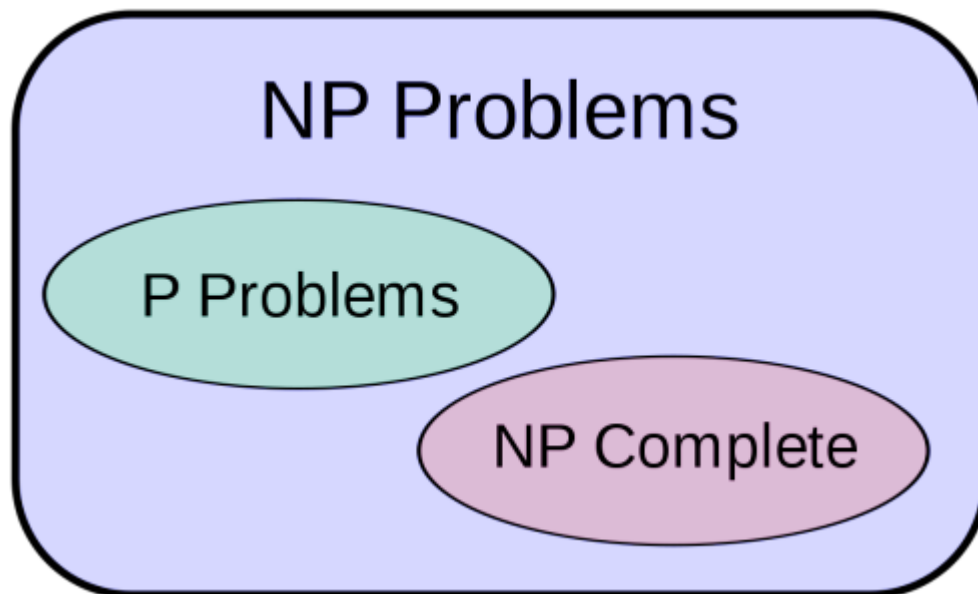
## 3. Problemi NP-Completi:

- Sono i problemi più difficili della classe NP. Ogni problema NP-completo può essere trasformato in qualsiasi altro problema NP-completo attraverso una trasformazione polinomiale.
- Esempio: problema del commesso viaggiatore (TSP), problema di soddisfacibilità booleana (SAT).

| Funzione      | Crescita            | Esempio                                 |
|---------------|---------------------|-----------------------------------------|
| $O(1)$        | Costante            | Accesso diretto a un array              |
| $O(\log n)$   | Logaritmica         | Ricerca binaria                         |
| $O(n)$        | Lineare             | Scansione di un array                   |
| $O(n \log n)$ | Lineare-logaritmica | Algoritmi di ordinamento efficienti     |
| $O(n^2)$      | Quadratica          | Ordinamento a bolle                     |
| $O(2^n)$      | Esponenziale        | Problemi di combinatoria                |
| $O(n!)$       | Fattoriale          | Problema del commesso viaggiatore (TSP) |

Se si riuscisse a dimostrare che anche un solo problema NP-completo è risolvibile in tempo polinomiale, tutti i problemi della classe NP diverrebbero polinomiali. Tuttavia, fino ad oggi, non è stata trovata alcuna dimostrazione né che  $P=NP$  né che  $P \neq NP$

Questa distinzione è fondamentale, poiché molti problemi pratici complessi appartengono a NP-completi (ad esempio, l'ottimizzazione delle risorse, la pianificazione industriale, la crittografia).

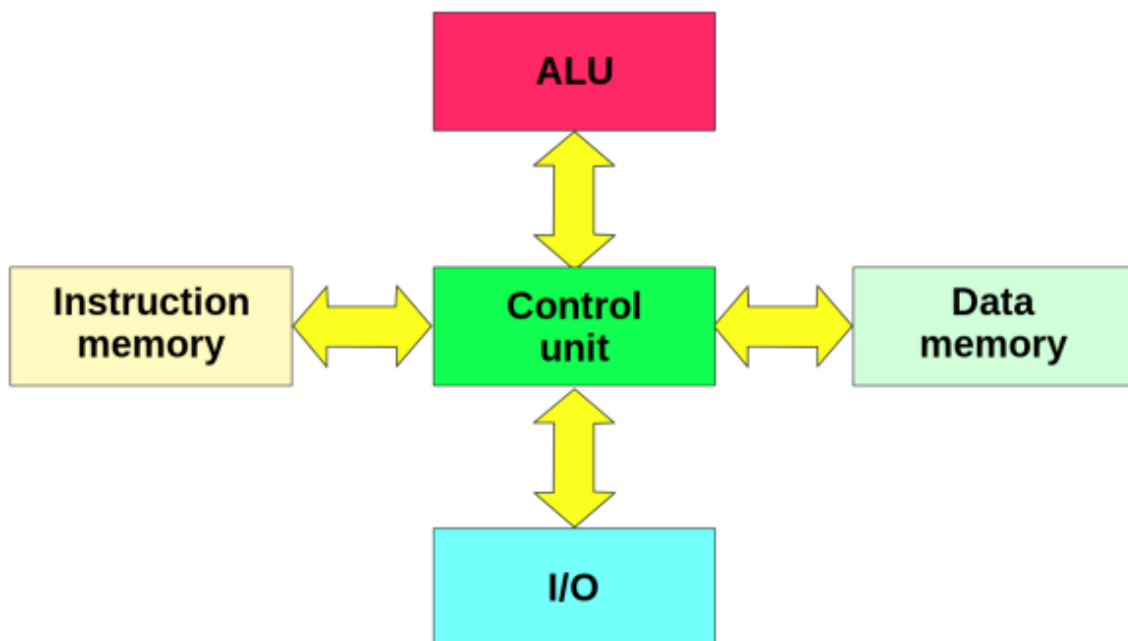


## Architetture dei Microprocessori

### Architettura Harvard

L'architettura Harvard si distingue per la separazione fisica tra la memoria delle istruzioni e la memoria dei dati. Ogni memoria ha un proprio bus dedicato, eliminando il collo di bottiglia che si verifica quando un unico bus è utilizzato sia per i dati sia per le istruzioni.

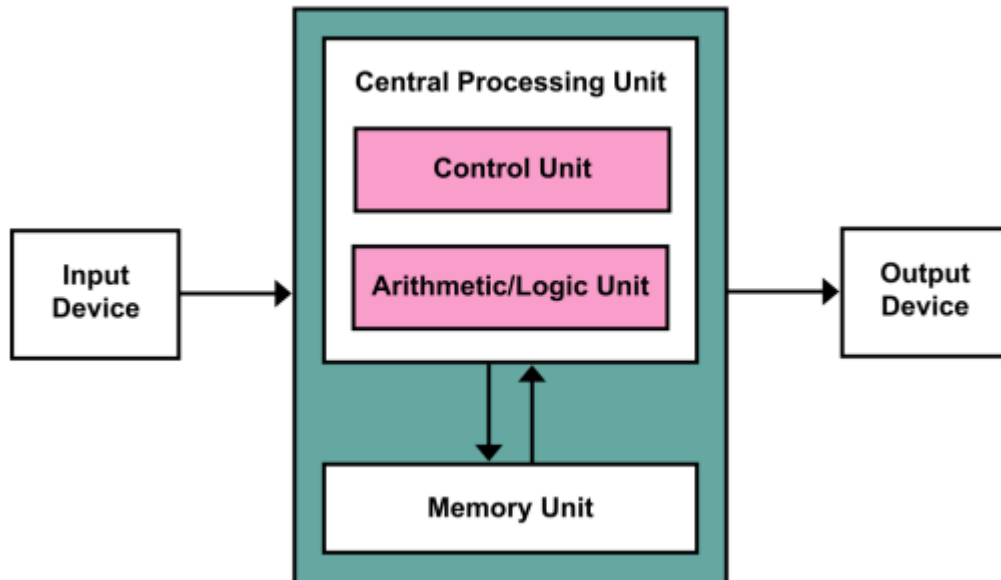
- **Componenti principali:**
  - **ALU (Arithmetic Logic Unit):** esegue operazioni aritmetiche e logiche.
  - **Memoria delle istruzioni:** contiene la sequenza di istruzioni per l'unità di controllo.
  - **Memoria dei dati:** memorizza i dati necessari per l'elaborazione.
  - **Bus separati:** garantiscono una maggiore velocità di trasferimento, migliorando il throughput tra l'unità di controllo e le memorie.
- **Vantaggi:**
  - Velocità superiore grazie alla simultaneità di accesso ai dati e alle istruzioni.
  - Maggiore efficienza nell'esecuzione delle istruzioni.



## Architettura di Von Neumann

L'architettura di Von Neumann utilizza una **memoria condivisa** per dati e istruzioni, accessibile tramite un unico bus a divisione di tempo. In questa configurazione, non vi è distinzione logica tra dati e istruzioni: entrambe sono rappresentate come sequenze binarie.

- **Componenti principali:**
  - **Unità di controllo e memoria condivisa:** la CPU accede a dati e istruzioni tramite lo stesso bus.
  - **ALU:** esegue le operazioni richieste.
- **Vantaggi:**
  - Riduzione dell'area necessaria, rendendo l'implementazione più economica e compatta.
  - Modello semplice e versatile.
- **Svantaggi:**
  - Il bus unico può diventare un collo di bottiglia, limitando la velocità di trasferimento e l'efficienza complessiva.



## System On Chip (SoC)

I microcontrollori moderni integrano in un singolo chip tutte le funzionalità necessarie per gestire sistemi embedded complessi. Questa evoluzione ha portato alla nascita dei **System on Chip (SoC)**.

- **Microcontrollori tradizionali:**

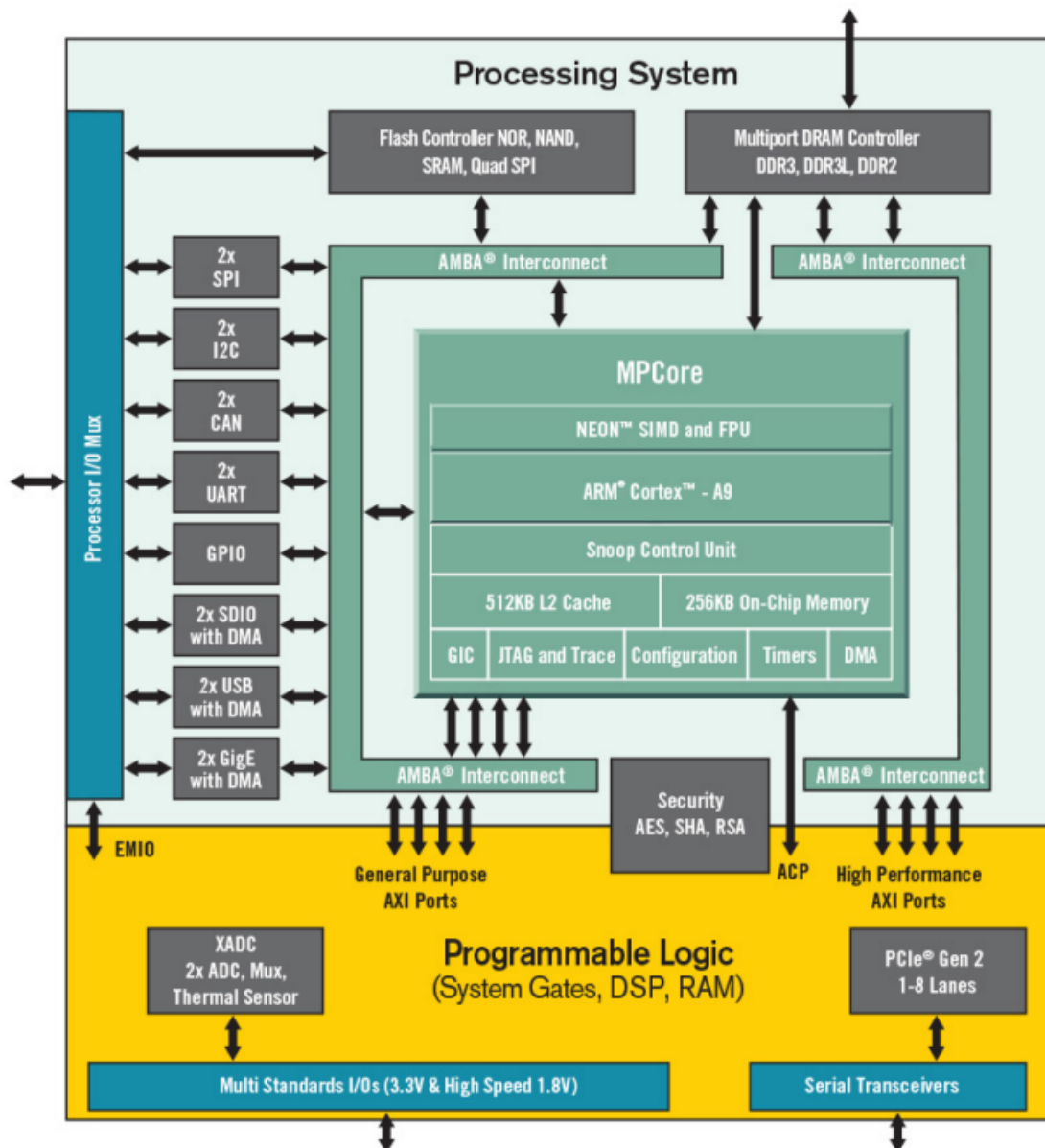
- Composti da diversi chip interconnessi su una scheda PCB (Printed Circuit Board).
- Ogni chip fornisce funzionalità specifiche (ad esempio, comunicazione, Wi-Fi, DSP, ADC/DAC).
- Design modulare, ma richiede più spazio fisico e consumo energetico.

- **SoC (System on Chip):**

- Integrazione di più componenti su un unico chip, come:
  - Processore centrale.
  - Controller per memoria e I/O.
  - Circuiti di comunicazione (Wi-Fi, Bluetooth).
  - Altre periferiche integrate (DSP, ADC/DAC).
- Compattezza e maggiore efficienza rispetto ai sistemi tradizionali.

- **Vantaggi dei SoC:**

- Riduzione delle dimensioni fisiche del sistema.
- Maggiore velocità di elaborazione grazie alla comunicazione diretta tra componenti integrati.
- Efficienza energetica migliorata.



## Strutture e Componenti

### CPU & Memoria

La **CPU (Central Process Unit)** rappresenta il cuore del microcontrollore, derivando da architetture esistenti o nuove progettazioni. Include un set di

istruzioni modificato per gestire l'I/O. Si occupa di eseguire i programmi e gestire le operazioni logiche e aritmetiche.

La memoria nei microcontrollori è suddivisa in tre principali categorie:

### 1. On-Chip Memory (interna):

- **NVRAM:** ospita il sistema operativo e i programmi; è una memoria non volatile.
- **RAM:** memorizza variabili durante l'esecuzione del programma; è volatile.

### 2. Off-Chip Memory (esterna):

- Estensioni di memoria di diversa natura, utili per ampliare le capacità del microcontrollore.

## I/O Pins

I pin di I/O sono elementi fondamentali per la comunicazione con il mondo esterno:

- Possono leggere e scrivere valori **alti (VDD)** o **bassi (GND)**.
- La loro direzione è programmabile.
- Sono spesso **multiplexati** con altre funzioni (ad esempio, linee di comunicazione o funzioni speciali).
- **Resistenze pull-up/pull-down:** utilizzate per accedere a risorse condivise come i bus.
- Supportano correnti di pilotaggio tipiche tra **20-60 mA**.
- Possono generare interruzioni durante le transizioni.

## Timers e Counters

Timer e contatori condividono la stessa struttura ma si differenziano per funzione:

- **Timer:** utilizzano una frequenza interna (clock) per contare intervalli di tempo, utili per generare segnali periodici o transizioni specifiche.
- **Contatori:** utilizzano una frequenza esterna per contare eventi fisici come transizioni di segnali esterni.

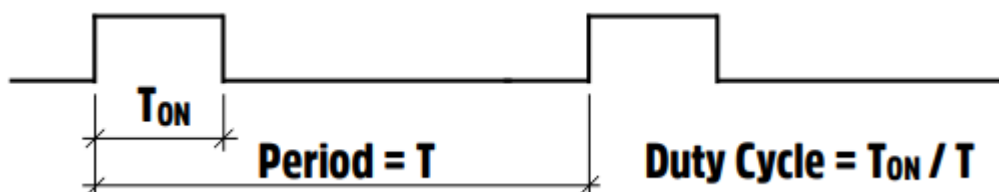
### Funzionamento:

- Entrambi contano fino a un valore programmabile. Al raggiungimento del valore, generano un'interruzione.
- Sono prerequisiti per la generazione di **PWM** (Pulse Width Modulation) e per la comunicazione sincrona esterna (ad esempio, UART).

## PWM (Pulse Width Modulation)

La PWM è una tecnica di modulazione digitale che genera una tensione media variabile tramite impulsi rettangolari:

- Il **duty cycle** rappresenta la proporzione tra il tempo "alto" del segnale e il periodo totale.
- I pin di uscita PWM generano treni di impulsi con caratteristiche programmabili (duty cycle e periodo).
- Per implementare la PWM si utilizzano timer interni, che in tal caso non possono essere utilizzati per altre funzioni.



## Capture Inputs

Gli ingressi di cattura contano eventi esterni:

- Un contatore associato all'ingresso si incrementa a ogni evento esterno (es. transizione da 0 a 1).
- Possono generare interruzioni al raggiungimento di un valore prestabilito.
- Utili per monitorare segnali o eventi periodici.

## A/D Converters

Gli ADC (Analog-to-Digital Converters) trasformano segnali analogici continui (es. tensione) in valori digitali discreti:

- Non sono presenti in tutti i microcontrollori.
- La conversione è iniziata via software.
- Precisione tipica: **8, 12 o 16 bit**.

- Gamma di tensioni convertibili: da 0 a circa **2-3 volte la tensione di alimentazione digitale**.

## D/A Converters

I DAC (Digital-to-Analog Converters) effettuano l'operazione inversa rispetto agli ADC:

- Convertono valori digitali in segnali analogici.
- Non sono presenti in tutti i microcontrollori.
- Precisione tipica: **1-2 parole del processore**.
- Corrente di pilotaggio tipica: **20-60 mA**.

## UART

Il **UART** è un dispositivo periferico programmabile utilizzato per la comunicazione digitale a bassa velocità in banda base. Si trova spesso nei sistemi di comunicazione sincroni o asincroni.

- **Caratteristiche principali:**
  - In passato, era un dispositivo esterno al core computazionale, ma oggi può essere integrato.
  - Supporta comunicazioni asincrone, come quelle utilizzate nei protocolli **RS232**.
  - Adatto per la trasmissione seriale di dati uno alla volta senza richiedere un segnale di clock esterno.

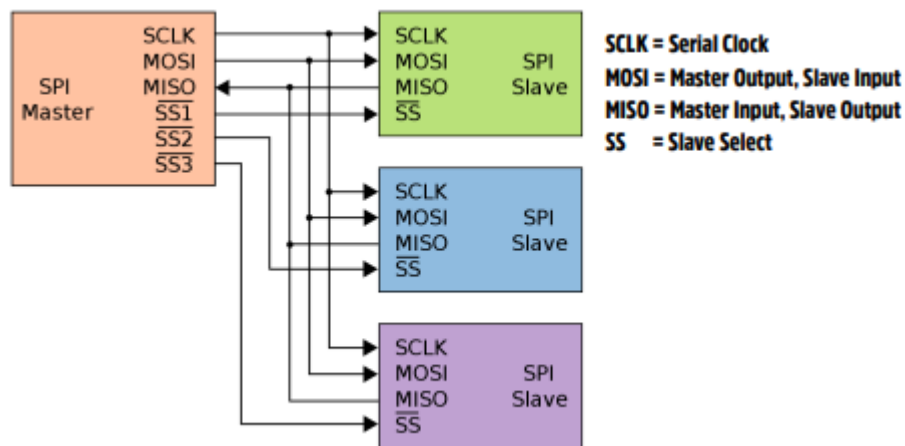


## SPI Protocol (Serial Peripheral Interface)

Lo **SPI** è un protocollo di comunicazione sincrona, inizialmente sviluppato come standard proprietario da Motorola, poi passato a Freescale e infine a NXP.

- **Caratteristiche principali:**
  - Utilizzato per la comunicazione tra dispositivi digitali.
  - Supporta diverse topologie, con quella **Master/Slave** come la più semplice.

- Linee principali:
  - **SCLK (Serial Clock)**: segnale di clock condiviso.
  - **MOSI (Master Output, Slave Input)**: dati inviati dal master.
  - **MISO (Master Input, Slave Output)**: dati inviati dallo slave.
  - **SS (Slave Select)**: selezione dello slave attivo.
- È ideale per trasferimenti ad alta velocità grazie al clock condiviso.

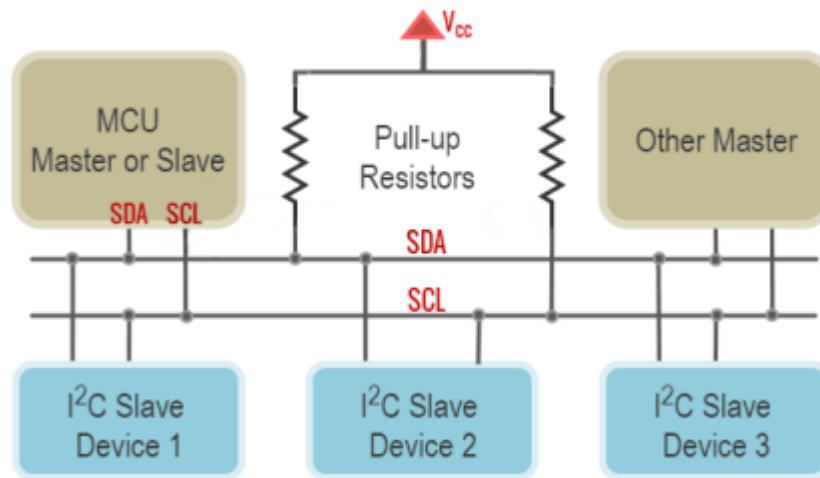


## Inter-Integrated Circuit Protocol

L'**I<sup>2</sup>C** è un protocollo di comunicazione asincrona standard, sviluppato originariamente da Philips (ora NXP), e reso disponibile gratuitamente.

### • Caratteristiche principali:

- Supporta una configurazione multi-master e multi-slave.
- Utilizzato per la comunicazione tra un microcontrollore e le sue periferiche.
- È progettato per operazioni a bassa velocità su due soli fili: **SDA** (Serial Data) e **SCL** (Serial Clock).
- È semplice da implementare e adatto a collegare dispositivi come sensori o EEPROM.

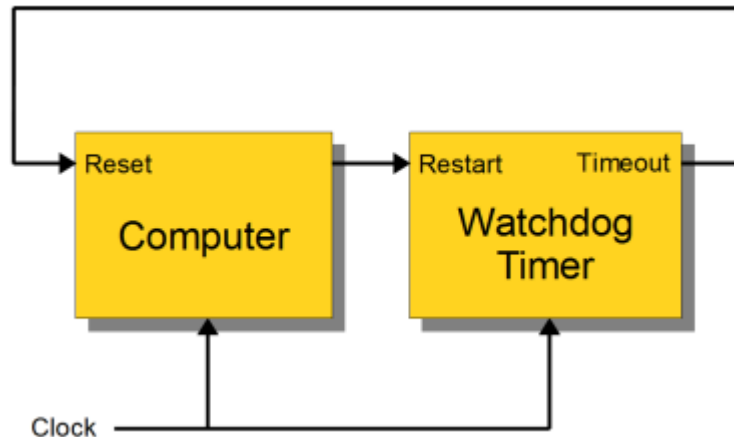


## Watchdog Timer (WDT)

Il **Watchdog Timer** è una funzione di sicurezza integrata nei microcontrollori, essenziale per garantire l'affidabilità dei sistemi, soprattutto in applicazioni industriali.

- **Caratteristiche principali:**

- Monitora l'esecuzione del software: se il software non resetta il WDT entro un tempo predefinito, si presume che il sistema sia in uno stato di errore.
- Quando il timer scade:
  - Genera un'interruzione non mascherabile.
  - Avvia una routine di sicurezza che mette il sistema in uno stato sicuro.
- Utilizzato per rilevare il blocco di algoritmi critici o malfunzionamenti imprevisti.
- Fondamentale per applicazioni in cui la continuità operativa e la sicurezza sono prioritarie.



## Memoria e Decodifica

### Organizzazione della Memoria

Ogni microprocessore o microcontrollore ha un'organizzazione predefinita del proprio spazio di memoria, che può essere interna o esterna. Questo comporta che:

- **Mappatura delle periferiche:**

Le periferiche esterne vengono assegnate a specifici indirizzi nello spazio di memoria. Ogni dispositivo deve essere correttamente abilitato quando il processore accede al suo indirizzo assegnato. Questo processo è noto come **decodifica degli indirizzi**.

- **Partizionamento della memoria:**

Lo spazio di memoria è suddiviso in sezioni predefinite:

- **Memoria di sistema:** Riservata al firmware o alle operazioni di base del microcontrollore.
- **Memoria utente:** Spazio per l'esecuzione dei programmi applicativi.
- **Stack:** Un'organizzazione a pila che supporta operazioni come chiamata e ritorno da funzioni.

La configurazione della memoria varia in base alle decisioni del progettista, ma la struttura generale segue sempre regole definite.

### MEMORY INTERFACE

Il microprocessore comunica con la memoria esterna tramite bus di indirizzi e dati. Per ottimizzare il numero di pin disponibili, i segnali possono essere

**multiplexati.**

- **Bus multiplexato:**

- Gli stessi pin possono trasportare dati o indirizzi in momenti diversi del ciclo di comunicazione.
- Durante la prima fase, il microprocessore emette gli indirizzi, che vengono memorizzati in registri esterni chiamati **Address Latch** (es. 373). Questi registri sono abilitati tramite il segnale **ALE (Address Latch Enable)**.

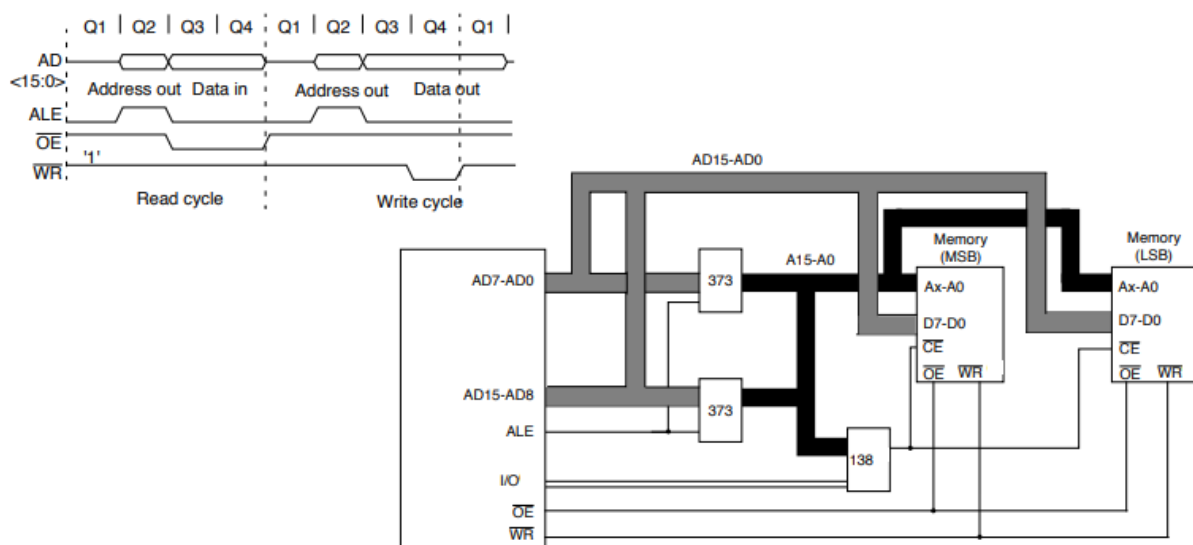
- **Gestione dei dati:**

Una volta memorizzati gli indirizzi, i segnali sulle linee diventano dati e vengono inviati alle porte di input/output della memoria.

- **Segnali di controllo:**

- **OE (Output Enable):** Abilita la lettura dalla memoria.
- **WR (Write):** Abilita la scrittura.
- **CE (Chip Enable):** Attiva il chip di memoria selezionato, generato tramite un decodificatore (es. 138).

In presenza di un bus dati a 16 bit, si possono usare due chip di memoria paralleli: uno per la parte bassa (D0-D7) e uno per la parte alta (D8-D15).



## Decodifica

La decodifica degli indirizzi consente al microcontrollore di selezionare specifiche sezioni di memoria o periferiche. La quantità di memoria indirizzabile dipende dal numero di bit del bus di indirizzi.

- **Spazio di indirizzamento:**

Con un bus di indirizzi a 16 bit, è possibile indirizzare  $2^{16} = 64\text{KB}$

- **Caso pratico:**

- Se si utilizza un chip di memoria da 8 KB (8K word), sono necessari 13 bit di indirizzo per coprirne l'intero spazio ( $2^{13}=8\text{KB}$ ).
- I 3 bit rimanenti vengono usati per selezionare quale tra 8 chip attivare, tramite un segnale di **Chip Enable** generato da una funzione combinatoria degli indirizzi.

$$(es. CE = F(ADDR_{15} - 13))$$

- **Esclusione di zone di memoria:**

Si può configurare il microcontrollore per ignorare certe aree di memoria, non includendo il chip corrispondente o non attivandolo.

## Modalità di interfaccia Periferica

Nei sistemi embedded, la comunicazione tra il microprocessore o microcontrollore e le periferiche può avvenire tramite tre principali modalità di interfaccia: **Polling**, **Interrupt** e **DMA (Direct Memory Access)**. Ciascuna modalità presenta vantaggi e svantaggi, adattandosi a specifiche esigenze applicative.

### Polling

Il **Polling** è una tecnica ciclica in cui il processore verifica continuamente lo stato delle periferiche tramite il controllo dei bit associati alle periferiche stesse.

- **Meccanismo di funzionamento:**

In un ciclo continuo, il processore interroga ciascuna periferica per verificare la presenza di nuovi dati. Se disponibili, li acquisisce e procede con la successiva periferica.

- **Vantaggi:**

- Semplicità di implementazione.
- Nessuna necessità di hardware complesso.

- **Svantaggi:**

- **Inefficienza:** La CPU rimane costantemente impegnata nel controllo delle periferiche, riducendo le risorse disponibili per altre operazioni.
- **Incompatibilità con il multitasking:** La natura continua del polling impedisce alla CPU di svolgere efficacemente più compiti in parallelo.
- **Dipendenza dal numero di periferiche:** Aumentando il numero di periferiche, cresce il carico sulla CPU.

## Interrupt

La modalità **Interrupt** consente alla CPU di interrompere il normale flusso di esecuzione per gestire eventi specifici generati da periferiche.

- **Meccanismo di funzionamento:**

Quando una periferica genera un evento (ad esempio, la ricezione di nuovi dati), invia un segnale di interrupt alla CPU. La CPU:

1. Sospende il programma corrente.
2. Esegue una **routine di interrupt**, un sottoprogramma che gestisce l'evento.
3. Dopo aver terminato la routine, riprende l'esecuzione del programma interrotto.

- **Caratteristiche degli interrupt:**

- **Mascherabili:** Possono essere disattivati temporaneamente, ad esempio durante l'esecuzione di una routine critica.
- **Non mascherabili (NMI):** Interrupt prioritari che non possono essere disattivati, utilizzati per eventi critici come surriscaldamento della CPU o caduta di alimentazione.
- **Preemptive:** Gli interrupt possono interrompere la routine in esecuzione, se hanno priorità più alta.

- **Vantaggi:**

- Ottimizza l'uso della CPU, che non è costretta a controllare ciclicamente le periferiche.
- Ideale per periferiche ad alta velocità.

- **Svantaggi:**

- Richiede un sistema di gestione degli interrupt (es. **Programmable Interrupt Controller**).
- La frequenza eccessiva degli interrupt può rallentare il sistema.

## DMA - Direct Memory Access

Il **DMA (Direct Memory Access)** è una modalità avanzata in cui le periferiche possono accedere direttamente alla memoria principale, senza coinvolgere la CPU per il trasferimento dei dati.

- **Meccanismo di funzionamento:**

1. La CPU avvia il trasferimento configurando il **controller DMA**.
2. Il controller DMA gestisce autonomamente il trasferimento di blocchi di dati tra la memoria e le periferiche.
3. La CPU può continuare le sue attività, intervenendo solo per iniziare o monitorare il processo.

- **Vantaggi:**

- **Efficienza elevata:**

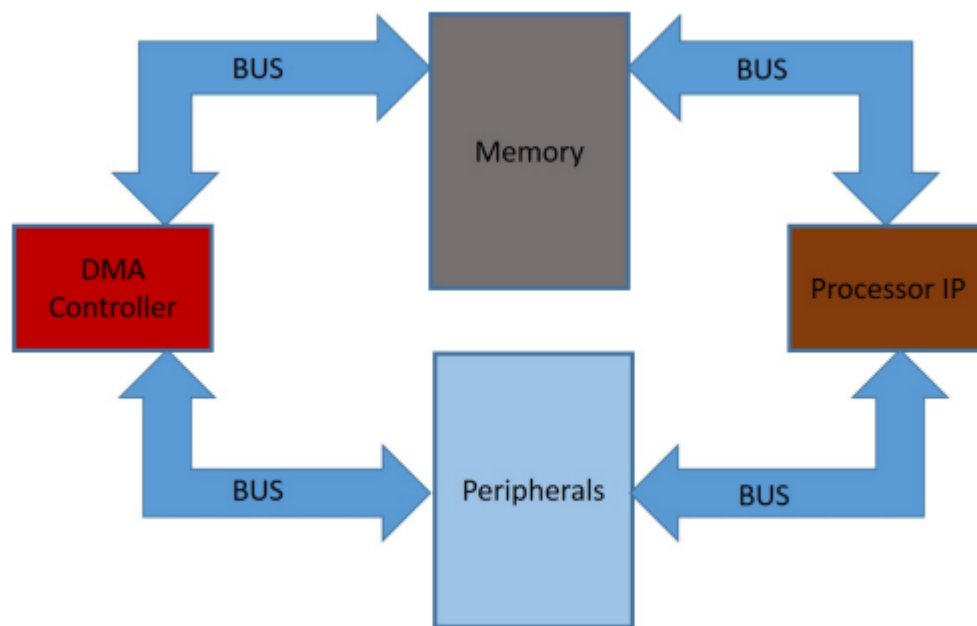
- Riduce il carico di interrupt sulla CPU.
    - Permette il trasferimento rapido di grandi quantità di dati tra memoria e periferiche.
  - Consente la comunicazione tra periferiche che operano a velocità diverse senza rallentamenti.

- **Svantaggi:**

- Richiede hardware dedicato (il controller DMA).
  - Maggiore complessità rispetto alle altre modalità.

- **Applicazioni tipiche:**

- Sistemi con periferiche ad alta velocità, come dischi rigidi, interfacce di rete o sistemi audio/video.
  - Situazioni in cui gli interrupt sarebbero troppo frequenti, rallentando la CPU.



# DSP & SoC - MAC

## DSP & SoC

### Digital Signal Processor

I Digital Signal Processors (DSP) sono microprocessori progettati specificamente per l'elaborazione numerica dei segnali. Il loro sviluppo è nato dalla necessità di tradurre le operazioni sui segnali da sistemi analogici a digitali, sfruttando la maggiore flessibilità, prevedibilità e stabilità offerte dal software digitale rispetto agli equivalenti sistemi hardware analogici. I DSP consentono di modificare parametri e algoritmi in modo dinamico senza la necessità di cambiare l'hardware, rendendo queste architetture ideali per una vasta gamma di applicazioni.

I DSP sono ampiamente utilizzati in ambiti embedded, tra cui telecomunicazioni, controllo di processo, radar, automotive e gestione di controllori. La loro capacità di elaborare segnali in modo continuo e in finestre temporali stringenti è cruciale per evitare la perdita di informazioni, che potrebbe compromettere le applicazioni finali.

Le architetture DSP si differenziano significativamente dai processori general purpose grazie a ottimizzazioni hardware e software per specifici carichi di lavoro, come l'elaborazione dei segnali digitali. Questa specificità li ha resi uno standard in molti settori.

### RAZIONALIZZARE

La progettazione dei DSP è stata guidata dalla necessità di creare microprocessori dedicati per domini specifici, che successivamente sono diventati una classe standard di dispositivi. Questo processo ha seguito il progressivo predominio dell'elaborazione digitale su quella analogica, grazie ai seguenti vantaggi:

1. **Prevedibilità:** I risultati delle elaborazioni digitali sono deterministici e replicabili.
2. **Stabilità:** I sistemi digitali sono meno sensibili a variazioni ambientali rispetto agli analogici.

3. **Flessibilità:** La modifica del software è molto più semplice e veloce rispetto alla riconfigurazione hardware.
4. **Compattezza hardware:** L'elaborazione digitale consente di integrare più funzionalità in meno spazio.

## PROPRIETA'

Le peculiarità hardware che distinguono i DSP dai processori general purpose includono:

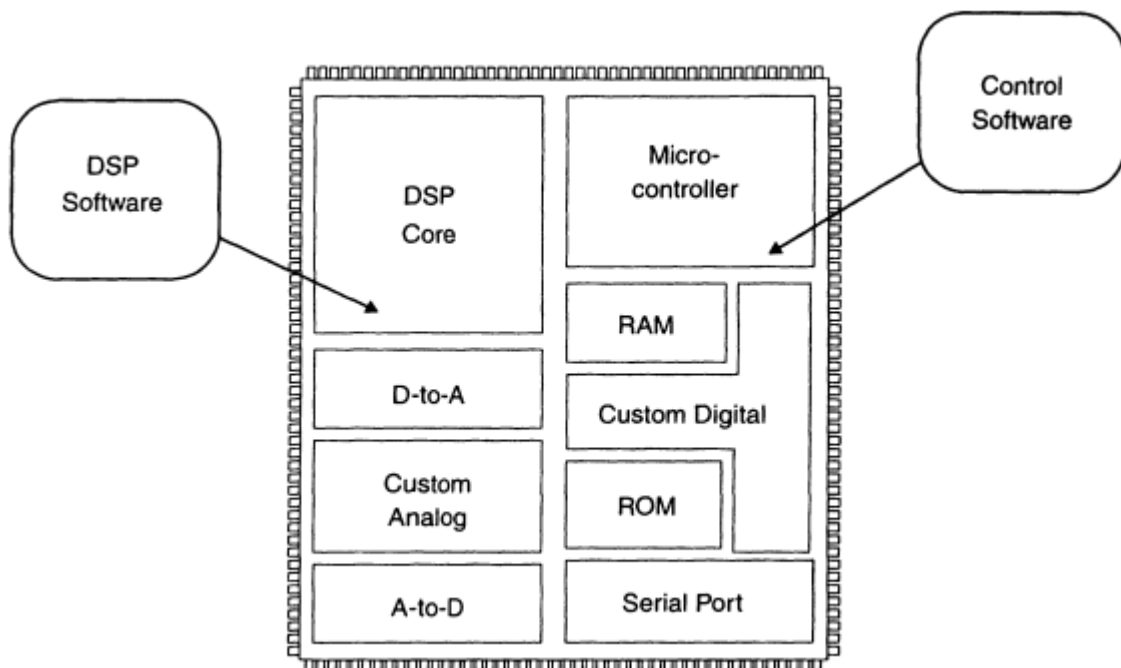
1. **Hardware dedicato per operazioni MAC (Multiply and Accumulate):**  
Essenziale per operazioni matematiche ripetitive ad alta velocità, come la convoluzione e i filtri.
2. **Accessi multipli alla memoria:** Consentono al DSP di accedere simultaneamente a più aree della memoria, migliorando l'efficienza.
3. **Modalità di indirizzamento dedicate:** Facilitano il trattamento di flussi di dati continui e sequenze numeriche.
4. **Strutture di controllo dedicate:** Ottimizzano la gestione dei segnali e delle operazioni.
5. **Periferiche dedicate on-chip:** ADC (Analog-to-Digital Converter), DAC (Digital-to-Analog Converter), e altre periferiche specifiche sono integrate direttamente nel chip per ridurre la latenza.
6. **Alta frequenza di funzionamento:** La velocità dei DSP è estremamente elevata, rendendoli adatti ad applicazioni ad alta intensità computazionale come radar e telecomunicazioni.

## DSP on System on Chip

Un'evoluzione importante è rappresentata dall'integrazione dei DSP nei SoC (System on Chip). Questo approccio combina i vantaggi dei DSP con altre funzionalità integrate su un unico chip, come core RISC (Reduced Instruction Set Computer) o CISC (Complex Instruction Set Computer), periferiche e memoria. I SoC con DSP offrono:

1. **Efficienza:** Architetture RISC semplificate garantiscono alte velocità di esecuzione per operazioni elementari.
2. **Flessibilità:** Le istruzioni più complesse delle architetture CISC possono essere utilizzate per compiti specifici, garantendo compatibilità con algoritmi avanzati.

3. **Compattezza:** L'integrazione riduce i costi di produzione e il consumo energetico.



## Hardware

I Digital Signal Processors (DSP) rappresentano una categoria di processori altamente specializzati progettati per gestire ed elaborare segnali digitali in tempo reale. Le loro caratteristiche hardware e architetture li rendono adatti a un'ampia varietà di applicazioni, tra cui telecomunicazioni, elaborazione audio e video, e grafica tridimensionale.

## Aritmetica

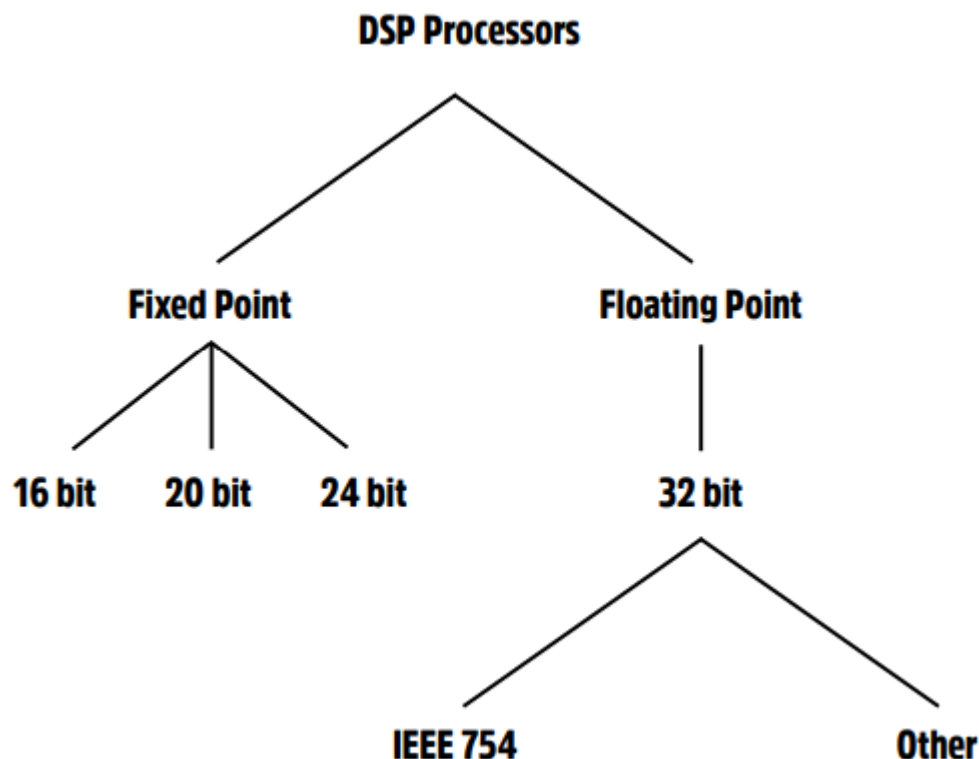
L'aritmetica nei DSP può essere implementata utilizzando rappresentazioni a **virgola fissa** o **virgola mobile**:

- **Virgola fissa:** I numeri sono rappresentati con una posizione predefinita per la parte intera e quella frazionaria. Questo approccio è efficiente dal punto di vista computazionale e spesso utilizzato in applicazioni come il condizionamento di segnali vocali, dove la dinamica del segnale è limitata.
- **Virgola mobile:** Utilizza standard come l'IEEE 754 per rappresentare numeri con una gamma dinamica molto più ampia. Questo tipo di aritmetica è ideale per applicazioni come l'elaborazione di immagini e la grafica tridimensionale, che richiedono maggiore precisione.

Le dimensioni dei dati possono variare a seconda delle applicazioni:

- 16, 20 o 24 bit per la rappresentazione fixed-point.
- 32 bit per la rappresentazione floating-point.

Ogni tipologia di DSP è ottimizzata per gestire in modo efficiente specifiche configurazioni aritmetiche e requisiti applicativi.



## MAC - Multiply and Accumulator

Il **MAC (Multiply and Accumulate)** è una componente hardware essenziale nei DSP, progettata per eseguire operazioni di moltiplicazione e somma in un singolo ciclo di clock. Questa unità è fondamentale per l'elaborazione del segnale digitale, dove le operazioni di somma e prodotto costituiscono la base di molti algoritmi.

### Funzionamento:

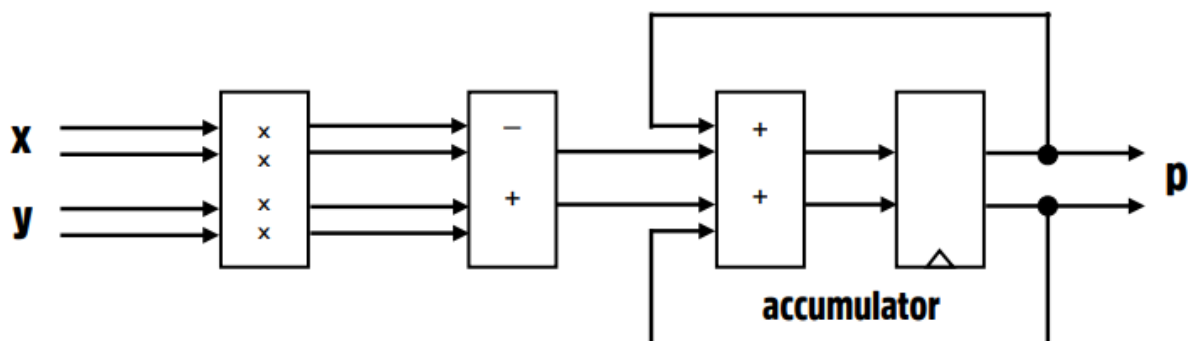
1. Moltiplica due numeri (ad esempio, due componenti di numeri complessi: parte reale e immaginaria).
2. Accumula il risultato con un valore precedentemente memorizzato in un registro.
3. Completa l'operazione in un solo ciclo di clock.

### ESEMPIO:

Per calcolare il prodotto di due numeri complessi, sono necessarie quattro moltiplicazioni ( $\times$ ), una somma ( $+$ ) e una sottrazione ( $-$ ). Un'unità MAC può svolgere queste operazioni simultaneamente, migliorando l'efficienza computazionale.

### Benefici:

- **Prestazioni elevate:** Permette di elaborare segnali in tempo reale.
- **Riduzione dei cicli di clock:** L'integrazione del MAC consente di eseguire operazioni complesse in un tempo estremamente ridotto.



### Von Neumann Architecture

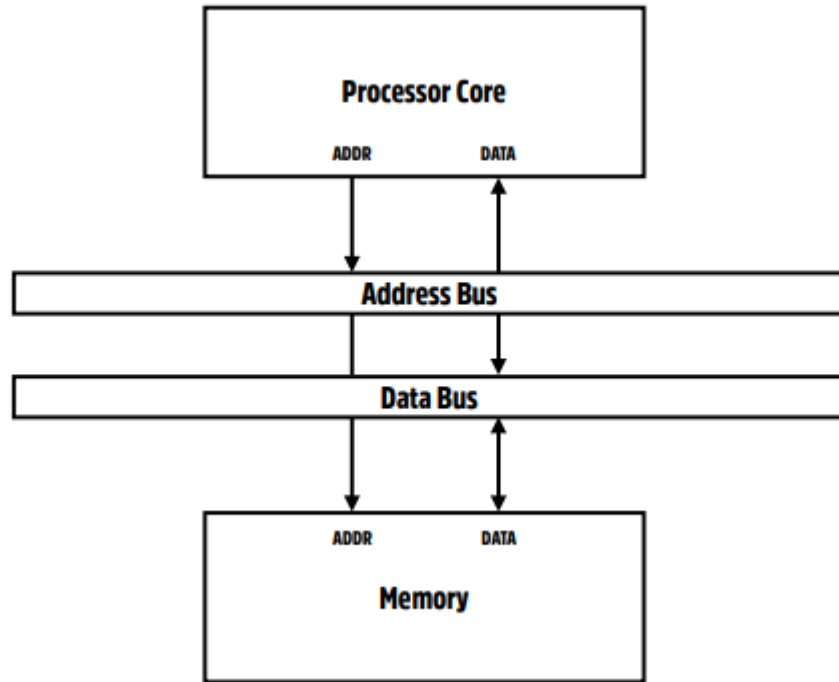
L'architettura di Von Neumann è caratterizzata dall'utilizzo di una singola memoria condivisa per dati e istruzioni. Questa memoria è connessa al processore tramite un unico bus, che viene utilizzato alternativamente per accedere alle istruzioni e ai dati.

### Vantaggi:

- **Semplicità progettuale:** La condivisione di un solo bus riduce la complessità del sistema.
- **Costo contenuto:** Meno componenti rispetto ad architetture alternative.

### Svantaggi:

- **Collo di bottiglia:** L'accesso condiviso al bus rallenta le operazioni in quanto istruzioni e dati competono per la stessa risorsa.
- **Inefficienza nei DSP:** L'elaborazione in tempo reale richiede una velocità di accesso che questa architettura non può garantire.



## Harvard Architecture

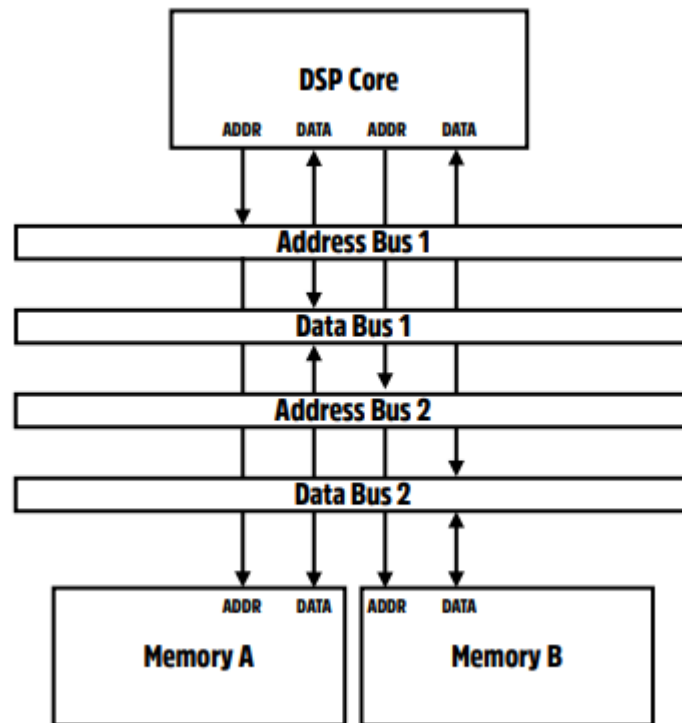
L'architettura Harvard separa fisicamente le memorie per dati e istruzioni, ognuna con il proprio bus dedicato. Questo approccio elimina il collo di bottiglia tipico dell'architettura di Von Neumann, permettendo accessi paralleli.

### Vantaggi:

- **Accesso simultaneo:** Permette al processore di leggere istruzioni e dati contemporaneamente, aumentando la velocità di esecuzione.
- **Efficienza:** Adatta per applicazioni in tempo reale, dove le prestazioni sono critiche.

### Svantaggi:

- **Costo maggiore:** L'aggiunta di bus e memorie separate aumenta i costi hardware.



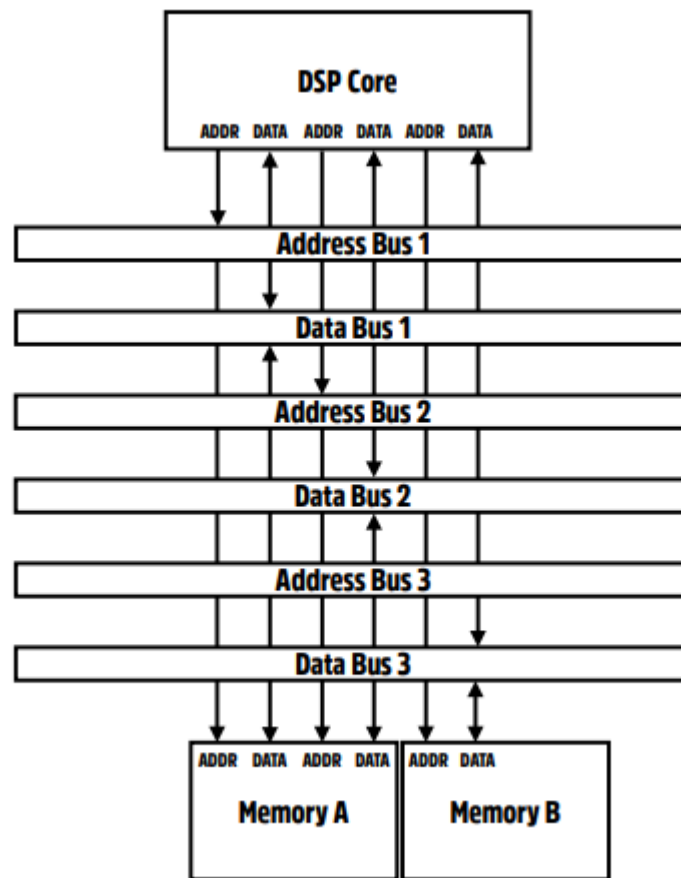
## Harvard Architecture - Dual-port Memory

Una variante avanzata dell'architettura Harvard prevede l'uso di **memorie dual-port**, che consentono due accessi simultanei alla stessa memoria. Questa configurazione è particolarmente utile nei DSP, dove è frequente la necessità di operare su più dati contemporaneamente.

Con l'uso di tre bus distinti:

1. **Bus per le istruzioni:** Dedito alla lettura delle istruzioni.
2. **Due bus per i dati:** Permettono la lettura simultanea di due operandi e la scrittura del risultato.

Questo approccio aumenta significativamente la velocità di esecuzione, eliminando i ritardi legati al trasferimento dei dati.



## Adressing Modes

Il termine *indirizzamento* si riferisce alla modalità con cui un microprocessore accede alla memoria, utilizzando indirizzi specifici per localizzare i dati necessari. Nei Digital Signal Processors (DSP), sono presenti modalità di indirizzamento che non si trovano comunemente nei microprocessori convenzionali. Questa caratteristica è dovuta ai requisiti software di alto livello richiesti dagli algoritmi di elaborazione del segnale digitale, che necessitano di modalità di accesso alla memoria altamente ottimizzate per garantire efficienza e velocità.

## DSP Adressing Examples

### INDIRIZZAMENTO IMMEDIATO

L'indirizzamento immediato utilizza una costante specificata direttamente all'interno dell'istruzione. Questo metodo consente di accedere immediatamente a un valore specifico senza dover interagire con la memoria esterna o i registri. È una modalità comune anche nei microprocessori convenzionali.

## **INDIRIZZAMENTO INDIRETTO**

Nell'indirizzamento indiretto, l'indirizzo di memoria desiderato è contenuto in un registro. L'operazione specifica quale registro dell'unità centrale contiene l'indirizzo della locazione di memoria a cui applicare l'operazione. Questa modalità è utile per accedere a dati memorizzati in modo dinamico e è anch'essa presente nei microprocessori standard.

## **INDIRIZZAMENTO PRE-POST INCREMENTO**

Questo tipo di indirizzamento è progettato per facilitare l'accesso a sequenze di dati, come accade in algoritmi che elaborano array o stream di valori consecutivi. Con il pre-incremento, il registro dell'indirizzo viene incrementato prima dell'accesso alla memoria, mentre con il post-incremento l'incremento avviene dopo l'accesso. Questa funzionalità automatizza il passaggio alla variabile successiva o precedente, evitando di doverlo specificare esplicitamente in ogni istruzione.

## **INDIRIZZAMENTO CIRCOLARE**

L'indirizzamento circolare è particolarmente utile per la gestione di buffer circolari, comunemente utilizzati negli algoritmi di elaborazione del segnale, come le code FIFO (*First-In-First-Out*). In questa modalità, un registro puntatore supporta un indirizzamento basato su modulo, mantenendo il puntatore all'interno di un intervallo definito. Quando il buffer raggiunge il suo limite, il puntatore ritorna automaticamente all'inizio, consentendo un utilizzo continuo senza sovrascrivere dati non ancora elaborati.

## **INDIRIZZAMENTO CON INVERSIONE DI BIT**

L'indirizzamento con inversione di bit è specificamente progettato per ottimizzare l'implementazione della Trasformata Rapida di Fourier (FFT). In questa modalità, i dati sono riorganizzati in base all'inversione dei bit del loro indirizzo, facilitando l'accesso ai dati nella sequenza richiesta dall'algoritmo FFT. Questo approccio migliora l'efficienza evitando operazioni di riordino dispendiose durante l'elaborazione.

## **Instruction Set**

Il set di istruzioni dei processori DSP (Digital Signal Processor) include sia istruzioni standard, comuni ai microprocessori convenzionali, sia istruzioni non standard progettate per ottimizzare l'elaborazione di segnali e algoritmi complessi. Tra le istruzioni principali troviamo:

## ISTRUZIONI NON STANDARD

### 1. MAC (Multiply and Accumulate):

- Questa istruzione esegue la moltiplicazione di due numeri e accumula il risultato in un registro, rendendo possibile la somma di prodotti in un unico ciclo macchina. Tale funzionalità è essenziale per algoritmi DSP come filtri digitali e trasformate rapide di Fourier (FFT).

### 2. Block Floating Point:

- Permette di gestire blocchi di memoria con una maggiore precisione, assegnando a molti significandi lo stesso esponente. Questo approccio è utile per operazioni che richiedono una manipolazione accurata dei numeri in virgola mobile.

### 3. Hardware Loops:

- Consentono l'esecuzione di cicli direttamente in hardware, senza necessità di istruzioni software per l'incremento del contatore o il controllo della condizione di fine ciclo. Questa caratteristica accelera significativamente le operazioni iterative.

### 4. Nested Hardware Loops:

- Implementano cicli annidati direttamente in hardware. La coalescenza hardware consente di eseguire cicli all'interno di altri cicli senza il supporto esplicito di software, migliorando l'efficienza.

### 5. Data Block Movement:

- Facilita il trasferimento rapido di blocchi di dati da una posizione di memoria a un'altra, riducendo il tempo richiesto per spostamenti di grandi volumi di dati.

## Miscellanea

Oltre al set di istruzioni, i DSP includono caratteristiche hardware aggiuntive che ampliano le loro funzionalità e li rendono adatti a un'ampia varietà di applicazioni:

### 1. Porte di comunicazione:

- **Porte seriali e parallele:** Consentono l'integrazione diretta con altri dispositivi o sistemi per lo scambio di dati.

### 2. Timer e Contatori:

- Utilizzati per gestire eventi temporali e sincronizzazioni.

### 3. Convertitori A/D e D/A:

- Permettono la conversione tra segnali analogici e digitali, rendendo i DSP fondamentali per applicazioni in cui è necessario interfacciarsi con il mondo fisico.

### 4. Gestione degli Interrupt:

- Fornisce un controllo efficiente sugli eventi asincroni, riducendo i tempi di latenza nelle risposte.

### 5. DMA (Direct Memory Access):

- Consente il trasferimento diretto dei dati tra memoria e periferiche senza coinvolgere la CPU, liberando risorse per altre operazioni.

### 6. Gestione energetica:

- **Power Management:** Include funzionalità avanzate per ridurre il consumo energetico, come:
  - **Low Voltage Operation:** Funzionamento a bassa tensione per ridurre il consumo.
  - **Sleep Mode e Idle Frequency Control:** Spegnimento o riduzione delle attività hardware non necessarie.

## Multifunctional SoC

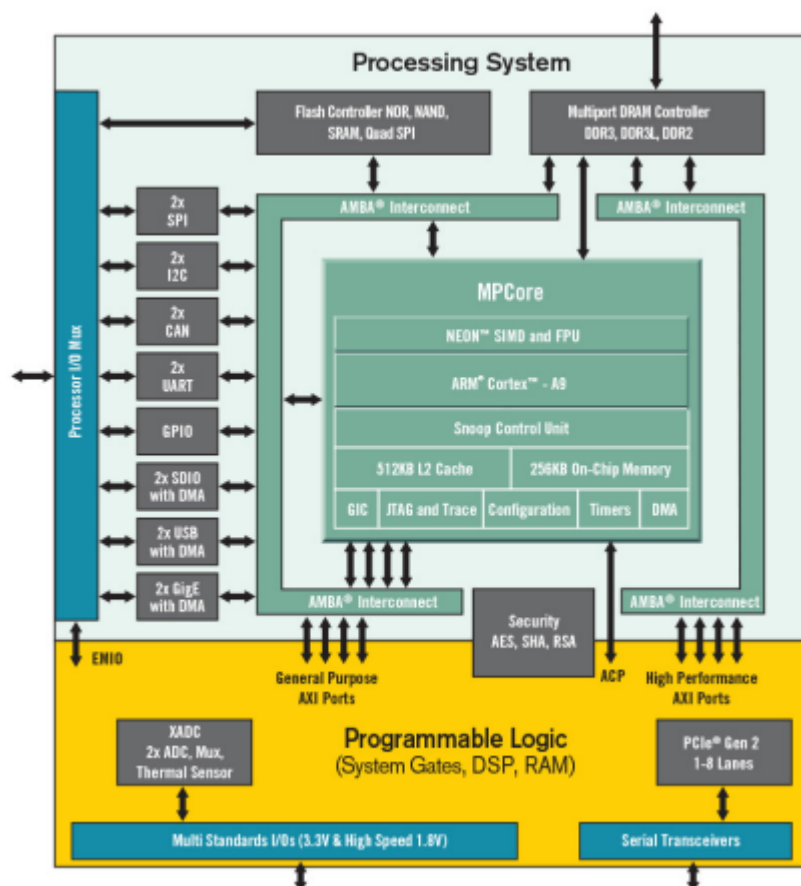
I System on Chip (SoC) multifunzionali rappresentano una delle evoluzioni più avanzate della tecnologia dei semiconduttori. Grazie a progressi significativi nei processi di realizzazione dei transistor MOS, è possibile integrare diverse tecnologie, come memorie non volatili, transistor bipolari e convertitori analogico-digitali, all'interno dello stesso chip. Questa complessità richiede un approccio ingegneristico avanzato per sviluppare soluzioni capaci di combinare blocchi funzionali differenti in un'unica piattaforma.

Un esempio significativo è rappresentato dall'architettura Xilinx Zynq-7000, che integra un core ARM ad alte prestazioni con componenti dedicati, come logiche programmabili tipo FPGA, consentendo un'ampia flessibilità progettuale. Questi SoC possono essere visti come una soluzione economica e tecnologica alternativa ai core DSP dedicati, offrendo una piattaforma unica per processi di elaborazione complessi.

## Xilinx Zynq - 7000

L'architettura Xilinx Zynq-7000 è progettata per combinare le capacità di elaborazione di un core ARM con una logica programmabile FPGA. Questa integrazione consente di:

- Sviluppare soluzioni personalizzate per applicazioni specifiche.
- Implementare funzionalità DSP programmabili attraverso la logica FPGA.
- Ottimizzare l'interfaccia tra core di calcolo e hardware dedicato, migliorando prestazioni ed efficienza energetica.



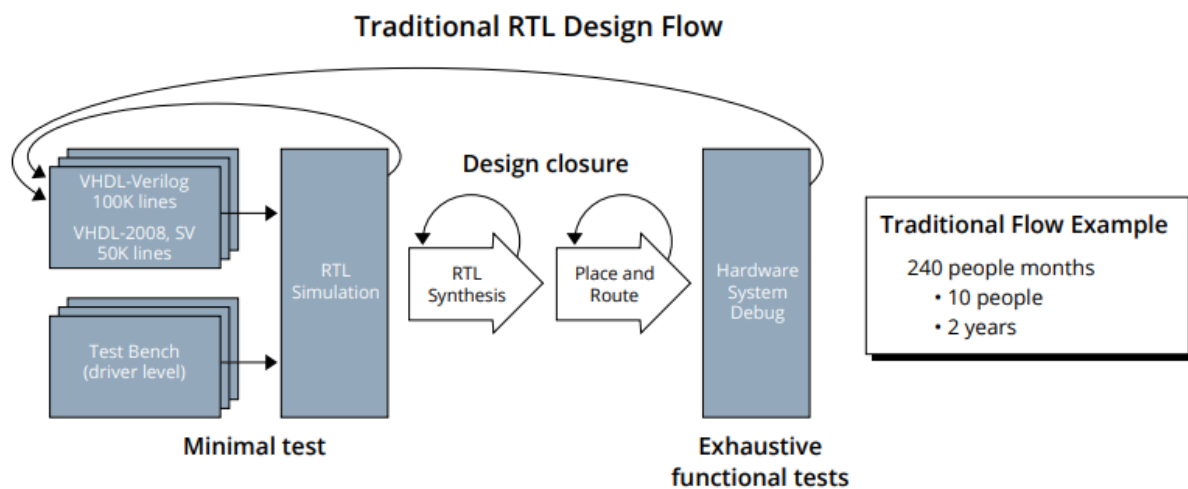
## Conventional RTL Synthesis

Il flusso tradizionale di progettazione hardware, noto come RTL (Register Transfer Level) Synthesis, è utilizzato per lo sviluppo di circuiti digitali complessi come ASIC e FPGA. I principali passaggi includono:

1. **Codice sorgente RTL:** Descrizione del comportamento del circuito con linguaggi come VHDL o Verilog (circa 100.000 righe di codice).

2. **Simulazione RTL:** Verifica iniziale del comportamento logico per identificare errori e garantire la correttezza funzionale del design.
3. **Sintesi RTL:** Conversione del codice RTL in una netlist gate-level, rappresentante le connessioni logiche implementabili fisicamente.
4. **Place and Route:** Allocazione fisica dei componenti logici e routing delle connessioni sul chip.
5. **Debug di sistema:** Verifica esaustiva su hardware reale per garantire il corretto funzionamento in tutte le condizioni operative.

Questo processo è lungo e intensivo, richiedendo fino a 240 mesi/persona.



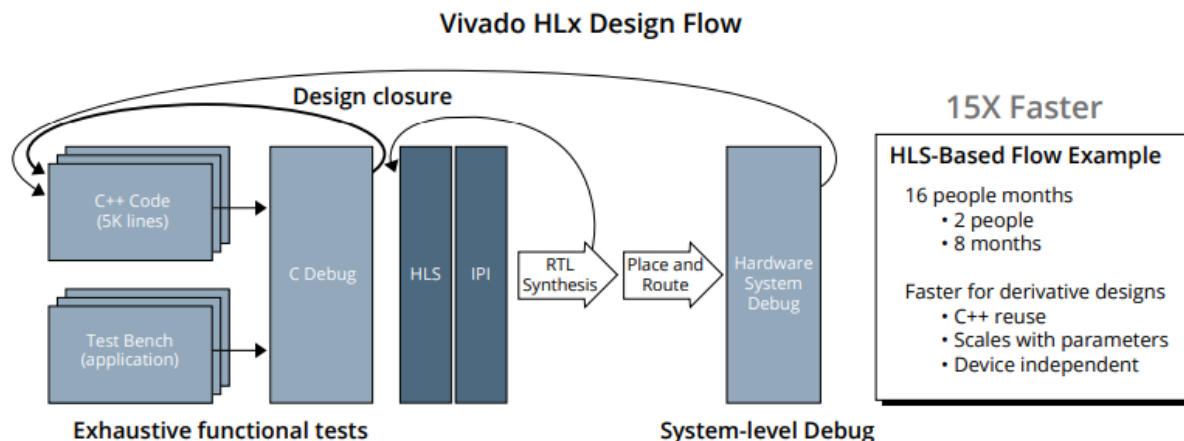
## Xilinx Vivado HLS

Vivado HLS (High-Level Synthesis) rappresenta un approccio innovativo alla progettazione hardware, riducendo significativamente i tempi di sviluppo. I principali passaggi sono:

1. **Codice C++:** La progettazione hardware inizia con una descrizione in linguaggio C++ (circa 5.000 righe di codice), più compatta e astratta rispetto ai linguaggi RTL.
2. **Test bench C++:** Simulazione del comportamento funzionale del design per individuare e correggere errori nelle prime fasi.
3. **Debug a livello C:** Identificazione rapida degli errori con possibilità di iterare rapidamente le modifiche.
4. **Sintesi HLS:** Conversione automatica del codice C++ in descrizione RTL ottimizzata per prestazioni, area e consumo energetico.

5. **Integrazione e Packaging IP:** Integrazione del design con moduli hardware esistenti (IP cores).
6. **Place and Route e Debug:** Sintesi RTL e test su hardware reale per garantire la funzionalità completa.

Questo flusso consente di ridurre i tempi di sviluppo fino a 16 mesi/persona, accelerando di 15 volte rispetto al metodo tradizionale.



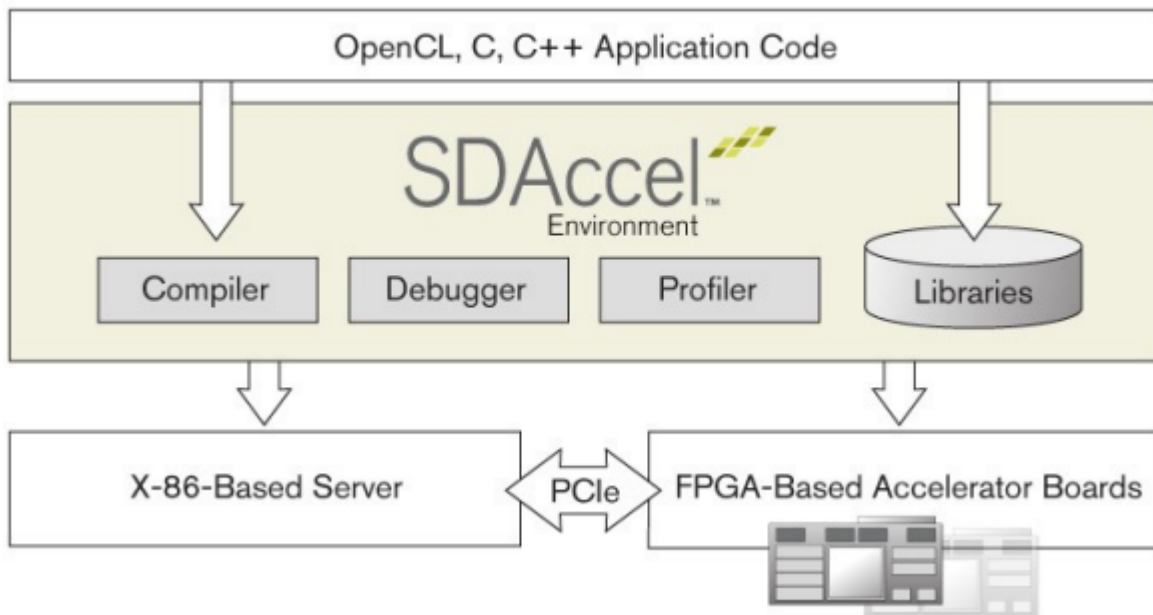
## Xilinx SDAccel

SDAccel offre un flusso di progettazione per sistemi basati su FPGA con focus su elevata astrazione e ottimizzazione automatica. I suoi elementi chiave includono:

- **Scrittura ad alto livello:** Codifica in linguaggi di alto livello come C, C++ o OpenCL.
- **Compilazione:** Conversione automatica del codice in una rappresentazione hardware sintetizzabile.
- **Debugging:** Identificazione e correzione degli errori tramite simulazioni comportamentali.
- **Profiling:** Analisi delle prestazioni (latenza, throughput) per identificare e risolvere i colli di bottiglia.
- **Librerie predefinite:** Utilizzo di librerie ottimizzate per FPGA che semplificano l'implementazione di funzioni complesse.

SDAccel consente di massimizzare le risorse dell'FPGA, semplificando lo sviluppo di applicazioni ad alte prestazioni.

## SDAccel - CPU/GPU Development Experience on FPGAs



## Multiply and Accumulate (MAC)

Il Multiply and Accumulate (MAC) è un acceleratore fondamentale per velocizzare le operazioni nel campo dell'elaborazione digitale dei segnali (DSP). Consente di combinare, in un unico ciclo di clock, operazioni di moltiplicazione, somma, differenza e accumulazione. Questa capacità riduce significativamente i tempi di calcolo e migliora l'efficienza computazionale, rendendolo ideale per applicazioni che richiedono elaborazioni complesse.

## Digital Signal Processing Domain

I processori DSP (Digital Signal Processors) sono progettati per eseguire algoritmi complessi quali filtraggi, trasformate di Fourier e convoluzioni. Questi algoritmi si basano su operazioni esprimibili come somme di prodotti. Ad esempio, per calcolare una convoluzione discreta tra due sequenze, i DSP utilizzano intensivamente moltiplicazioni e somme, rendendo il MAC uno strumento indispensabile.

Un esempio numerico di convoluzione discreta è il seguente:

Se  $x[n]$  e  $y[n]$  rappresentano segnali di input, il MAC può eseguire in modo ottimale ogni passo di questa operazione, riducendo la latenza rispetto a implementazioni tradizionali.

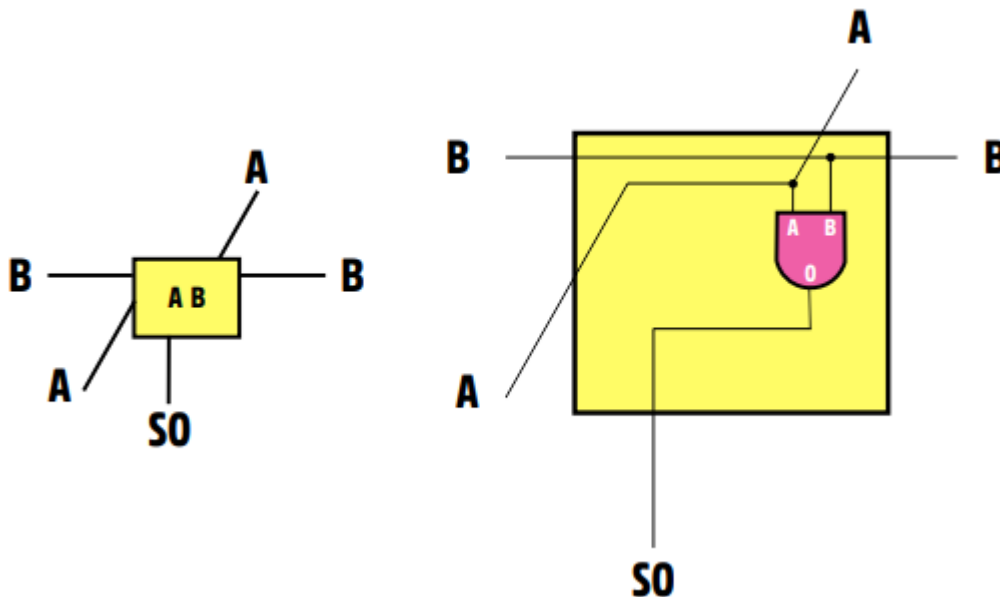
## The Complex Product

Il prodotto complesso viene calcolato utilizzando la moltiplicazione binaria. Questa tecnica divide l'operazione in prodotti parziali tra i bit di due numeri, che vengono poi sommati insieme propagando i riporti necessari. Ad esempio, moltiplicando i numeri binari (11 in decimale) e (13 in decimale), il risultato finale è una sequenza di operazioni logiche e aritmetiche che portano a (143 in decimale).

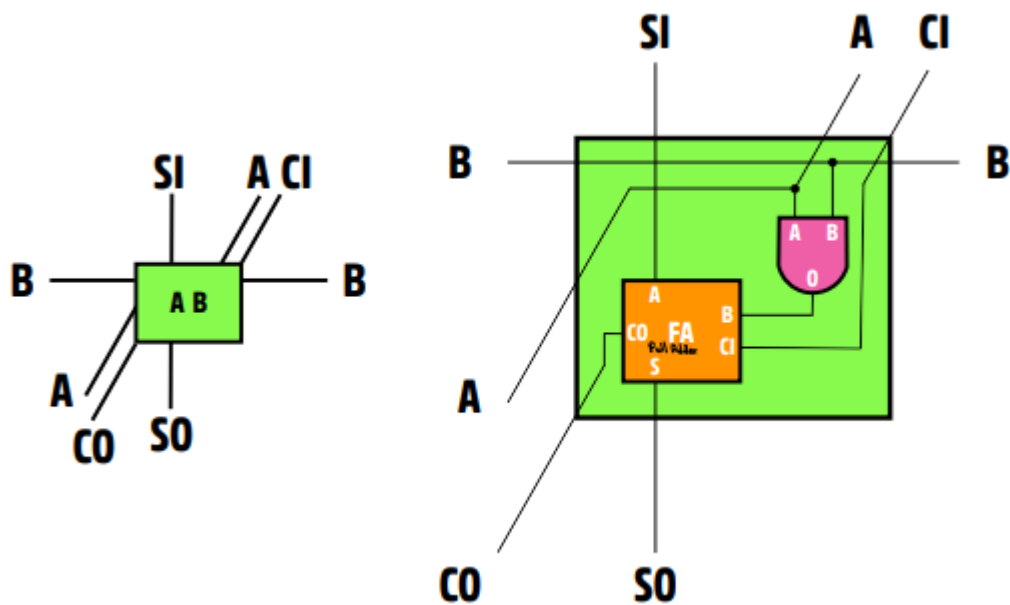
## Multiplication Blocks

La costruzione di un moltiplicatore parallelo è suddivisa in diverse righe di componenti funzionali, ciascuna con un ruolo specifico:

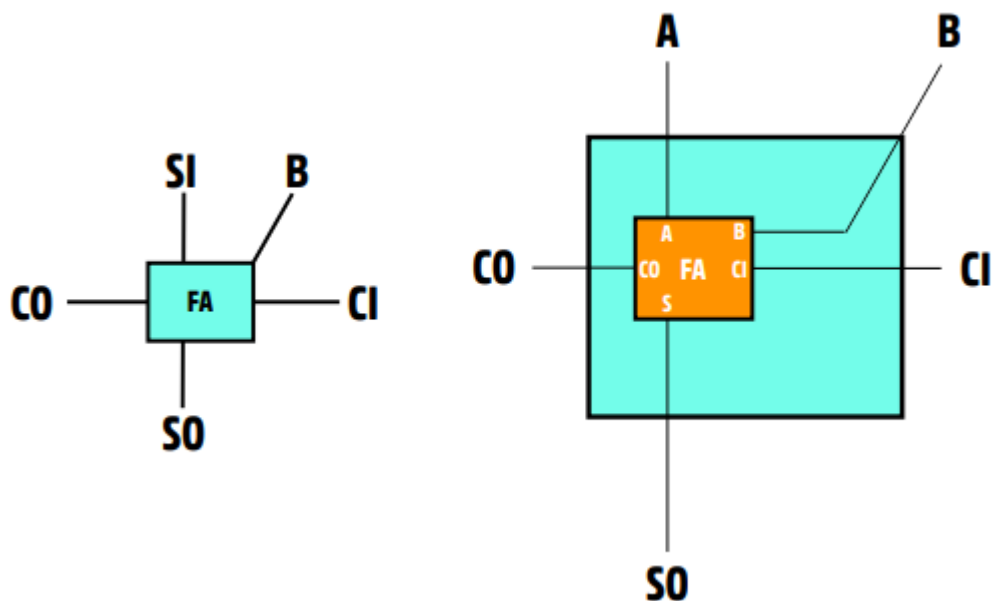
1. **Prima riga:** Contiene moltiplicatori logici (AND) per generare i prodotti parziali dei bit.



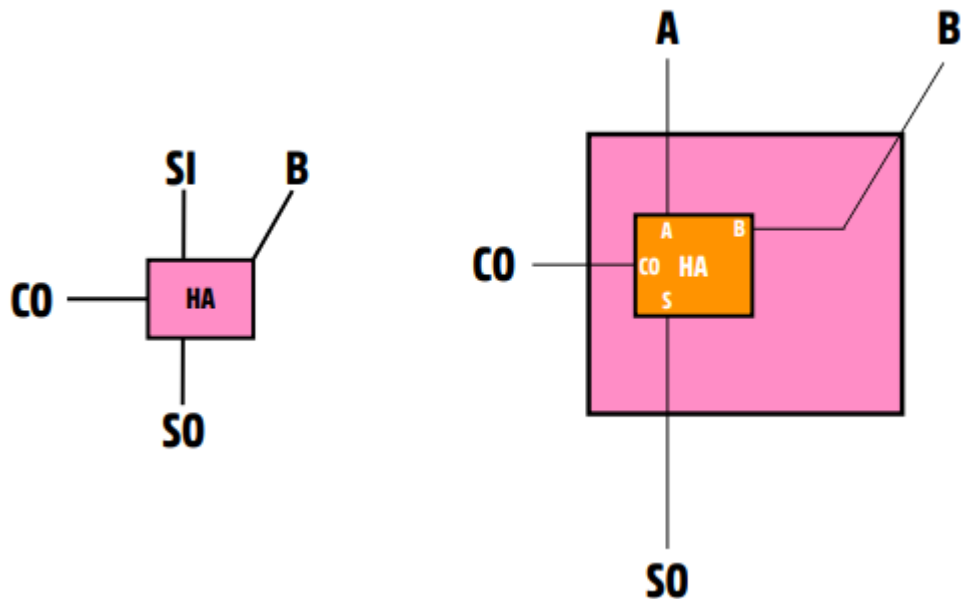
1. **Seconda riga:** Aggiunge half-adder per sommare prodotti parziali senza riporti.
2. **Terza riga:** Integra full-adder per gestire somme con riporti, combinando tre ingressi: due prodotti parziali e un carry-in.



3. **Quarta riga:** Introduce componenti combinati che includono moltiplicatori (AND) e full-adder per ottimizzare il calcolo.



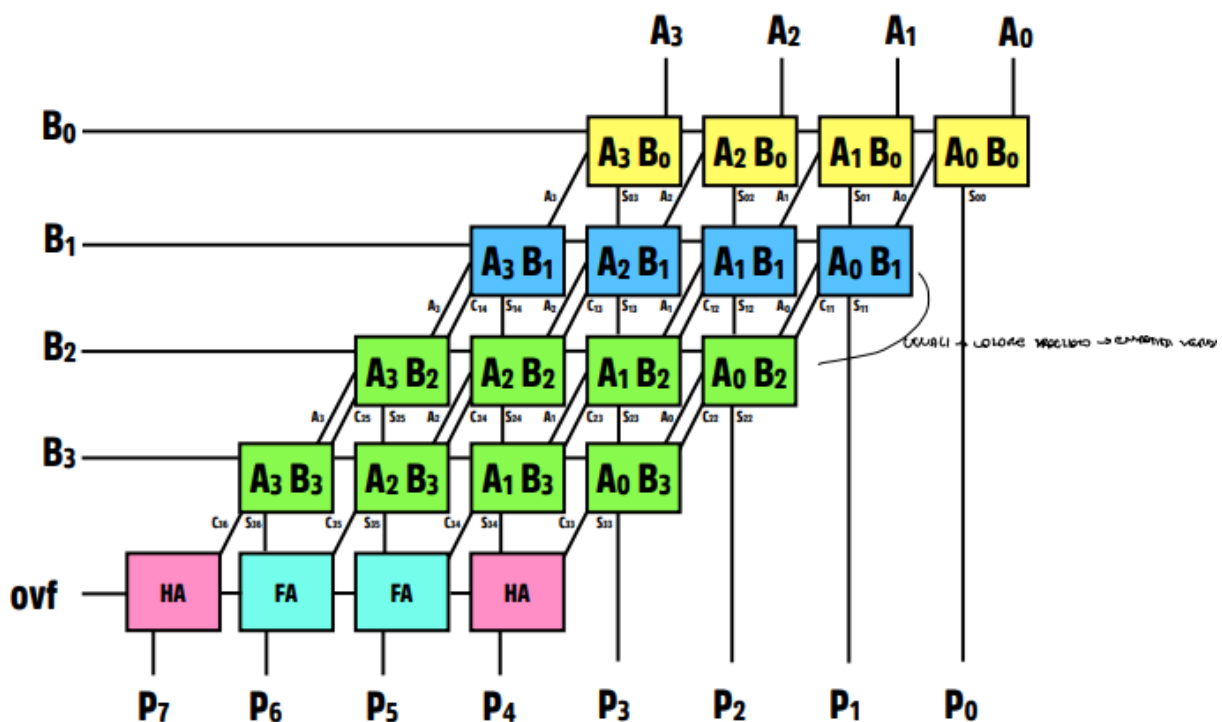
4. **Quinta riga:** Utilizza blocchi avanzati per migliorare l'efficienza complessiva, gestendo somme più complesse.



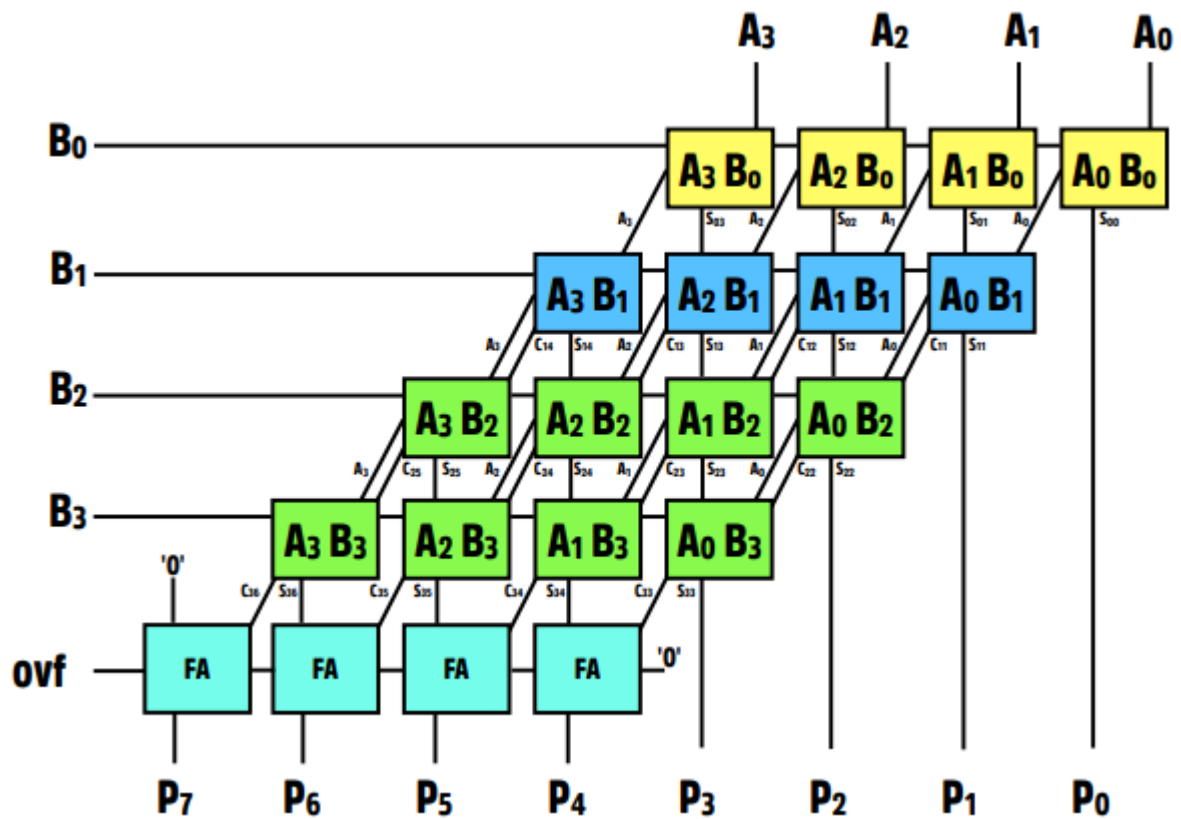
## Parallel Multiplier

Un moltiplicatore parallelo può essere implementato in diverse versioni, ciascuna ottimizzata per specifiche esigenze:

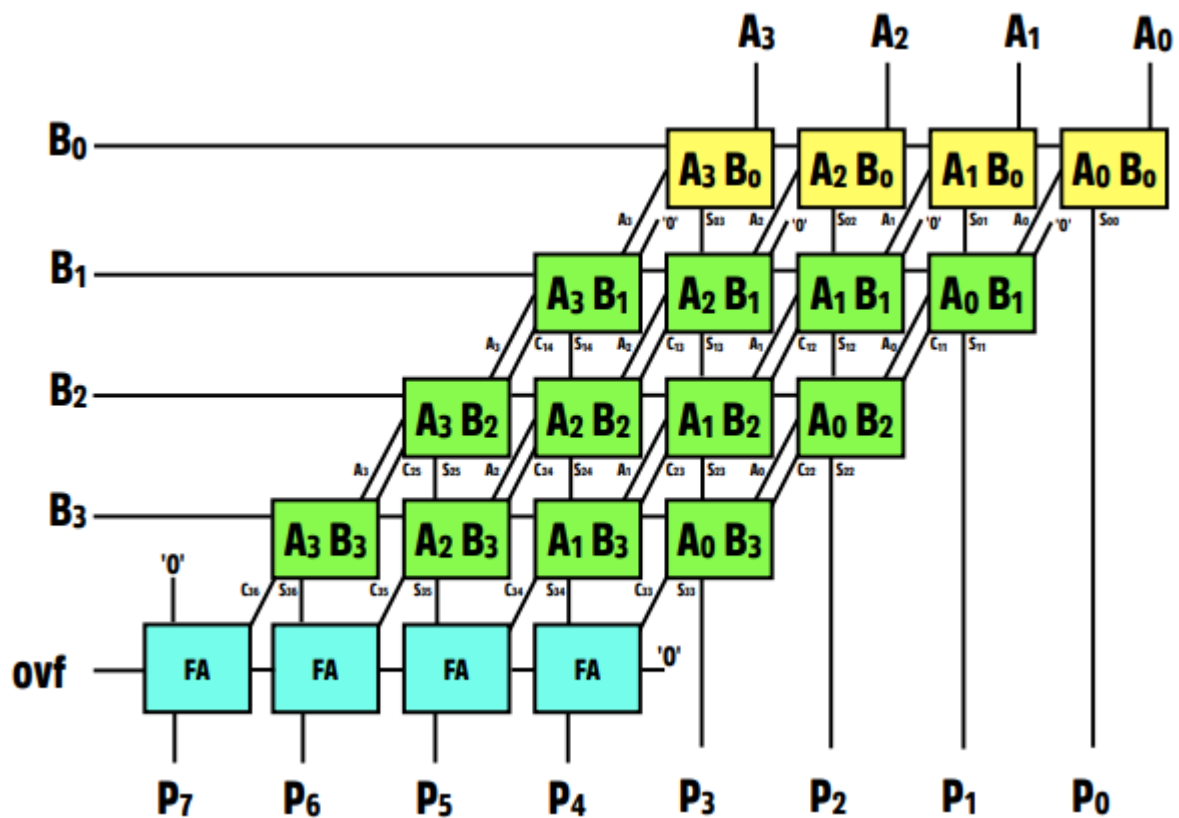
- **Ver.1:** Schema base con moltiplicatori semplici e un sommatore.



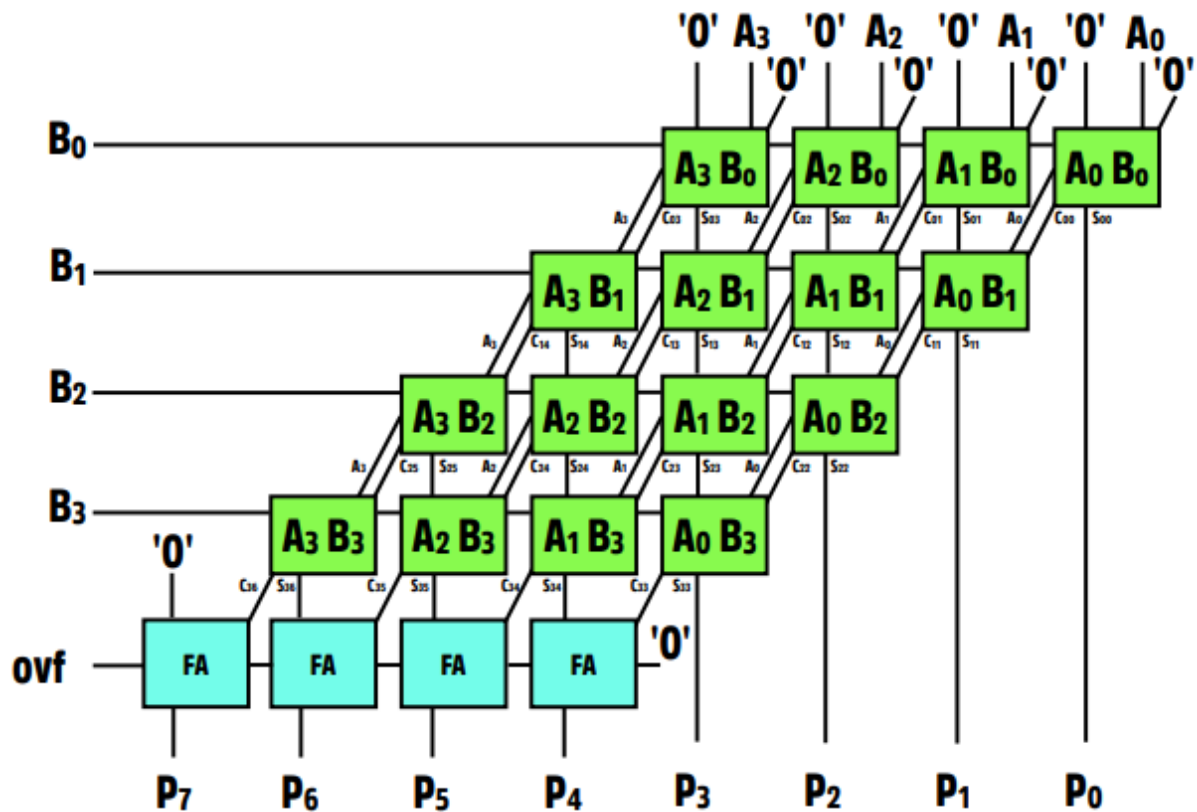
- **Ver.2:** Introduzione di padding per i bit meno significativi, utile per numeri con segno o maggiore precisione.



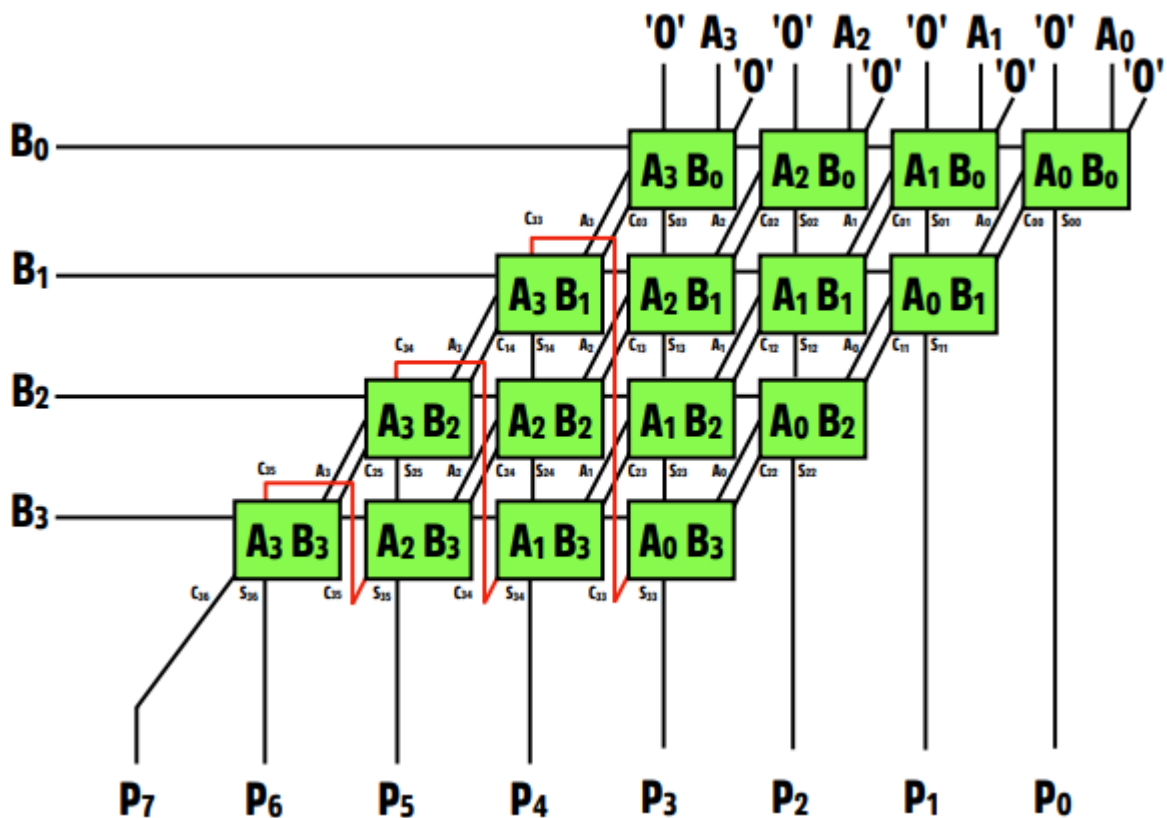
- **Ver.3:** Integrazione di blocchi avanzati (Tipo 4 e Tipo 5) per ridurre i ritardi di propagazione.



- **Ver.4:** Ottimizzazioni attraverso blocchi condivisi o pre-calcolo di operazioni ripetitive.



- **Ver.5:** Miglioramenti significativi in velocità ed efficienza grazie a un design avanzato.



## Fixed - Point Notation

La notazione a virgola fissa è ampiamente utilizzata per rappresentare numeri nei sistemi hardware. Le principali configurazioni includono:

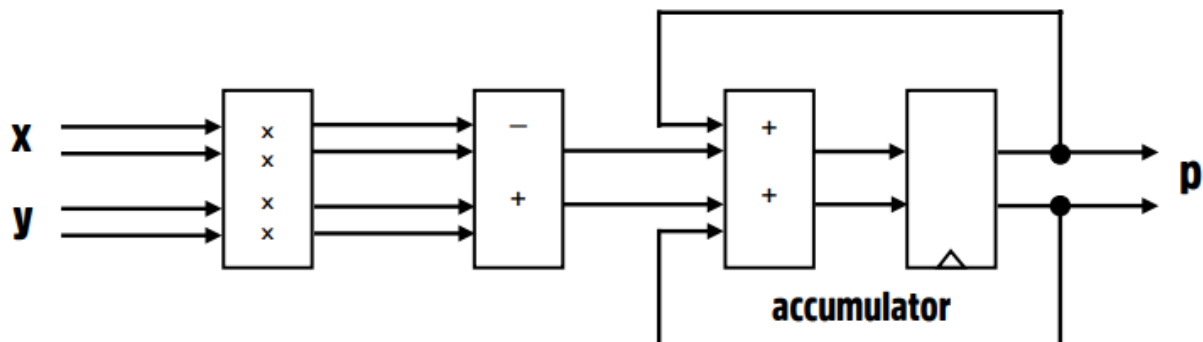
- **Unsigned:** Numeri positivi rappresentati con valori binari.
- **Signed (complemento a due):** Rappresentazione di numeri con segno, che consente di evitare il doppio zero e di gestire intervalli asimmetrici tra numeri positivi e negativi.

Ad esempio, con 4 bit unsigned, il range rappresentabile è , mentre in complemento a due è .

|                   |        |          |          |          |          |          |          |          |          |          |           |           |           |           |           |           |
|-------------------|--------|----------|----------|----------|----------|----------|----------|----------|----------|----------|-----------|-----------|-----------|-----------|-----------|-----------|
| <b>bit index</b>  | 15     | 14       | 13       | 12       | 11       | 10       | 9        | 8        | 7        | 6        | 5         | 4         | 3         | 2         | 1         | 0         |
| <b>bit weight</b> | $-2^0$ | $2^{-1}$ | $2^{-2}$ | $2^{-3}$ | $2^{-4}$ | $2^{-5}$ | $2^{-6}$ | $2^{-7}$ | $2^{-8}$ | $2^{-9}$ | $2^{-10}$ | $2^{-11}$ | $2^{-12}$ | $2^{-13}$ | $2^{-14}$ | $2^{-15}$ |

## The Single Cycle MAC

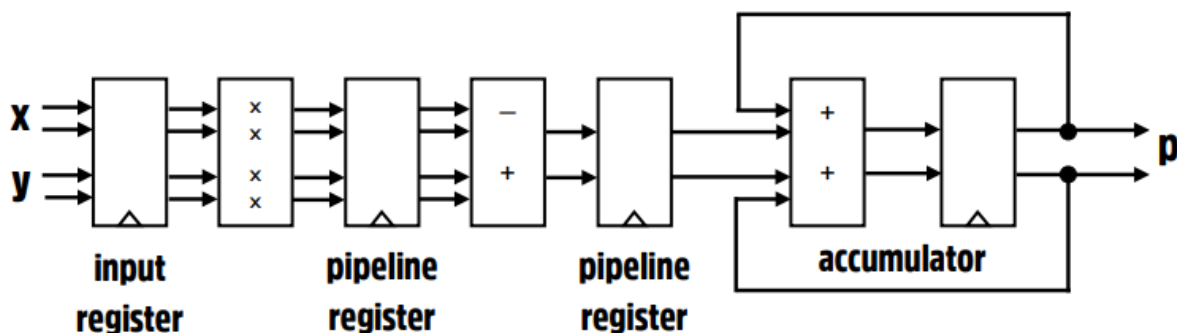
Un MAC a ciclo singolo esegue somme e prodotti accumulati in un solo ciclo di clock. Ogni ciclo consente di memorizzare i risultati parziali, pronti per essere elaborati successivamente. Questo design è particolarmente efficace in applicazioni a bassa latenza, dove l'efficienza di calcolo è prioritaria.



## The Pipelined MAC

Il MAC pipelined suddivide la catena di elaborazione in sotto-catene sincrone, consentendo l'esecuzione parallela di operazioni. Sebbene la latenza aumenti (tempo richiesto per un singolo dato per attraversare il sistema), il throughput cresce significativamente, garantendo un flusso continuo di dati.

Ad esempio, in un MAC pipelined con quattro stadi, mentre il primo stadio elabora un nuovo dato, i successivi processano i dati precedenti. Ciò aumenta il tasso di uscita dei dati, migliorando le prestazioni complessive.

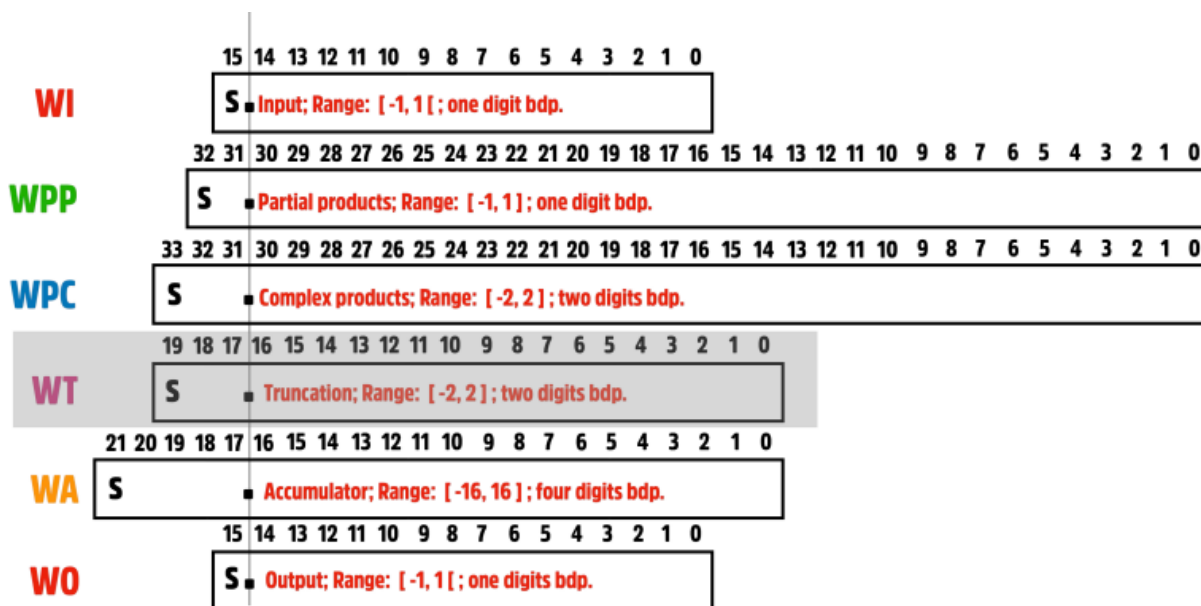
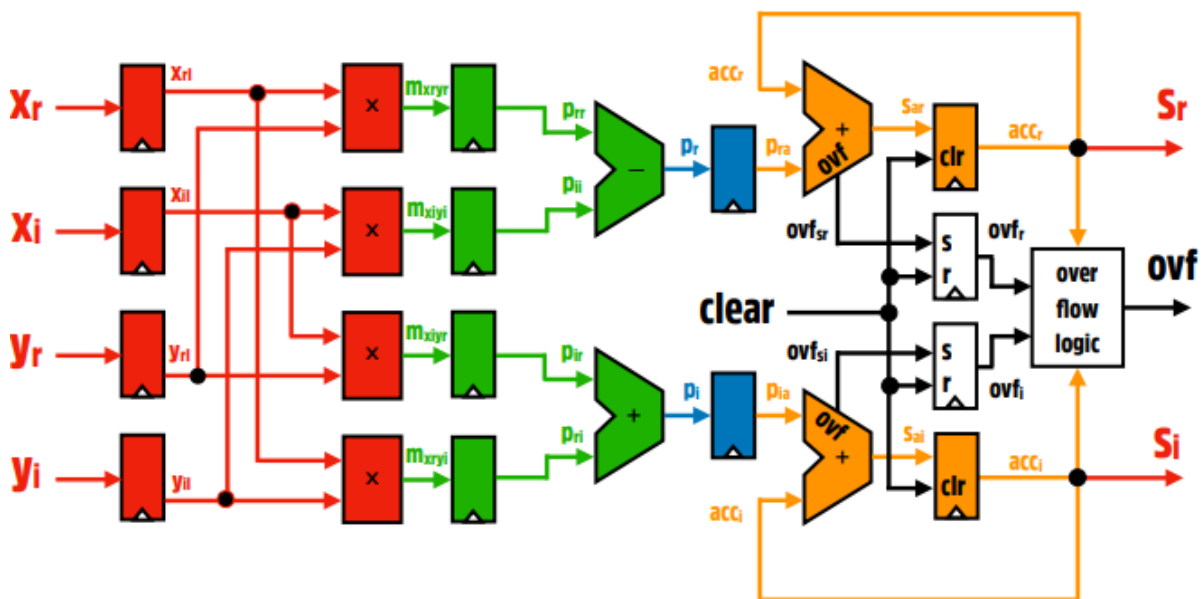


## The Structure - Different format Along the MAC

La struttura di un MAC pipelined include:

- **Registri di memorizzazione intermedia:** Utilizzati per sincronizzare i dati tra gli stadi.
- **Moltiplicatori paralleli:** Per eseguire i calcoli parziali rapidamente.
- **Sommatori:** Per combinare i risultati dei prodotti parziali.
- **Data Path uniformato:** Progettato per evitare cammini critici, garantendo una propagazione uniforme dei segnali.

Un esempio pratico di data path pipelined è l'implementazione di un sistema con 4 registri, 4 moltiplicatori e 2 sommatori. Questo design ottimizza la velocità di elaborazione mantenendo costante la qualità dei risultati.



# MIPS Architecture

## The Micro Processor Quantitative Design

**The Micro Processor Quantitative Design** è un approccio sistematico all'analisi, progettazione e ottimizzazione dei microprocessori basato su principi quantitativi. Questo metodo si concentra sulla misurazione e valutazione delle prestazioni, dell'efficienza e dei compromessi di progettazione nei microprocessori, utilizzando metriche come il numero di cicli di clock per istruzione, la latenza e il throughput.

L'obiettivo è quello di massimizzare l'efficienza del processore attraverso l'analisi dettagliata delle istruzioni, della pipeline e delle unità funzionali. Viene posta particolare attenzione al design del datapath e delle linee di controllo, considerando configurazioni monolitiche, pipeline o multi-ciclo. Questo consente di bilanciare le prestazioni con i costi di implementazione in termini di risorse hardware e consumo energetico.

## Speedup

Il **speedup** rappresenta il miglioramento delle prestazioni di un sistema grazie a una nuova soluzione hardware o architetturale. È definito come il rapporto tra il tempo di esecuzione senza la nuova soluzione ( $t_{senza}$ ) e il tempo di esecuzione con la nuova soluzione ( $t_{con}$ ):

D

Affinché la nuova soluzione sia vantaggiosa, deve risultare  $t_{con} < t_{senza} \Rightarrow S > 1$ . L'obiettivo del design quantitativo è dimostrare che le scelte adottate migliorano effettivamente le prestazioni rispetto a soluzioni precedenti.

## The Amhdal Law

La **legge di Amdahl** descrive il limite massimo di miglioramento di un sistema, anche quando viene ottimizzata solo una parte del processo. L'equazione è:

$$S = 1 / [(1 - f_{enhanced}) + (f_{enhanced} / S_{enhanced})]$$

Dove:

- $f_{enanched}$  è la frazione di codice ottimizzata,
- $Senhancement$  è lo speedup della parte ottimizzata.

Ad esempio, se il 40% del tempo della CPU è speso in calcoli numerici e questa parte è migliorata di 10 volte ( $Senhancement = 10$ ), il miglioramento complessivo ( $S$ ) sarà:

$$S = 1 / [(1 - 0.4) + (0.4/10)] \approx 1.56$$

## Examples

### ESEMPIO 1

Questo esempio mostra come i limiti del miglioramento dipendano dalla frazione non ottimizzata.

Una CPU spende il 20% del tempo eseguendo radici quadrate ( $f_{enanched} = 0,2$ ) e il 50% del tempo in altre istruzioni a virgola mobile ( $f_{enanched} = 0,5$ )

.

- Miglioramento radici quadrate ( $Senanchred = 10$ ) :

$$S = 1 / [(1 - 0.2) + (0.2/10)] \approx 1.22$$

- Miglioramento altre soluzioni ( $Senanchred = 1,6$ ) :

$$S = 1 / [(1 - 0.5) + (0.5/1.6)] \approx 1.23$$

In questo caso, migliorare il blocco più grande è leggermente più vantaggioso.

## The Performance Equation

### Instruction Count (IC) & Clock Per Instruction (CPI)

L'equazione di performance misura il tempo totale di esecuzione della CPU:

$$CPUtime = IC * CPI * T_{cycle}$$

Dove:

- $IC$  è il **numero di istruzioni** necessarie per un programma,

- *CPI* è il **numero medio di cicli di clock per istruzione**,
- *Tcycle* è la durata di un ciclo di clock.

Questa equazione permette di individuare aree di ottimizzazione:

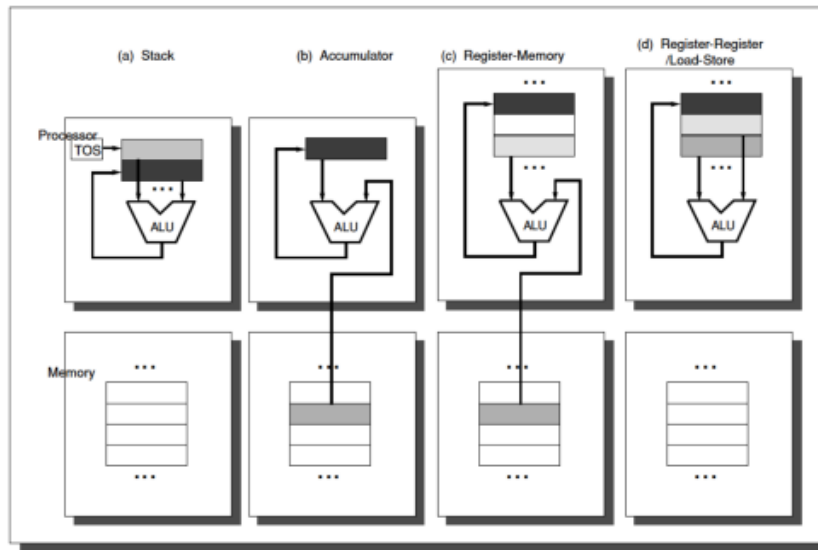
- Ridurre *IC* migliorando il compilatore.
- Ottimizzare *CPI* modificando l'organizzazione interna o l'architettura.
- Diminuire *Tcycle* migliorando la tecnologia hardware.

## Micro Processor Architectural Types

### The Evolution

L'evoluzione delle architetture dei microprocessori ha seguito un percorso scandito dai progressi tecnologici e dalla ricerca di prestazioni sempre maggiori. Negli anni '80, le architetture predominanti erano quelle basate sull'accumulatore. Ciò era dovuto al fatto che le tecnologie integrate erano agli albori e non permettevano soluzioni più complesse. Con il progredire della tecnologia, emersero architetture più sofisticate, note come **CISC (Complex Instruction Set Computer)**, progettate per eseguire operazioni complesse in un minor numero di istruzioni.

Tuttavia, agli inizi degli anni '90 si dimostrò che mantenere l'hardware semplice e veloce, approccio noto come **RISC (Reduced Instruction Set Computer)**, garantiva migliori prestazioni. Fu in questo contesto che iniziarono a diffondersi le cosiddette **architetture register-to-register**. Queste architetture sfruttavano appieno la località dei dati, introducendo l'uso delle memorie cache. Tale innovazione ridusse drasticamente gli accessi alla memoria centrale, incrementando significativamente le prestazioni dei processori.



| Stack  | Accumulator | Register<br>(register-memory) | Register<br>(load-store) |
|--------|-------------|-------------------------------|--------------------------|
| Push A | Load A      | Load R1, A                    | Load R1, A               |
| Push B | Add B       | Add R3, R1, B                 | Load R2, B               |
| Add    | Store C     | Store R3, C                   | Add R3, R1, R2           |
| Pop C  |             |                               | Store R3, C              |

## Architectural Types

Le architetture di microprocessore si dividono in diversi tipi:

- **Stack:** i dati vengono memorizzati in una pila. Operazioni come "push" o "pop" accedono alla cima della pila.
- **Accumulator:** un operando è contenuto in un registro dedicato, mentre l'altro è memorizzato in memoria.
- **Register-Memory:** gli operandi possono essere registri o locazioni di memoria.
- **Register-Register:** entrambi gli operandi risiedono nei registri, riducendo l'accesso alla memoria e aumentando la velocità.

Le architetture **RISC** (come MIPS) favoriscono soluzioni "Register-Register" o "Stack" per la loro semplicità e velocità

## Eight Little Endians

Le architetture possono differire nel modo in cui memorizzano i dati in memoria:

- **Little-endian:** il byte meno significativo (LSB) è memorizzato all'indirizzo più basso.
- **Big-endian:** il byte più significativo (MSB) è memorizzato all'indirizzo più basso.

Queste scelte influenzano la compatibilità e le prestazioni delle architetture.

## Memory Modes and Alignment

L'accesso alla memoria dipende dalla dimensione dei dati (ad esempio, 1 byte o 4 byte) e dall'allineamento della memoria. Un allineamento corretto garantisce un accesso rapido ai dati, evitando sprechi di risorse hardware. In alcuni casi, è possibile eliminare l'allineamento per risparmiare risorse, ma ciò potrebbe influire negativamente sulle prestazioni complessive.

| Width of object:      | Value of 3 low order bits of byte address: |            |         |            |         |            |            |            |           |
|-----------------------|--------------------------------------------|------------|---------|------------|---------|------------|------------|------------|-----------|
|                       | 0                                          | 1          | 2       | 3          | 4       | 5          | 6          | 7          |           |
| 1 Byte (Byte)         | Aligned                                    | Aligned    | Aligned | Aligned    | Aligned | Aligned    | Aligned    | Aligned    |           |
| 2 Bytes (Half word)   | Aligned                                    |            | Aligned |            | Aligned |            | Aligned    |            |           |
| 2 Bytes (Half word)   |                                            | Misaligned |         | Misaligned |         | Misaligned |            | Misalign.  |           |
| 4 Bytes (Word)        | Aligned                                    |            |         |            | Aligned |            |            |            |           |
| 4 Bytes (Word)        |                                            | Misaligned |         |            |         | Misaligned |            |            |           |
| 4 Bytes (Word)        |                                            | Misaligned |         |            |         |            | Misaligned |            |           |
| 4 Bytes (Word)        |                                            | Misaligned |         |            |         |            |            | Misalign.  |           |
| 8 bytes (Double word) | Aligned                                    |            |         |            |         |            |            |            |           |
| 8 bytes (Double word) |                                            | Misaligned |         |            |         |            |            |            |           |
| 8 bytes (Double word) |                                            | Misaligned |         |            |         |            |            |            |           |
| 8 bytes (Double word) |                                            | Misaligned |         |            |         |            |            |            |           |
| 8 bytes (Double word) |                                            |            |         |            |         | Misaligned |            |            |           |
| 8 bytes (Double word) |                                            |            |         |            |         |            | Misaligned |            |           |
| 8 bytes (Double word) |                                            |            |         |            |         |            |            | Misaligned |           |
| 8 bytes (Double word) |                                            |            |         |            |         |            |            |            | Misalign. |

## MIPS Addressing Modes

I modi di indirizzamento definiscono come i dati vengono recuperati o memorizzati. Le principali modalità di indirizzamento nel MIPS includono:

- **Immediato:** Il dato è incluso direttamente nell'istruzione, rendendo l'accesso rapido poiché non è necessario recuperarlo da un registro o dalla memoria.
- **Displacement:** Un registro contiene un indirizzo base, a cui si aggiunge un offset specificato nell'istruzione. Ad esempio, se un registro *R1* contiene un

indirizzo base, un offset di 100 sarà sommato per accedere alla posizione di memoria desiderata.

- **Register-Indirect:** Il registro punta a un indirizzo in memoria. Ad esempio,  $R1$  contiene l'indirizzo della posizione in memoria da cui leggere o scrivere.

$R1R1$

- **Direct:** Un registro punta a un altro registro, che a sua volta punta al valore desiderato. Ad esempio,  $R3 \rightarrow R7 \rightarrow R8$ , dove il valore è memorizzato in  $R8$ .

| Addressing mode    | Example instruction  | Meaning                                                                                                                        | When used                                                                                                                                |
|--------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Register           | Add R4, R3           | $\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Regs}[R3]$                                                                 | When a value is in a register.                                                                                                           |
| Immediate          | Add R4, #3           | $\text{Regs}[R4] \leftarrow \text{Regs}[R4] + 3$                                                                               | For constants.                                                                                                                           |
| Displacement       | Add R4, 100(R1)      | $\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[100 + \text{Regs}[R1]]$                                               | Accessing local variables (+ simulates register indirect, direct addressing modes)                                                       |
| Register indirect  | Add R4, (R1)         | $\text{Regs}[R4] \leftarrow \text{Regs}[R4] + \text{Mem}[\text{Regs}[R1]]$                                                     | Accessing using a pointer or a computed address.                                                                                         |
| Indexed            | Add R3, (R1 + R2)    | $\text{Regs}[R3] \leftarrow \text{Regs}[R3] + \text{Mem}[\text{Regs}[R1] + \text{Regs}[R2]]$                                   | Sometimes useful in array addressing: R1 = base of array; R2 = index amount.                                                             |
| Direct or absolute | Add R1, (1001)       | $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[1001]$                                                                | Sometimes useful for accessing static data; address constant may need to be large.                                                       |
| Memory indirect    | Add R1, @(R3)        | $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$                                         | If R3 is the address of a pointer $p$ , then mode yields $*p$ .                                                                          |
| Autoincrement      | Add R1, (R2) +       | $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$<br>$\text{Regs}[R2] \leftarrow \text{Regs}[R2] + d$ | Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, $d$ . |
| Autodecrement      | Add R1, -(R2)        | $\text{Regs}[R2] \leftarrow \text{Regs}[R2] - d$<br>$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]]$ | Same use as autoincrement. Autodecrement/increment can also act as push/pop to implement a stack.                                        |
| Scaled             | Add R1, 100(R2) [R3] | $\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] + \text{Regs}[R3] * d]$                         | Used to index arrays. May be applied to any indexed addressing mode in some computers.                                                   |

## The Sw/Hw Interface

L'interfaccia tra software e hardware, definita come **Instruction Set Architecture (ISA)**, specifica come il software interagisce con i componenti hardware. Un'architettura come MIPS adotta una struttura fissa delle istruzioni per semplificare la progettazione, evitando la complessità di istruzioni di lunghezza variabile.

## MIPS Instruction

Il set di istruzioni MIPS comprende tre principali tipi di istruzioni:

- **Tipo R (Register):** Le operazioni coinvolgono tre registri. La struttura include:
  - **op** : Codice operativo.
  - **rs** , **rt** : Registri sorgenti.
  - **rd** : Registro destinazione.
  - **shamt** : Valore di shift per operazioni di spostamento.
  - **funct** : Specifica l'operazione aritmetica da eseguire (es. somma, sottrazione).
- **Tipo I (Immediate):** Include un valore costante o un indirizzo immediato. Risparmia tempo poiché evita di caricare valori da un registro.
- **Tipo J (Jump):** Utilizzato per salti a indirizzi specifici. Comprende:
  - **Jump**: Salto diretto.
  - **Branch**: Salto condizionato basato su una condizione tra registri.

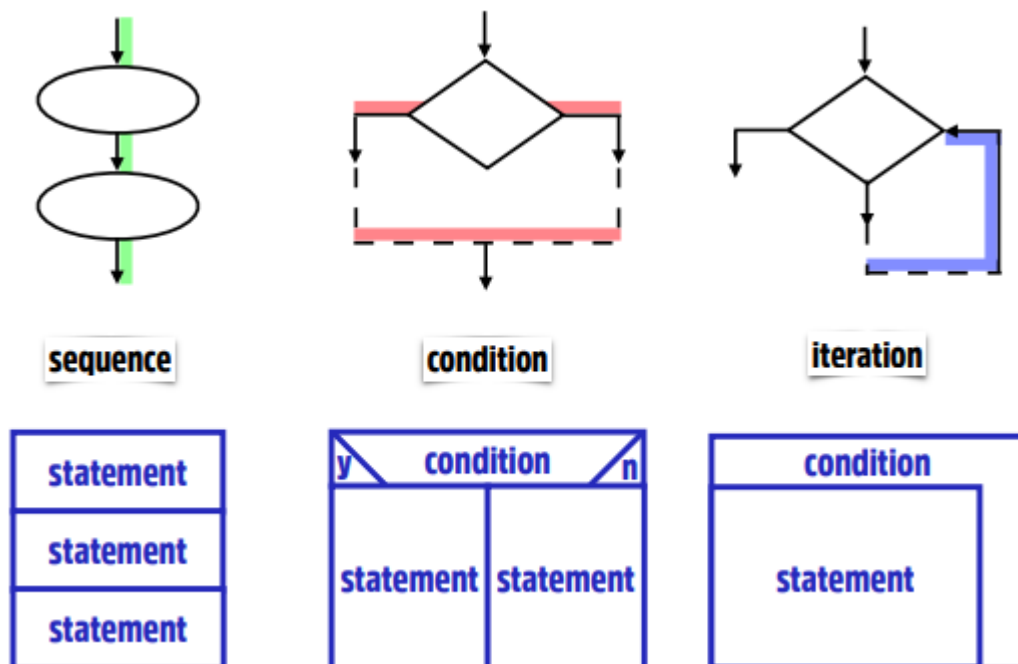
| Category           | Instruction             | Example            | Meaning                                           | Comments                            |
|--------------------|-------------------------|--------------------|---------------------------------------------------|-------------------------------------|
| Arithmetic         | add                     | add \$s1,\$s2,\$s3 | $\$s1 = \$s2 + \$s3$                              | Three register operands             |
|                    | subtract                | sub \$s1,\$s2,\$s3 | $\$s1 = \$s2 - \$s3$                              | Three register operands             |
|                    | add immediate           | addi \$s1,\$s2,100 | $\$s1 = \$s2 + 100$                               | Used to add constants               |
| Data transfer      | load word               | lw \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Word from memory to register        |
|                    | store word              | sw \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Word from register to memory        |
|                    | load half               | lh \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Halfword memory to register         |
|                    | store half              | sh \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Halfword register to memory         |
|                    | load byte               | lb \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Byte from memory to register        |
|                    | store byte              | sb \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Byte from register to memory        |
| Logical            | load upper immed.       | lui \$s1,100       | $\$s1 = 100 * 2^{16}$                             | Loads constant in upper 16 bits     |
|                    | and                     | and \$s1,\$s2,\$s3 | $\$s1 = \$s2 \& \$s3$                             | Three reg. operands; bit-by-bit AND |
|                    | or                      | or \$s1,\$s2,\$s3  | $\$s1 = \$s2   \$s3$                              | Three reg. operands; bit-by-bit OR  |
|                    | nor                     | nor \$s1,\$s2,\$s3 | $\$s1 = \sim (\$s2   \$s3)$                       | Three reg. operands; bit-by-bit NOR |
|                    | and immediate           | andi \$s1,\$s2,100 | $\$s1 = \$s2 \& 100$                              | Bit-by-bit AND reg with constant    |
|                    | or immediate            | ori \$s1,\$s2,100  | $\$s1 = \$s2   100$                               | Bit-by-bit OR reg with constant     |
| Conditional branch | shift left logical      | sll \$s1,\$s2,10   | $\$s1 = \$s2 \ll 10$                              | Shift left by constant              |
|                    | shift right logical     | srl \$s1,\$s2,10   | $\$s1 = \$s2 \gg 10$                              | Shift right by constant             |
|                    | branch on equal         | beq \$s1,\$s2,25   | if ( $\$s1 == \$s2$ ) go to PC + 4 + 100          | Equal test; PC-relative branch      |
|                    | branch on not equal     | bne \$s1,\$s2,25   | if ( $\$s1 \neq \$s2$ ) go to PC + 4 + 100        | Not equal test; PC-relative         |
| Unconditional jump | set on less than        | slt \$s1,\$s2,\$s3 | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$ | Compare less than; for beq, bne     |
|                    | set less than immediate | slti \$s1,\$s2,100 | if ( $\$s2 < 100$ ) $\$s1 = 1$ ; else $\$s1 = 0$  | Compare less than constant          |
| Unconditional jump | jump                    | j 2500             | go to 10000                                       | Jump to target address              |
|                    | jump register           | jr \$ra            | go to \$ra                                        | For switch, procedure return        |
|                    | jump and link           | jal 2500           | $\$ra = PC + 4$ ; go to 10000                     | For procedure call                  |

## Bohm- Jacopoi Theorem

Secondo il teorema di Böhm-Jacopini, un linguaggio di programmazione è Turing completo se include tre costrutti fondamentali:

1. **Sequenza:** Esecuzione lineare delle istruzioni.
2. **Salti condizionati:** Permettono l'esecuzione condizionale (es. `if-then`).
3. **Iterazioni:** Consentono cicli ripetuti (es. `while`).

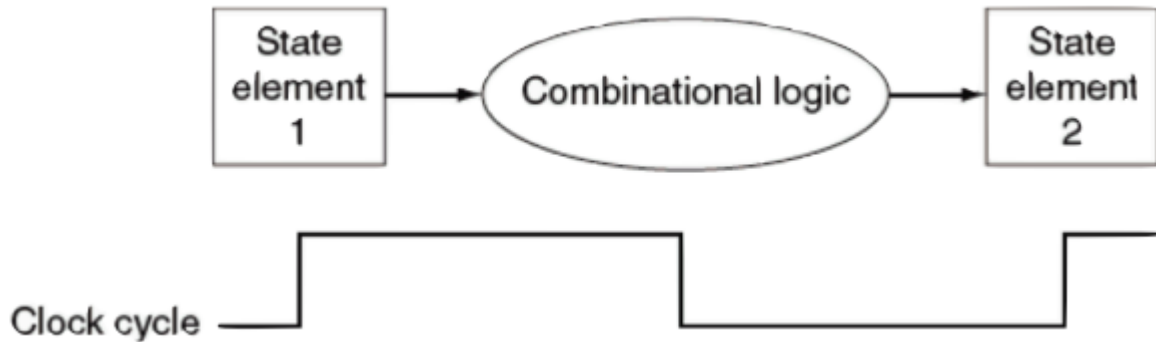
Questi costrutti sono fondamentali per garantire flessibilità e potenza di calcolo.



## The MIPS Single-Cycle Architecture

### Definition and Execution Model

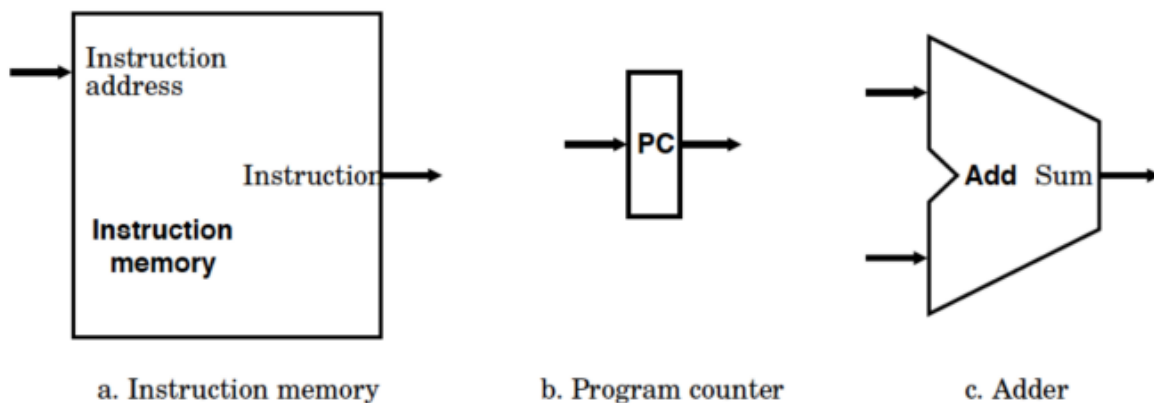
L'architettura a ciclo singolo MIPS esegue ogni istruzione in un singolo ciclo di clock. La durata del ciclo è determinata dal percorso critico, ovvero l'istruzione con la latenza più alta. Questo modello garantisce semplicità ma penalizza le istruzioni con latenza inferiore.

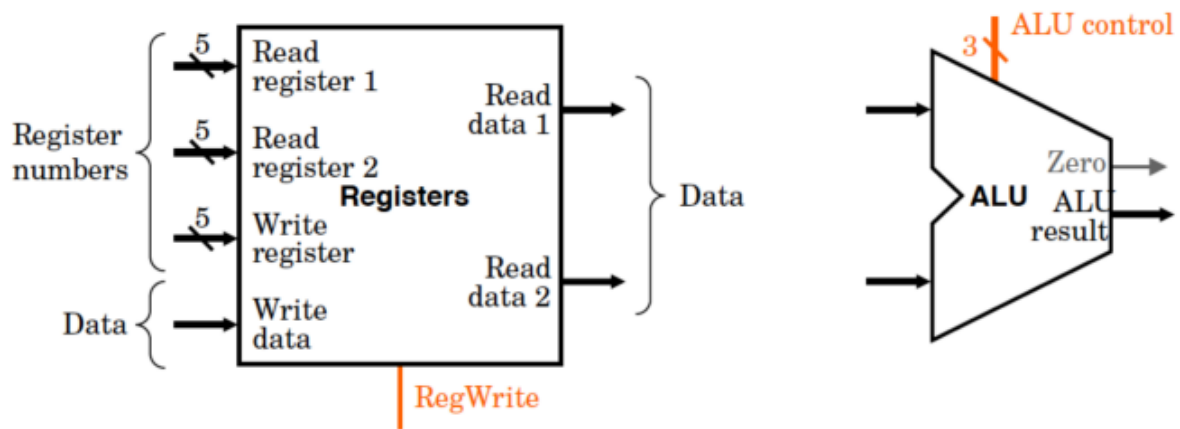


## The Components

L'architettura include quattro sezioni principali:

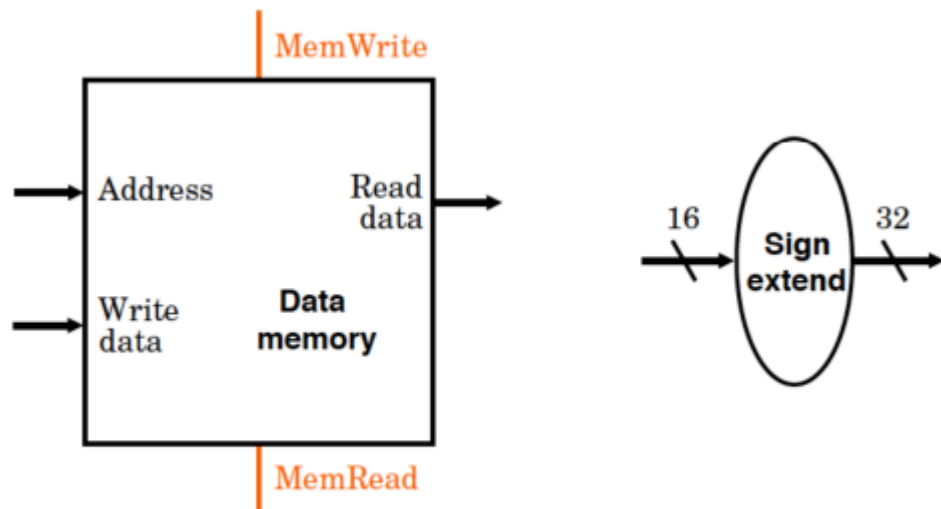
1. **Fetch Section:** Recupera l'istruzione dalla memoria utilizzando il Program Counter (PC) e la memoria istruzioni.
2. **Arithmetic Section:** Include registri e l'ALU (Arithmetic Logic Unit) per operazioni aritmetiche e logiche.
3. **Data Memory Access Section:** Gestisce la lettura e la scrittura dei dati in memoria.
4. **Conditional Branch Section:** Implementa i salti condizionati.





a. Registers

b. ALU



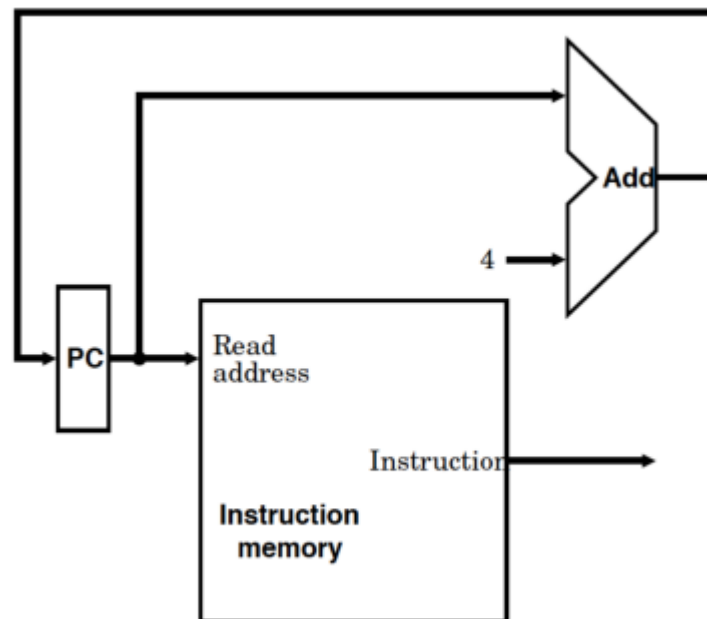
a. Data memory unit

b. Sign-extension unit

## The Fetch Section

Comprende:

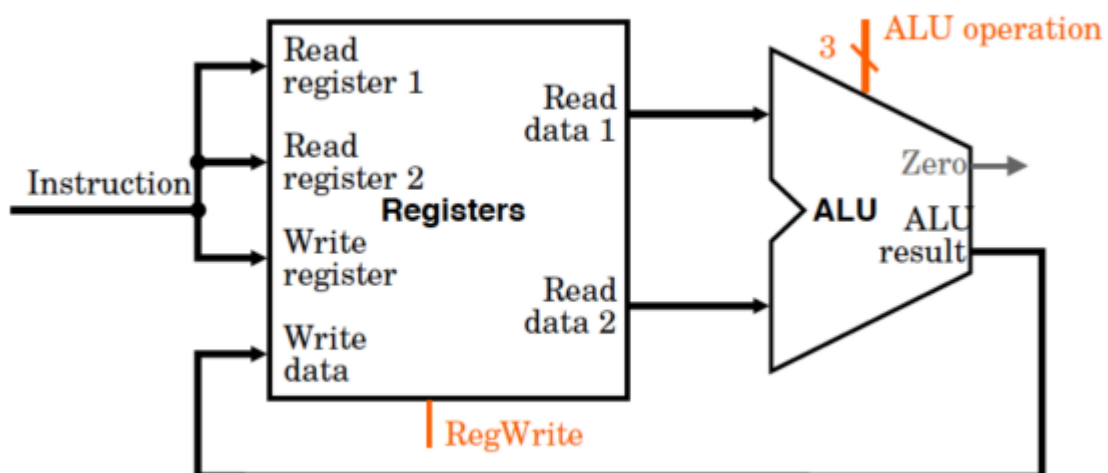
- **Program Counter (PC):** Registra l'indirizzo dell'istruzione corrente.
- **Memory Access:** Recupera le istruzioni.
- **Adder:** Incrementa il PC di 4 byte per passare alla prossima istruzione.



## The Arithmetic Section

Gestisce operazioni sui registri e include:

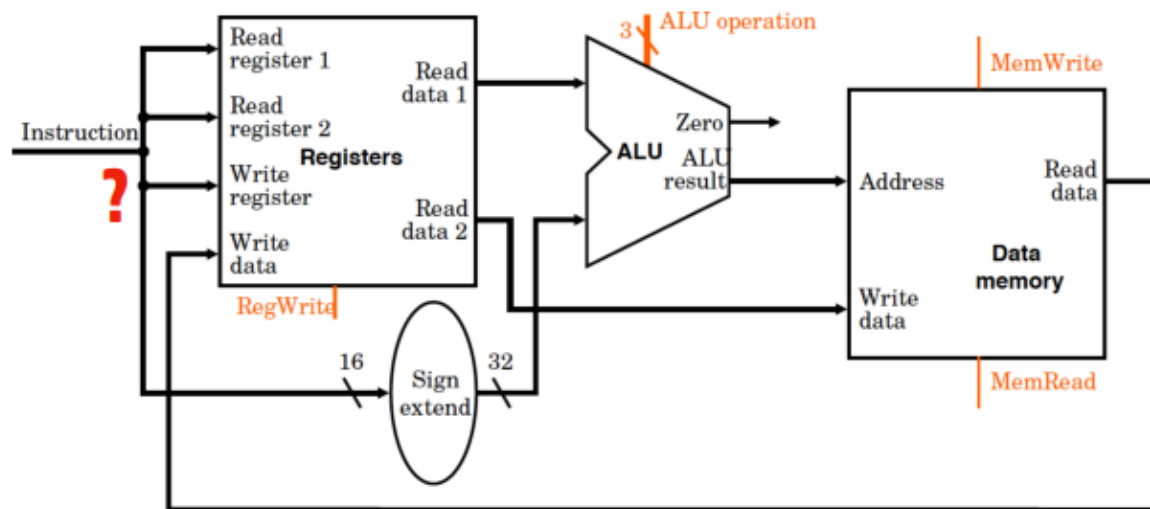
- **Read Register 1 & 2:** Decodifica dei registri sorgenti.
- **Write Register:** Specifica il registro destinazione.
- **ALU:** Esegue le operazioni.



## The Data Memory Acces Section

Comprende:

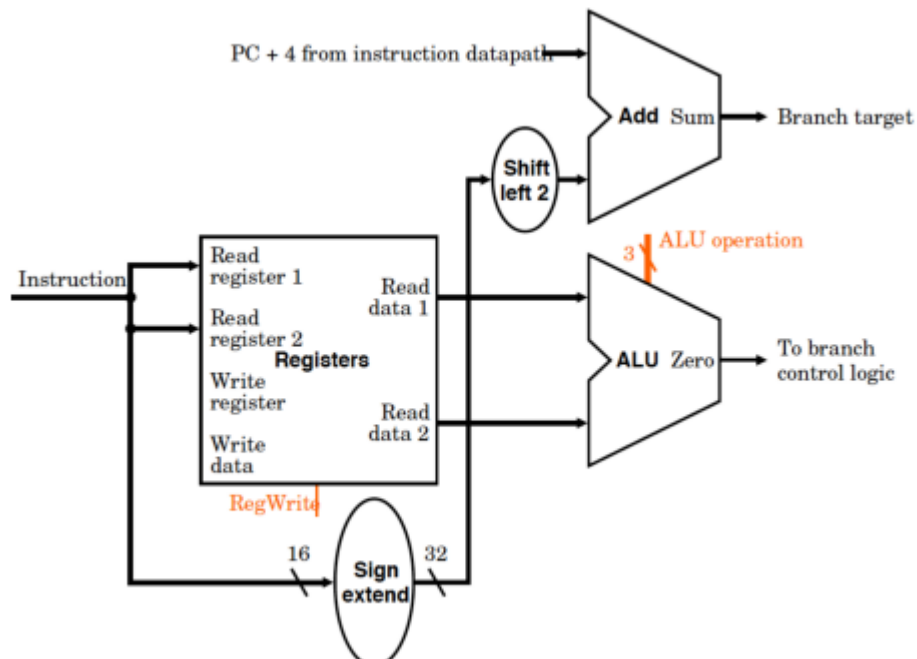
- **Memory:** Accesso ai dati.
- **Sign Extender:** Espande gli offset da 16 a 32 bit per l'indirizzamento.



## The Conditional Branch Section

Consente i salti condizionati con componenti come:

- **ALU:** Confronta i valori.
- **Shifter:** Allinea gli indirizzi (es. 4 byte per istruzione).



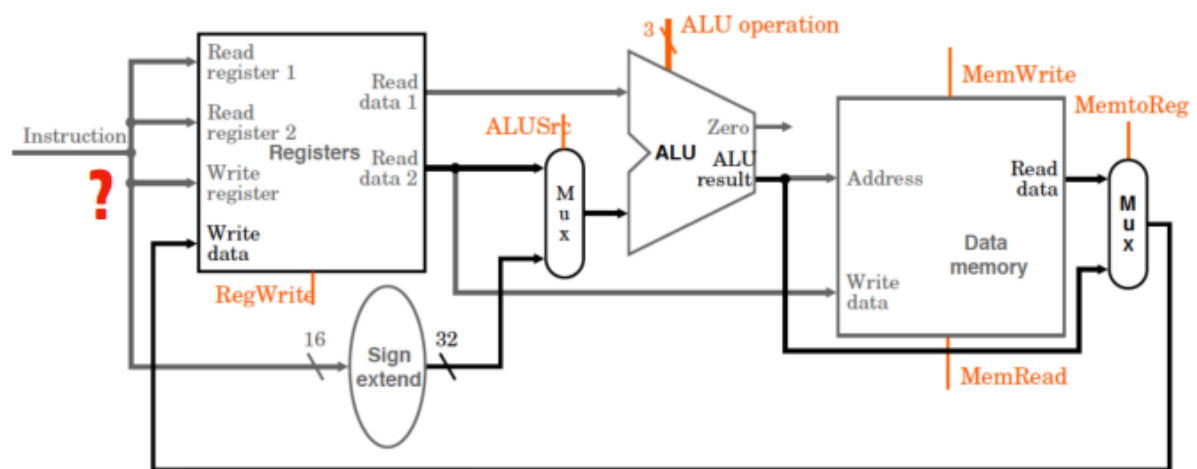
## The Four Sections: How to Integrate Them?

L'integrazione delle sezioni avviene utilizzando **multiplexer (MUX)** per combinare le varie operazioni:

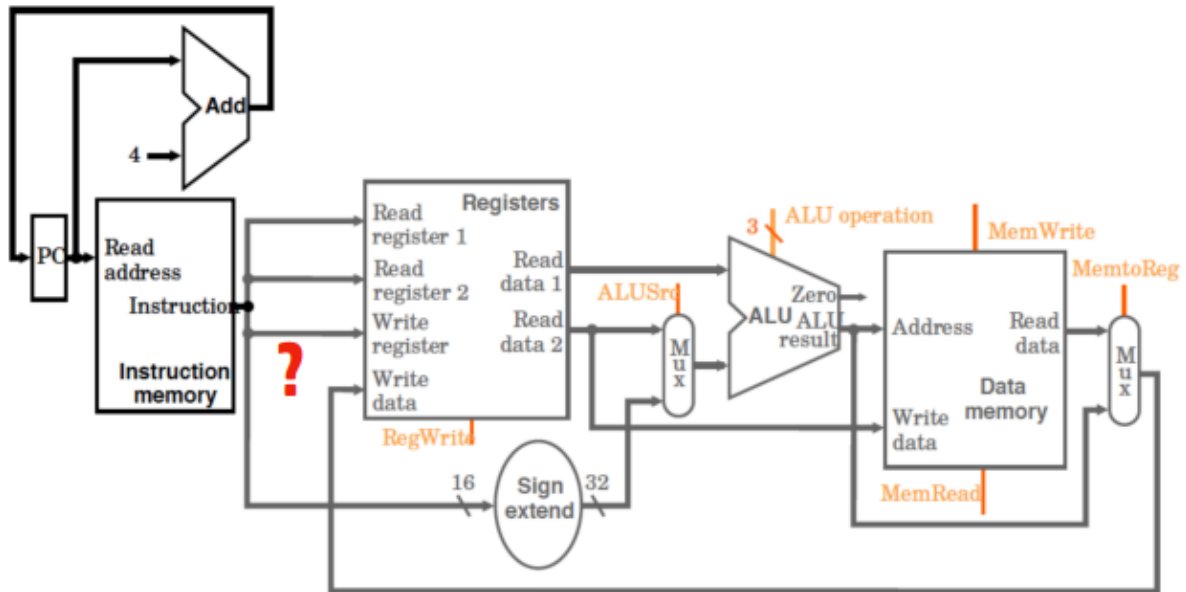
- **Fetch + Arithmetic + Memory Access:** Collegando PC, ALU e memoria.
- **Branch Section:** Aggiunta considerando registri condivisi e il segnale di salto.

L'integrazione finale include il controllo globale tramite una **Finite State Machine**, che sincronizza tutte le sezioni per garantire un'esecuzione efficiente.

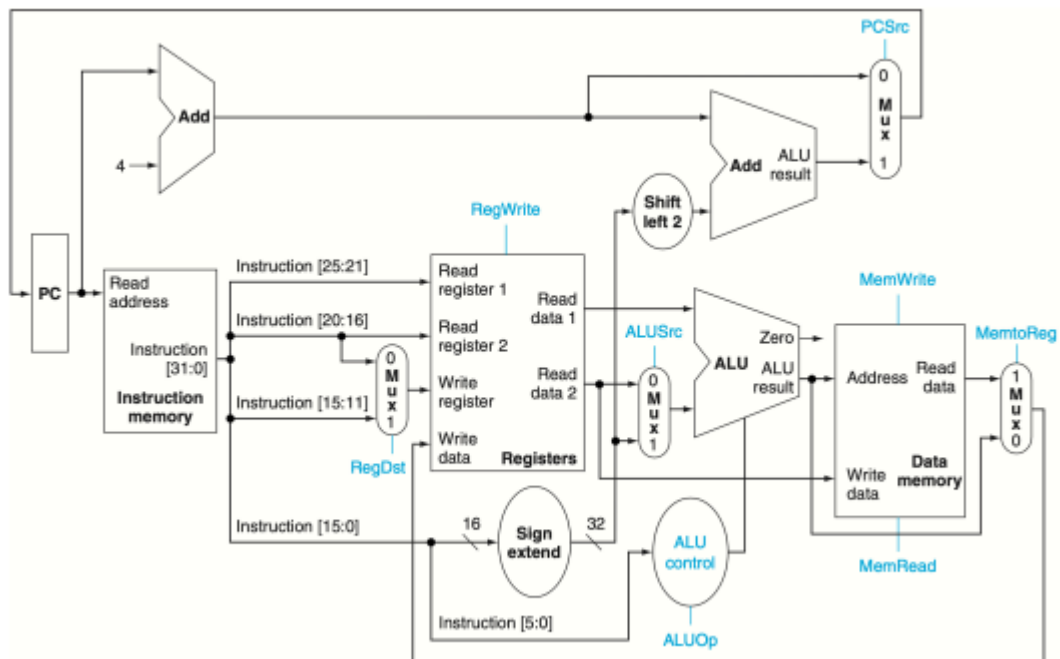
### FIRST INTEGRATION - ARITHMETIC & DMA



### THIRD INTEGRATION - FETCH



#### FOURTH INTEGRATION - BRANCH SECTION



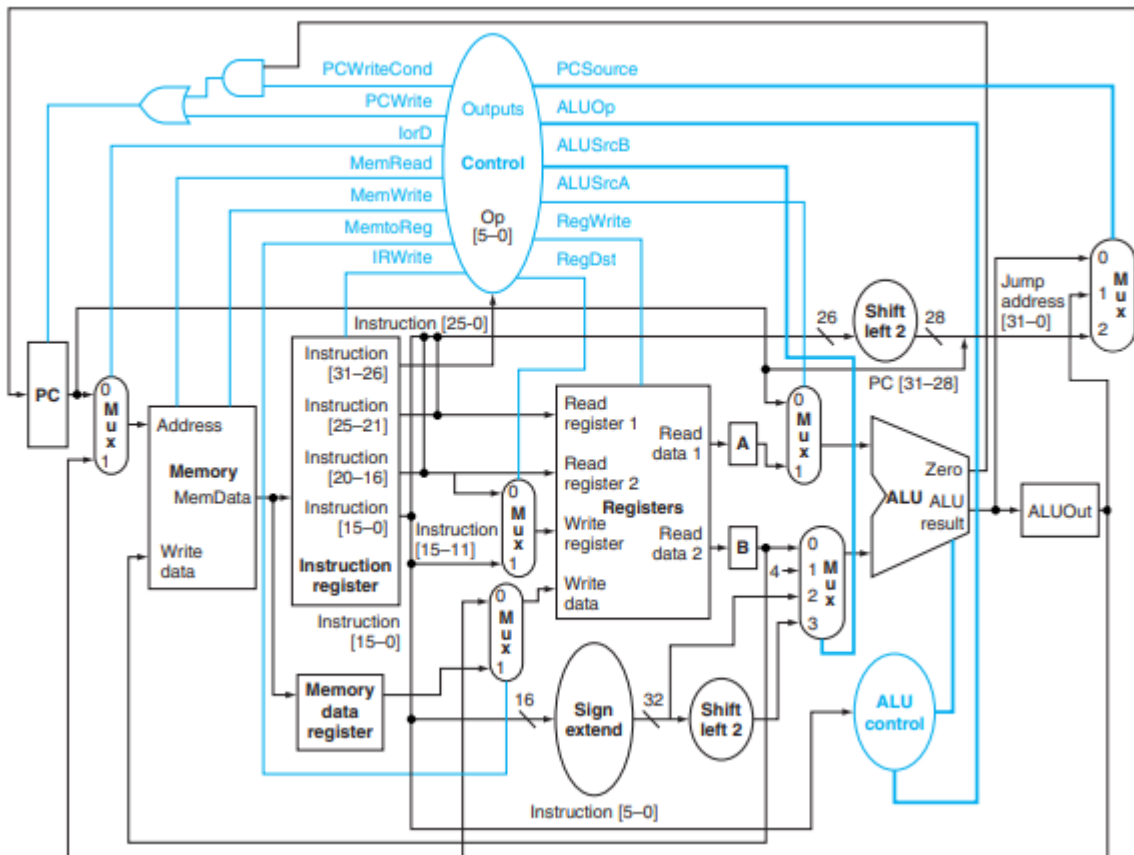
#### FIFTH SECTION - ADD CONTROL & JUMP



- **Multiplexer:** utilizzati per selezionare la sorgente del nuovo valore del PC.
- **Segnali di controllo:** *PCSource*, *PCWrite* e *PCWriteCond*. Quest'ultimo decide se eseguire un branch condizionale.
- Supporto per salti (*jump*).

## Instruction Fetch

- **Fetch dell'Istruzione:**
  - $IR \text{ (Instruction Register)} \leq \text{Memoria}[PC]$
  - $PC \leq PC + 4$
- **Decodifica e Fetch dei Registri:**
  - $A \leq \text{Reg}[IR[25:21]]$
  - $B \leq \text{Reg}[IR[20:15]]$
  - $ALUOut \leq PC + (\text{sign\_extend}(IR[15:0]) \ll 2)$
- **Esecuzione, Calcolo Indirizzo, Branch o Salto:**
  - **Esecuzione (R-type):**  $ALUOut \leq A \text{ op } B$
  - **Accesso Memoria:**  $ALUOut \leq A + \text{sign\_extend}(IR[15:0])$
  - **Branch:** Se  $(A == B)$ , allora  $PC \leq ALUOut$
  - **Jump:**  $PC \leq PC[31:28] \& (IR[25:0] \& "00")$
- **Accesso Memoria:**
  - $\text{Memoria}[ALUOut] \leq B$  (scrittura) oppure
  - $MDR \leq \text{Memoria}[ALUOut]$  (lettura)
- **Scrittura nel Registro:**
  - $\text{Reg}[IR[20:16]] \leq MDR$



## Configurazione dei Segnali di Controllo

- *MemtoReg*: Seleziona i dati dalla memoria per la scrittura nel registro.
- *RegWrite*: Abilita la scrittura nel file di registri.
- *RegDst*: Determina se il registro di destinazione è specificato da *rt* (bits 20:16).

## Riepilogo

| Nome Fase            | Istruzioni R-type   | Istruzioni di memoria          | Branch                    | Jump                       |
|----------------------|---------------------|--------------------------------|---------------------------|----------------------------|
| Fetch Istruzione     | IR <= Memoria[PC]   | IR <= Memoria[PC]              | IR <= Memoria[PC]         | IR <= Memoria[PC]          |
| Decodifica/Fetch Reg | A <= Reg[IR[25:21]] | A <= Reg[IR[25:21]]            | A <= Reg[IR[25:21]]       | A <= Reg[IR[25:21]]        |
|                      | B <= Reg[IR[20:16]] | B <= Reg[IR[20:16]]            | B <= Reg[IR[20:16]]       | B <= Reg[IR[20:16]]        |
| Esecuzione           | ALUOut <= A op B    | ALUOut <= A + sign_extend(...) | PC <= ALUOut (se condiz.) | PC <= {PC[31:28], IR[...]} |

# Cache Principles

## Introduzione

La memoria cache è un componente essenziale dell'architettura dei sistemi di calcolo, progettata per migliorare le prestazioni della CPU riducendo i tempi di accesso alla memoria. L'idea di base è semplice: tenere a portata di mano i dati usati di frequente, sfruttando le metriche di **località spaziale** e **località temporale**.

## The Rationale

La cache si basa sui principi di località per garantire che le operazioni più comuni siano eseguite con accesso rapido. In particolare:

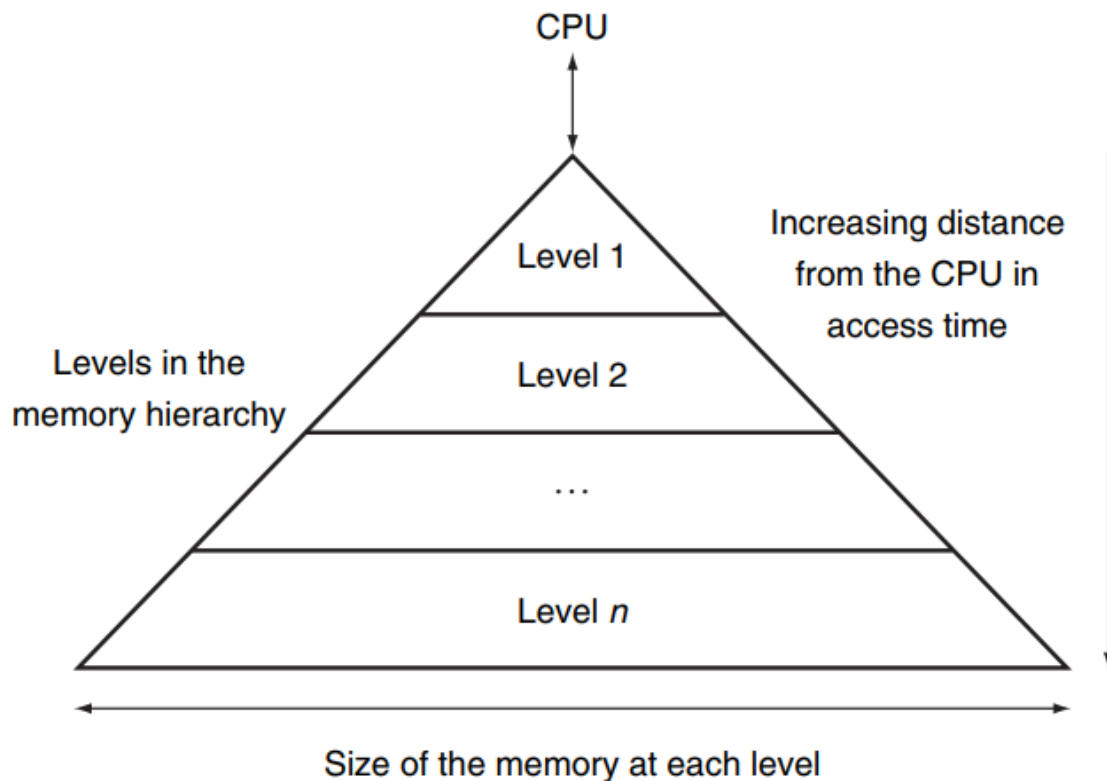
- **Località temporale:** i dati recentemente utilizzati hanno un'alta probabilità di essere riutilizzati.
- **Località spaziale:** i dati vicini a quelli recentemente utilizzati saranno probabilmente richiesti in breve tempo.

## The Memory Hierarchy

Il concetto di gerarchia di memoria sfrutta livelli successivi di memorie con dimensioni e velocità diverse:

- **Livello 1 (L1):** molto vicino alla CPU, piccola e veloce.
- **Livello 2 (L2):** intermedio per dimensione e velocità.
- **Livello n (ad esempio RAM e memoria secondaria):** più grandi ma più lente.

Man mano che aumenta la distanza dalla CPU, aumentano i tempi di accesso e la dimensione delle memorie.



La gerarchia della memoria è strutturata per ridurre i tempi medi di accesso alla memoria grazie ai livelli superiori, che sono piccoli ma veloci.

## The Cache Figure of Merit

Le metriche principali per valutare una cache sono:

- **Dimensione:** quantità di dati memorizzabili.
- **Hit Rate:** frequenza con cui i dati richiesti sono trovati nella cache.
- **Miss Rate:** complementare all'Hit Rate ( $1 - \text{Hit Rate}$ ).
- **Hit Time:** tempo necessario per accedere ai dati in cache.
- **Miss Penalty:** tempo impiegato per recuperare i dati mancanti dalla memoria principale.

## Cache Architecture

### Single Block, Direct Mapped, Cache Architecture

In questa architettura, ogni linea della cache può memorizzare un solo blocco di dati.

**Suddivisione dell'indirizzo (32 bit):**

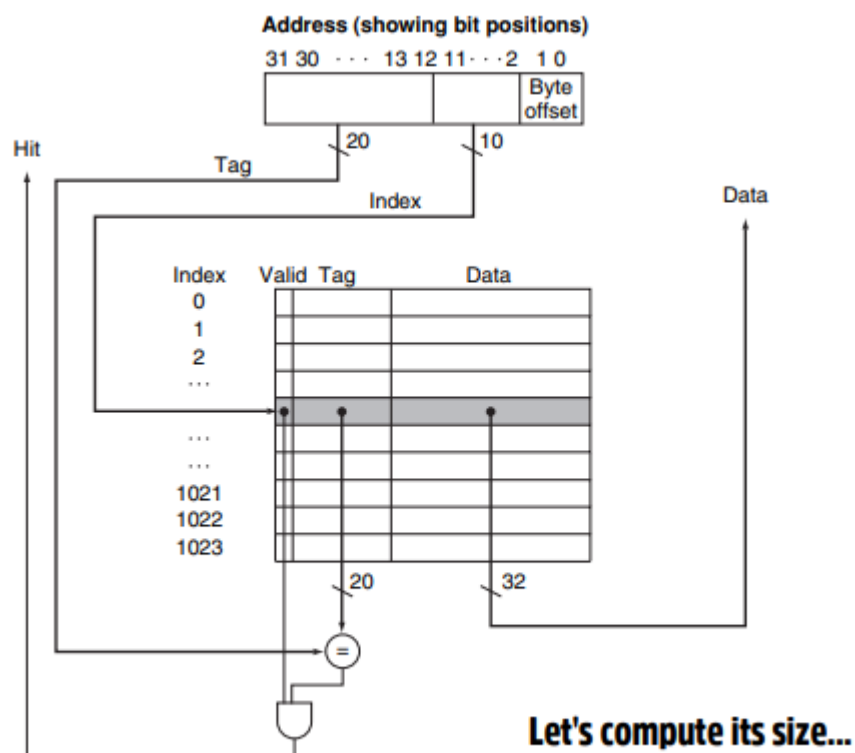
- **Tag (31-12):** Utilizzato per identificare il blocco di memoria.
- **Index (11-2):** Indica quale linea della cache contiene il dato richiesto.
- **Byte Offset (1-0):** Specifica il byte specifico all'interno del blocco.

### Funzionamento:

1. L'**Index** individua direttamente la linea della cache.
2. Il **Tag** dell'indirizzo viene confrontato con il **Tag** memorizzato nella linea selezionata.
3. Se il confronto è positivo e la linea è valida, si ha un **hit**; altrimenti, un **miss**.
4. Il **Byte Offset** seleziona il byte specifico all'interno del blocco.

### Logica di selezione e verifica:

- L'**Index** seleziona la linea della cache.
- Un comparatore verifica se il **Tag** dell'indirizzo corrisponde al **Tag** memorizzato.
- Il byte richiesto viene selezionato tramite il **Byte Offset** utilizzando un **multiplexer**.



**Let's compute its size...**  
**And what's the time penalty?**

## Multi-block, Cache Architecture

In questa architettura, ogni linea della cache può contenere più blocchi di dati.

### Suddivisione dell'indirizzo (32 bit):

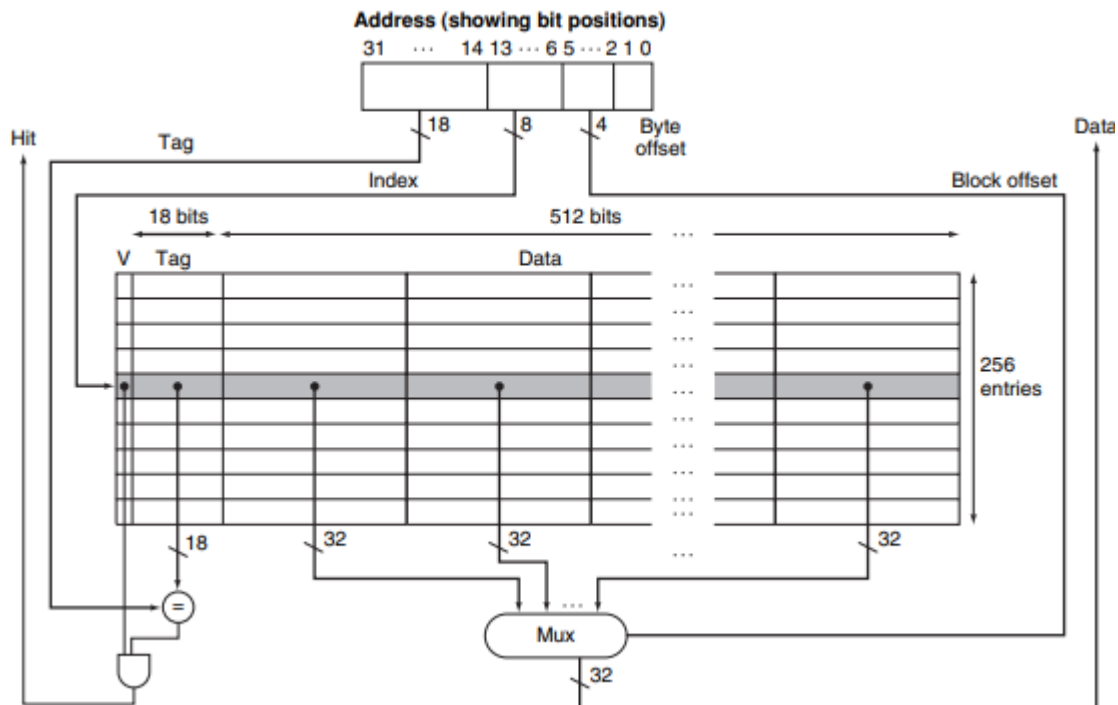
- **Tag (31-14):** Utilizzato per identificare il blocco di memoria.
- **Index (13-6):** Indica quale linea della cache contiene il gruppo di blocchi associato.
- **Block Offset (5-2):** Specifica quale blocco selezionare all'interno della linea.
- **Byte Offset (1-0):** Individua il byte specifico all'interno del blocco.

### Funzionamento:

1. L'**Index** seleziona la linea della cache.
2. Il **Tag** viene confrontato con il **Tag** memorizzato nella linea selezionata.
3. Il **Block Offset** individua il blocco specifico all'interno della linea.
4. Il **Byte Offset** seleziona il byte richiesto all'interno del blocco selezionato.

### Logica di selezione e verifica:

- La verifica del **Tag** avviene tramite un confronto tra il **Tag** dell'indirizzo e quello memorizzato nella linea selezionata.
- Un **multiplexer** controllato dal **Block Offset** seleziona il blocco corretto.
- Un ulteriore **multiplexer** utilizza il **Byte Offset** per selezionare il byte richiesto.



## Set Associative Cache Architecture

Questa architettura combina le caratteristiche della cache diretta e della cache completamente associativa. Ogni linea è organizzata in più set (gruppi di linee).

### Suddivisione dell'indirizzo (32 bit):

- **Tag (31-9):** Utilizzato per identificare il blocco di memoria.
- **Index (8-2):** Indica quale set contiene il blocco richiesto.
- **Byte Offset (1-0):** Specifica il byte specifico all'interno del blocco.

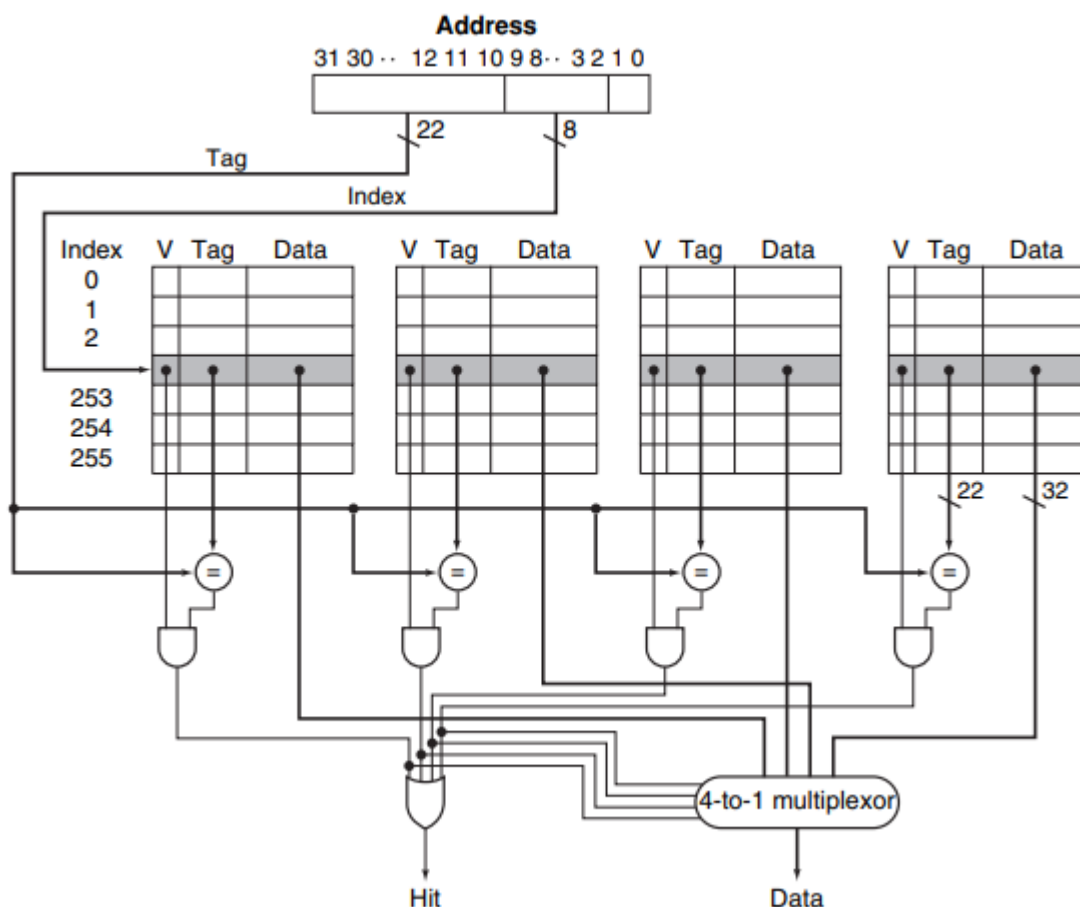
### Funzionamento:

1. L'**Index** seleziona il set della cache in cui cercare il dato.
2. Il **Tag** dell'indirizzo viene confrontato con i **Tag** memorizzati all'interno di tutte le linee del set selezionato.
3. Se uno dei confronti è positivo e la linea è valida, si ha un **hit**; altrimenti, un **miss**.
4. Il **Byte Offset** seleziona il byte richiesto all'interno del blocco.

### Logica di selezione e verifica:

- Il **Tag** viene confrontato in parallelo con i **Tag** di tutte le linee del set selezionato.

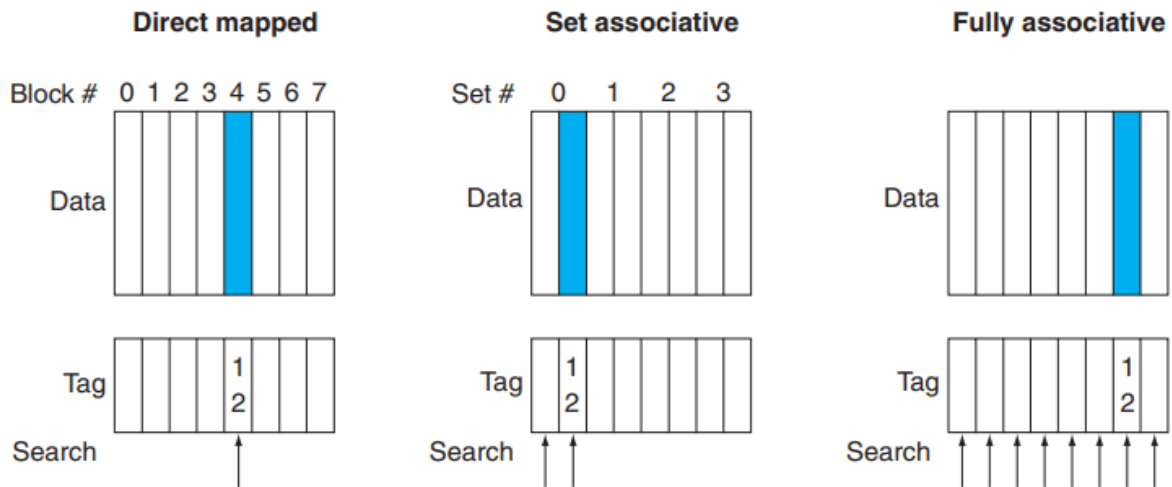
- Un **multiplexer** controllato dal risultato del confronto seleziona il blocco corretto.
- Il byte richiesto viene selezionato tramite il **Byte Offset** utilizzando un **multiplexer** aggiuntivo.



## Cache Organization

L'organizzazione della cache influenza il tasso di miss e la velocità complessiva:

1. **Direct Mapped:** ogni blocco di memoria è mappato a una sola posizione nella cache.
2. **Set Associative:** ogni blocco può essere memorizzato in uno dei vari set (ad esempio, 2-way, 4-way).
3. **Fully Associative:** ogni blocco può essere memorizzato in qualsiasi posizione nella cache.



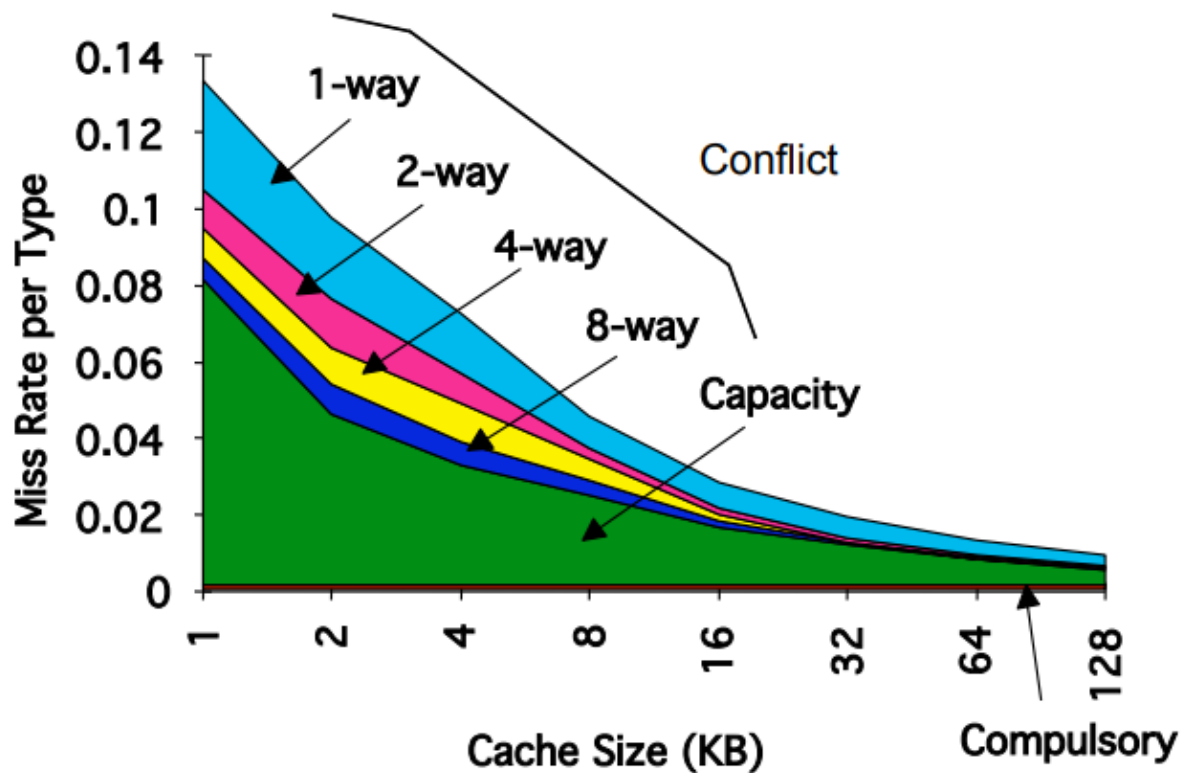
L'aumento dell'associatività riduce il tasso di miss, ma aumenta il tempo di hit e la complessità hardware.

| Associativity | Data miss rate |
|---------------|----------------|
| 1             | 10.3%          |
| 2             | 8.6%           |
| 4             | 8.3%           |
| 8             | 8.1%           |

## Reducing Misses

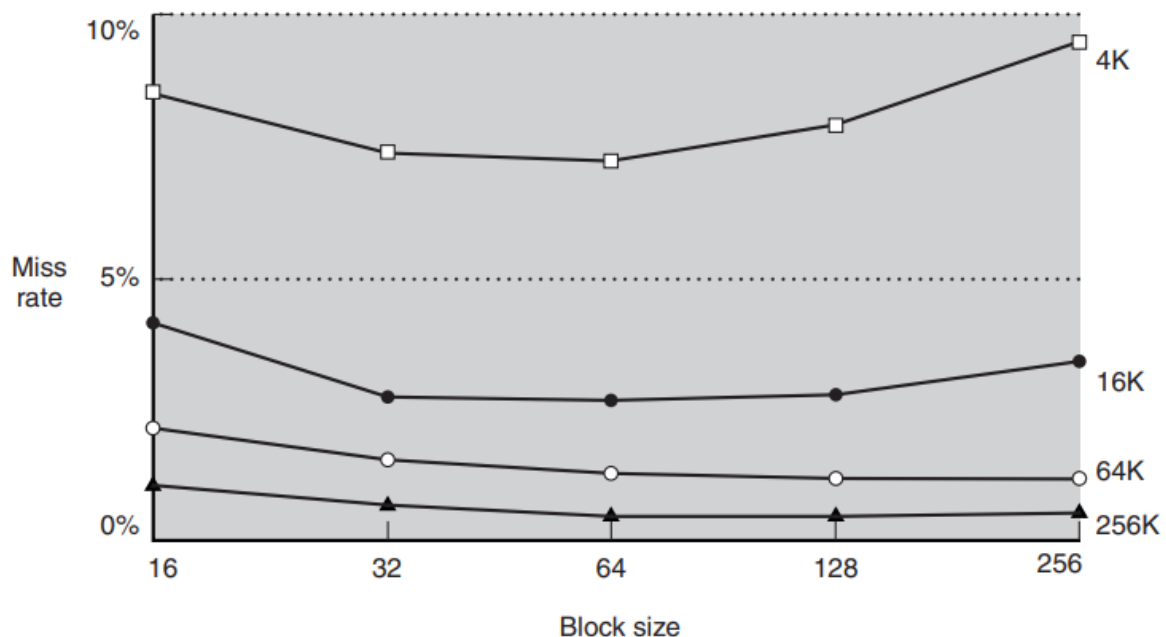
### Misses Classification: the 3Cs

- **Compulsory Misses:** accadono al primo accesso ai dati. Ineliminabili.
- **Capacity Misses:** causati da una cache troppo piccola per contenere tutti i dati richiesti. Soluzione: aumentare la dimensione della cache.
- **Conflict Misses:** accadono quando più dati competono per la stessa posizione nella cache. Soluzione: aumentare l'associatività.



## Miss Rate vs. Block Size

Il tasso di miss varia con la dimensione del blocco. Blocchi più grandi possono aumentare l'efficienza per dati sequenziali, ma se troppo grandi rispetto alla dimensione della cache, il tasso di miss può aumentare.



# Coherence Strategies (when writing)

## Write Through

Ogni scrittura nella cache aggiorna simultaneamente la memoria principale.

### Vantaggi:

- Cache e memoria sono sempre coerenti.
- Maggiore latenza dovuta agli aggiornamenti continui.

## Write Back

I dati nella memoria principale vengono aggiornati solo quando devono essere sostituiti nella cache.

### Vantaggi:

- Maggiore efficienza, riducendo gli accessi alla memoria principale.
- Cache e memoria non sono sempre coerenti.

# Virtual Memory

La memoria virtuale simula una memoria di grandi dimensioni utilizzando lo spazio su disco.

### Caratteristiche:

- **Paging:** suddivide la memoria in blocchi (pagine) per una gestione efficiente.
- **Disk-Memory Swap:** scambio dinamico di pagine tra disco e RAM.
- **Indirizzi virtuali:** tradotti in indirizzi fisici per accedere ai dati reali.